Material Suplementar: Introdução a Linguagem Python

Projeto e Análise de Experimentos

Prof. Carlos Henrique Gomes Ferreira

UFOP

decom
departamento
de computação

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

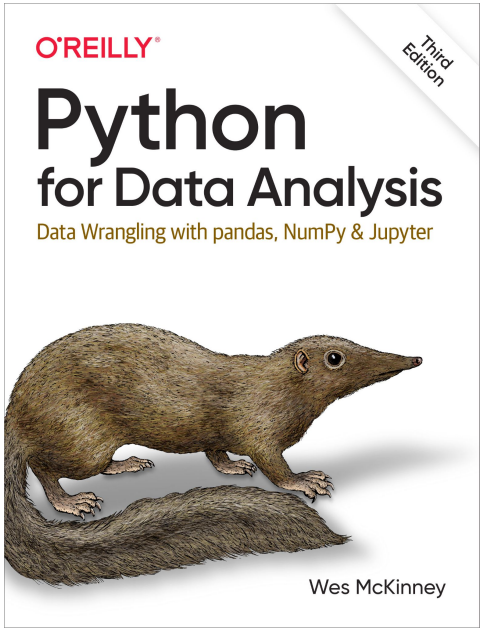
Python para Análise de Dados

Este curso usará Python para as atividades práticas

- Portanto, é fundamental ter o domínio **básico**
- Não são objetivos desta aula:
  - Exaurir todos os recursos de Python
  - Fazer tudo da forma mais otimizada possível

Referência gratuita e online

- Disponível online: https://wesmckinney.com/book/
- Errata também disponível: Link
- Disponível para vendas em vários sites: ~R$120,00



Introdução a Python

Python é uma linguagem de programação popular para análise de dados e computação científica

Tem uma comunidade bastante ativa e bibliotecas de código aberto fortes (pandas, scikit-learn) para análise de dados

Facilita a integração de códigos C, C++ e FORTRAN, sendo útil para otimizar partes críticas do código

É uma linguagem interpretada: Não possui o mesmo desempenho da linguagem compilada. Mas… Da para otimizar várias coisas

Introdução a Python

Python pode ser menos adequado para algumas aplicações devido ao *Global Interpreter Lock (GIL)* - Trava de exclusão mútua que admite apenas uma thread

- Impede que mais de uma instrução seja executada simultaneamente, limitando a execução paralela em programas *multithreaded*

Mas é um problema contornável com algumas bibliotecas

- Uso do módulo *multiprocessing* para criar processos independentes em vez de threads
    - Cada processo Python recebe seu próprio interpretador Python e espaço de memória
- Bibliotecas e extensões escritas em C ou C++ podem gerenciar suas próprias threads e executar tarefas em paralelo
- Recursos JIT (Just-In-Time) como Numba: Permitem a compilação em tempo real de partes críticas do código para melhorar o desempenho sem sair do ambiente Python

Bibliotecas/Módulos Essenciais em Python

NumPy: Base para computação numérica em Python, oferece arrays multidimensionais e operações matemáticas eficientes

Pandas: Estruturas de dados de alto nível e ferramentas para manipulação de dados tabulares, foco em análise de dados

MatplotLib: Biblioteca para criação de gráficos e visualizações de dados em duas dimensões

SciPy: Conjunto de pacotes para matemática, ciência e engenharia, estendendo as funcionalidades do NumPy

Scikit-learn: Toolkit para aprendizado de máquina, oferecendo algoritmos para classificação, regressão, agrupamento, entre outros

Bibliotecas/Módulos Essenciais em Python

Statsmodels: Pacote para análise estatística, focado em modelos estatísticos e testes econométricos

NetworkX: Biblioteca para a criação, manipulação e estudo de estruturas, dinâmicas e funções de redes

Igraph: Biblioteca eficiente para análise de redes, oferecendo ferramentas rápidas para estruturas de dados complexas e algoritmos de grafos

Bibliotecas/Módulos Essenciais em Python

Statsmodels: Pacote para análise estatística, focado em modelos estatísticos e testes econométricos

NetworkX: Biblioteca para a criação, manipulação e estudo de estruturas, dinâmicas e funções de redes

Igraph: Biblioteca eficiente para análise de redes, oferecendo ferramentas rápidas para estruturas de dados complexas e algoritmos de grafos

Qual o ambiente de execução?

Interpretador de Python

Faça download e instale a versão na sua máquina no site oficial

Python executa programas linha por linha e seu interpretador pode ser iniciado no terminal com o comando python

```
$ python
Python 3.10.4 | packaged by conda-forge | (main, Mar 24 2022, 17:38:57)
[GCC 10.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 5
>>> print(a)
5
```

Interpretador de Python

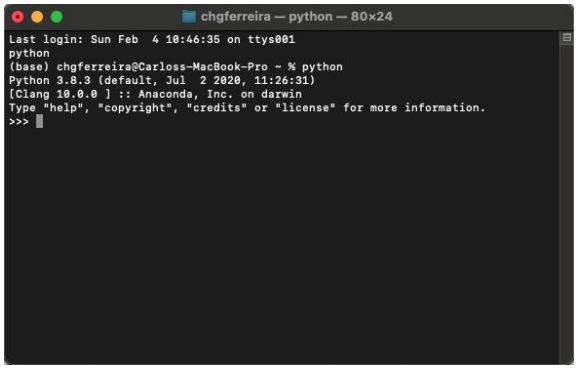
O IPython, um interpretador Python avançado, ou Jupyter notebooks, cadernos de código web do projeto IPython

```
$ ipython
Python 3.10.4 | packaged by conda-forge | (main, Mar 24 2022, 17:38:57)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.31.1 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %run hello_world.py
Hello world

In [2]:
```

Alguns Ambientes de Execução



Python Shell

PyCharm

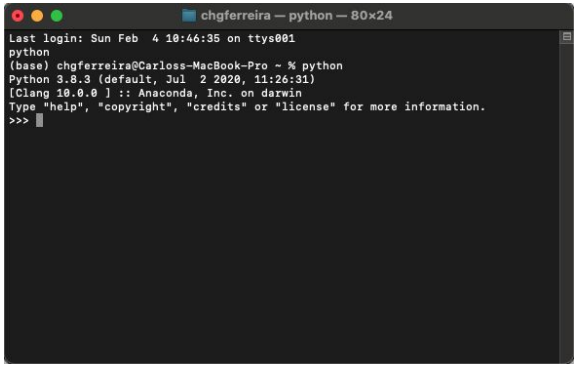
Visual Studio Code

spyder

jupyter

ATOM

Alguns Ambientes de Execução



Python Shell

PyCharm

Visual Studio Code

spyder

jupyter

ATOM

Ambiente Jupyter: Características

Permite a execução de trechos específicos do código Python, facilitando testes e experimentos incrementais

Possibilita explorar visualizações de resultados, comentários, textos em markdown e código em um único documento, facilitando a apresentação

Notebooks são salvos como .ipynb, facilitando a execução e colaboração em diferentes sistemas

Conecta-se a um processo Python em execução (kernel) para a execução do código

Instalando Python e Jupyter com Anaconda

Foco na versão Python 3.10+ juntamente com uma plataforma para o gerenciamento de pacotes - Anaconda

Anaconda: Distribuição gratuita e de código aberto do Python (e R) para computação científica e análise de dados

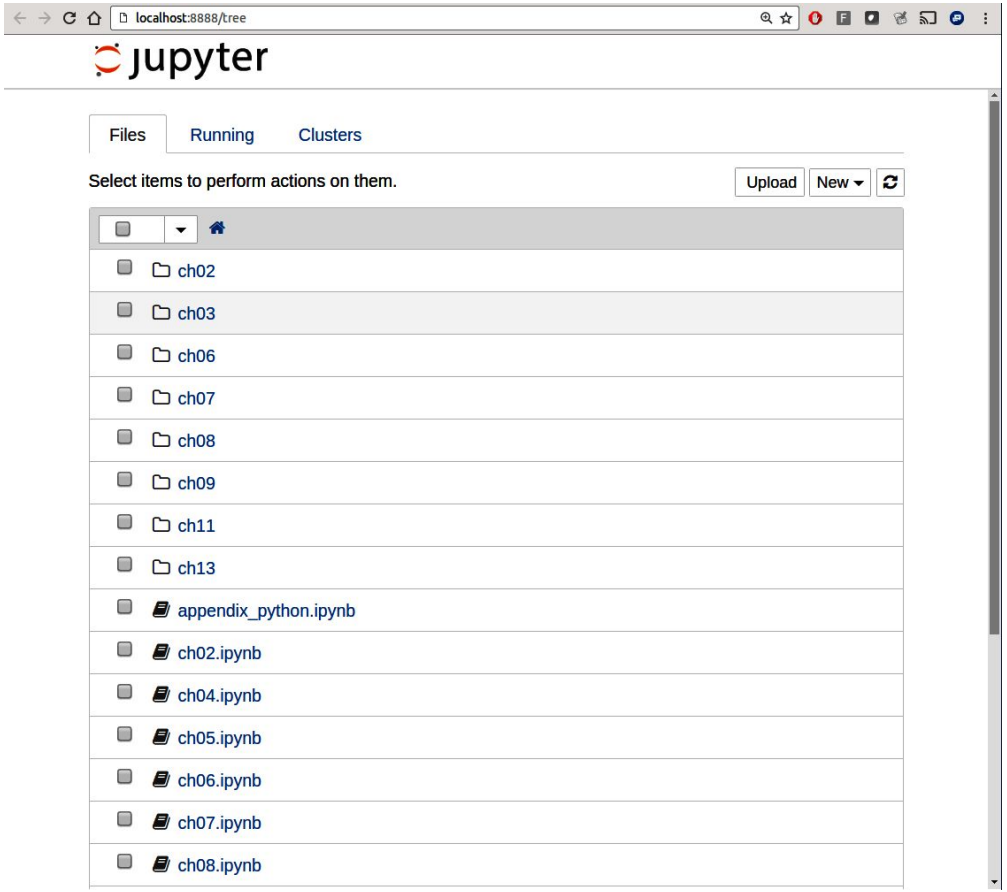
- Visa simplificar o gerenciamento de pacotes e a implantação
- Vem com uma coleção pré-instalada de bibliotecas e ferramentas para Python

Tutorial completo de instalação e configuração

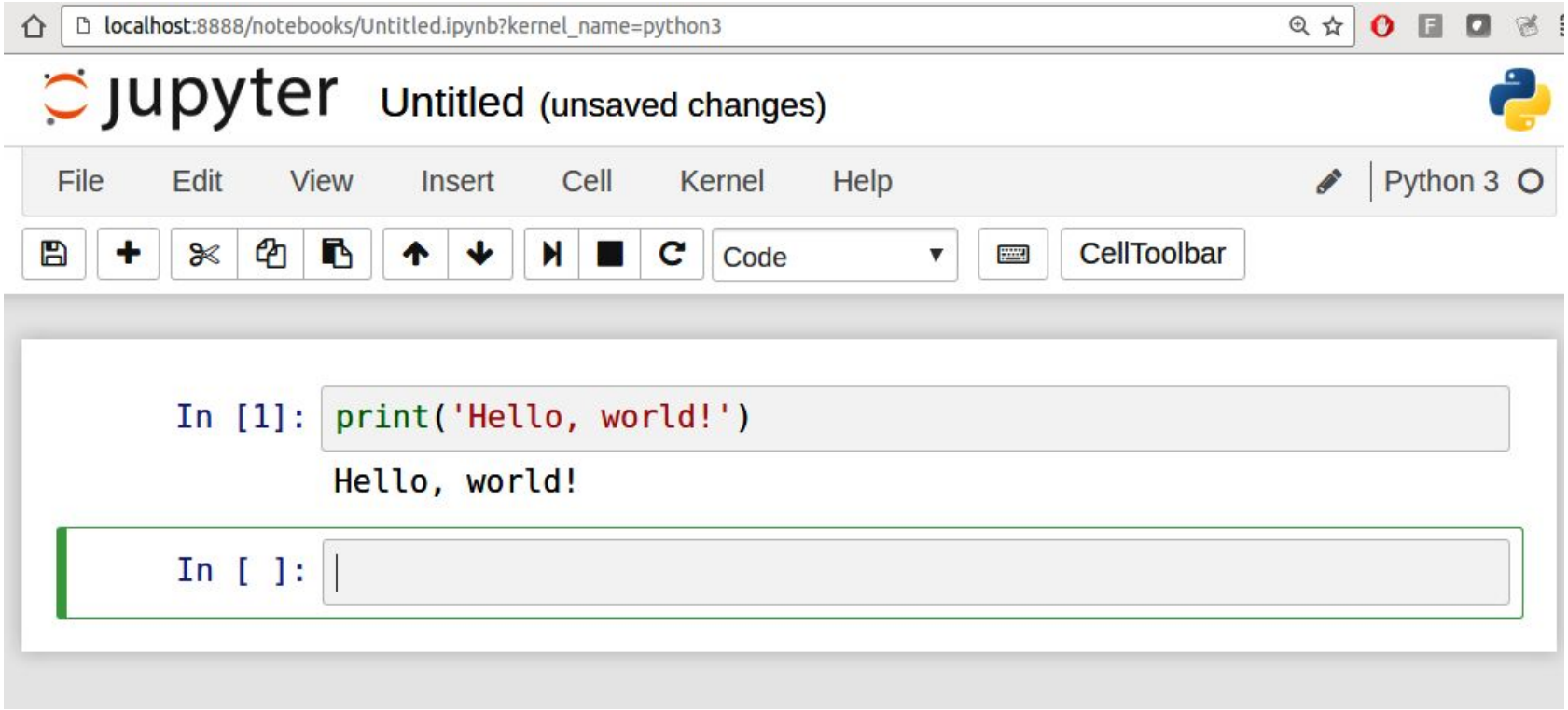
- https://docs.anaconda.com/free/anaconda/install/

Ambiente Jupyter Local

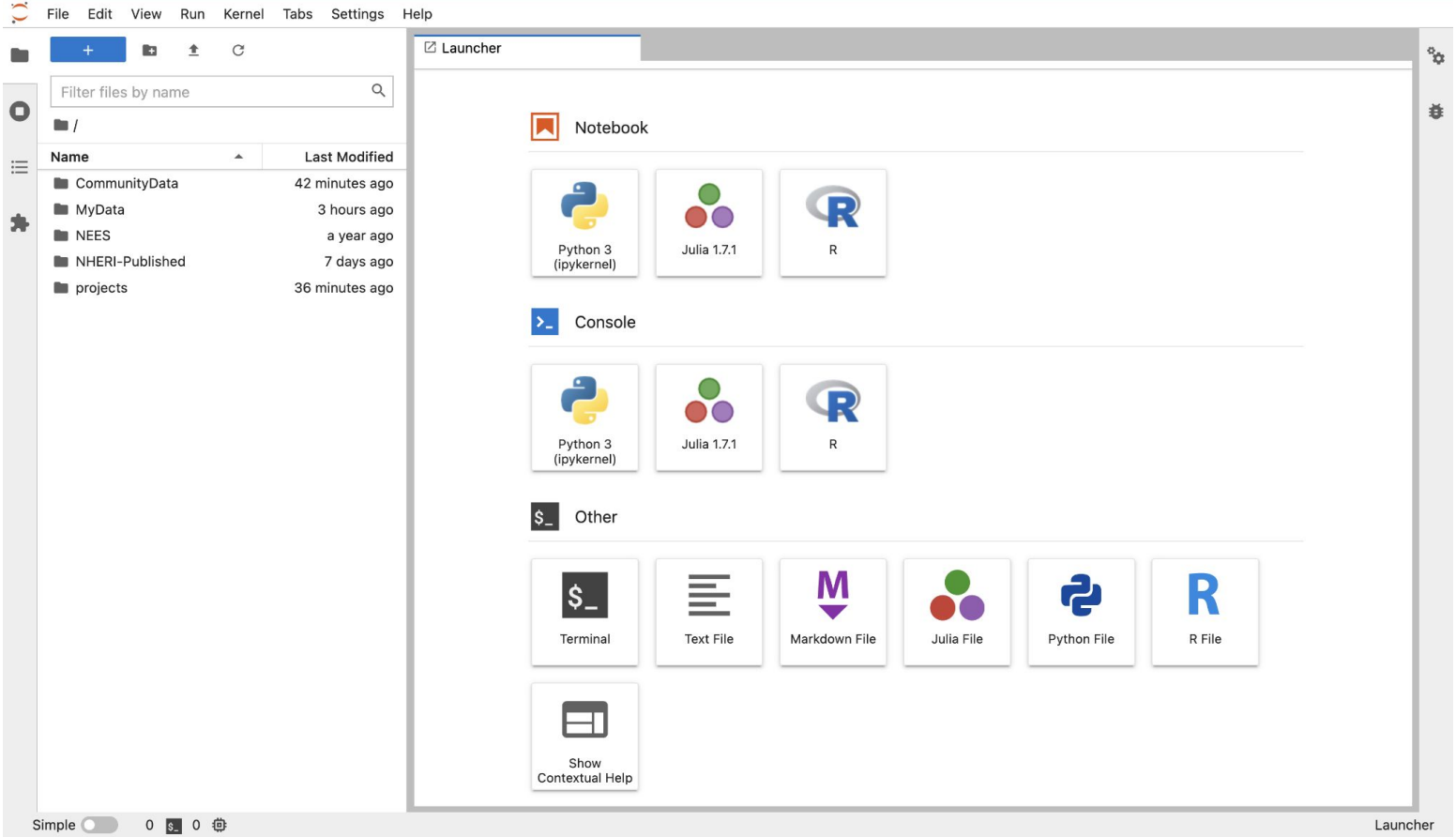
Após instalar o Anaconda, você pode usar o comando “jupyter notebook” no terminal para abrir o Jupyter



Ambiente Jupyter Local



Ambiente Jupyter Hub Local



Google Colab

Gratuito e na Nuvem: Sem configuração local, pois executa totalmente online

- Limites da Versão Gratuita: Máximo de 12 horas por sessão e restrições em GPU

Características:

- Compatível com bibliotecas como NumPy, pandas e TensorFlow
- Importação/exportação fácil com GitHub ou Google Drive
- Suporta Markdown e LaTeX para documentação
- Integração com Google Drive para acesso e armazenamento de dados

Como acessar?

1. Acesse o Google Colab: Utilize sua conta Google para entrar no Google Colaboratory

2. Carregue Notebooks: Vá em “Arquivo” > “Fazer upload do notebook` para adicionar arquivos `.ipynb`

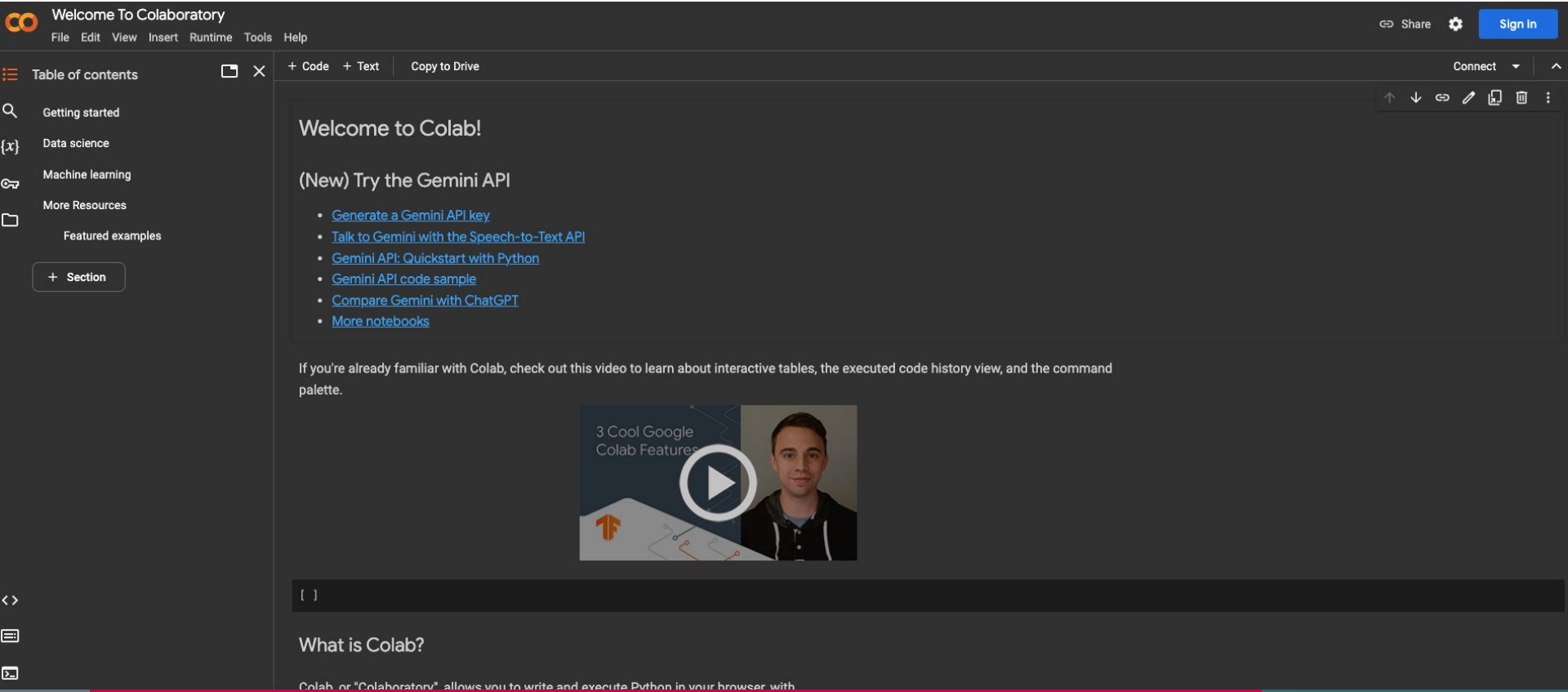
3. Trabalhe com Arquivos Python: As tarefas incluem notebooks `.ipynb` e arquivos `.py`. Use os notebooks como guia para desenvolver funções nos `.py`

4. Importe Arquivos `.py`: Após desenvolver no `.py`, importe-os no notebook com `from nome_do_modulo import nome_da_funcao`

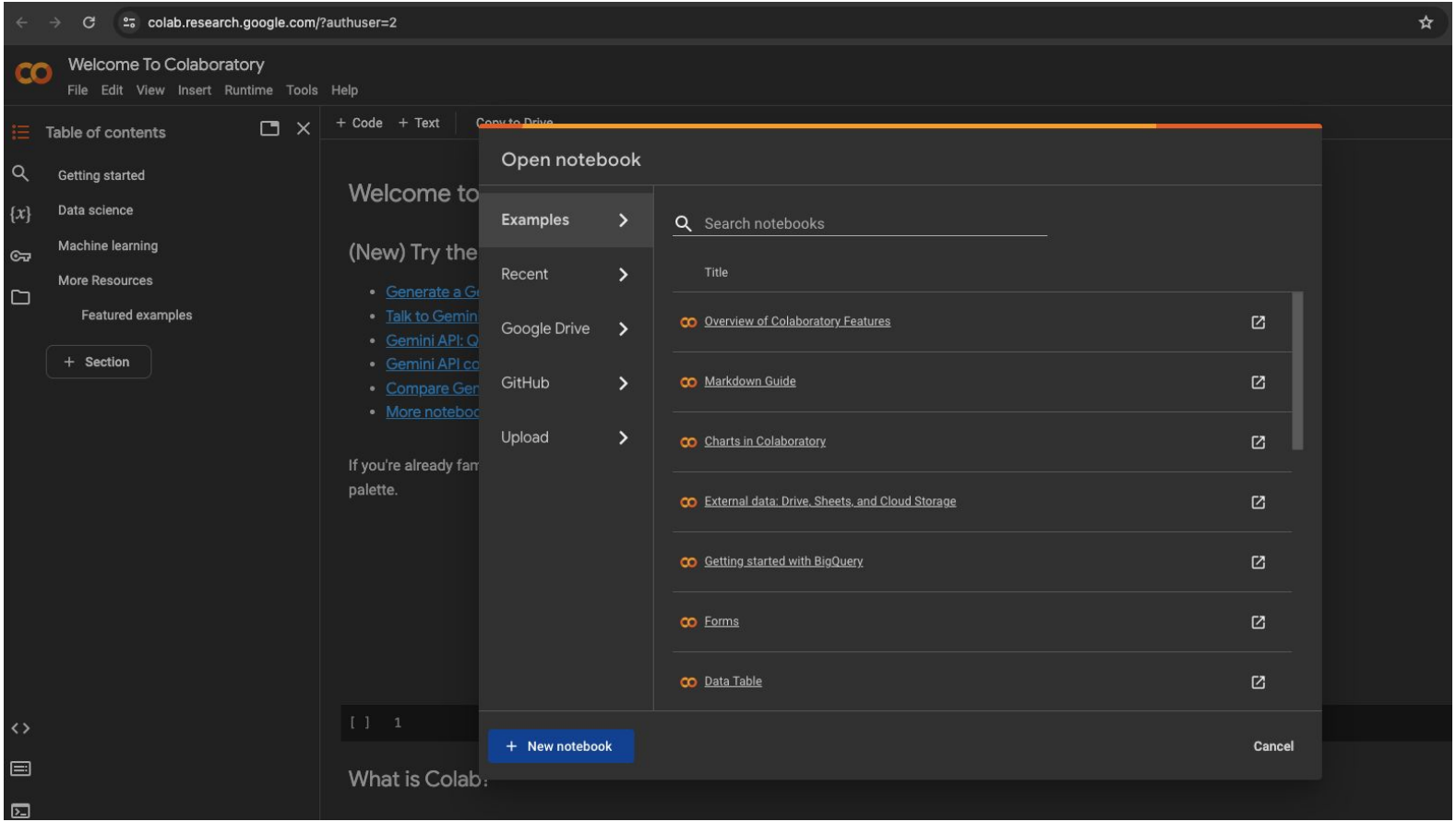
5. Edite e Teste: Edite seus `.py` no Colab e teste-os nos notebooks. Conecte-se ao Google Drive para uma edição mais fácil

Como acessar?

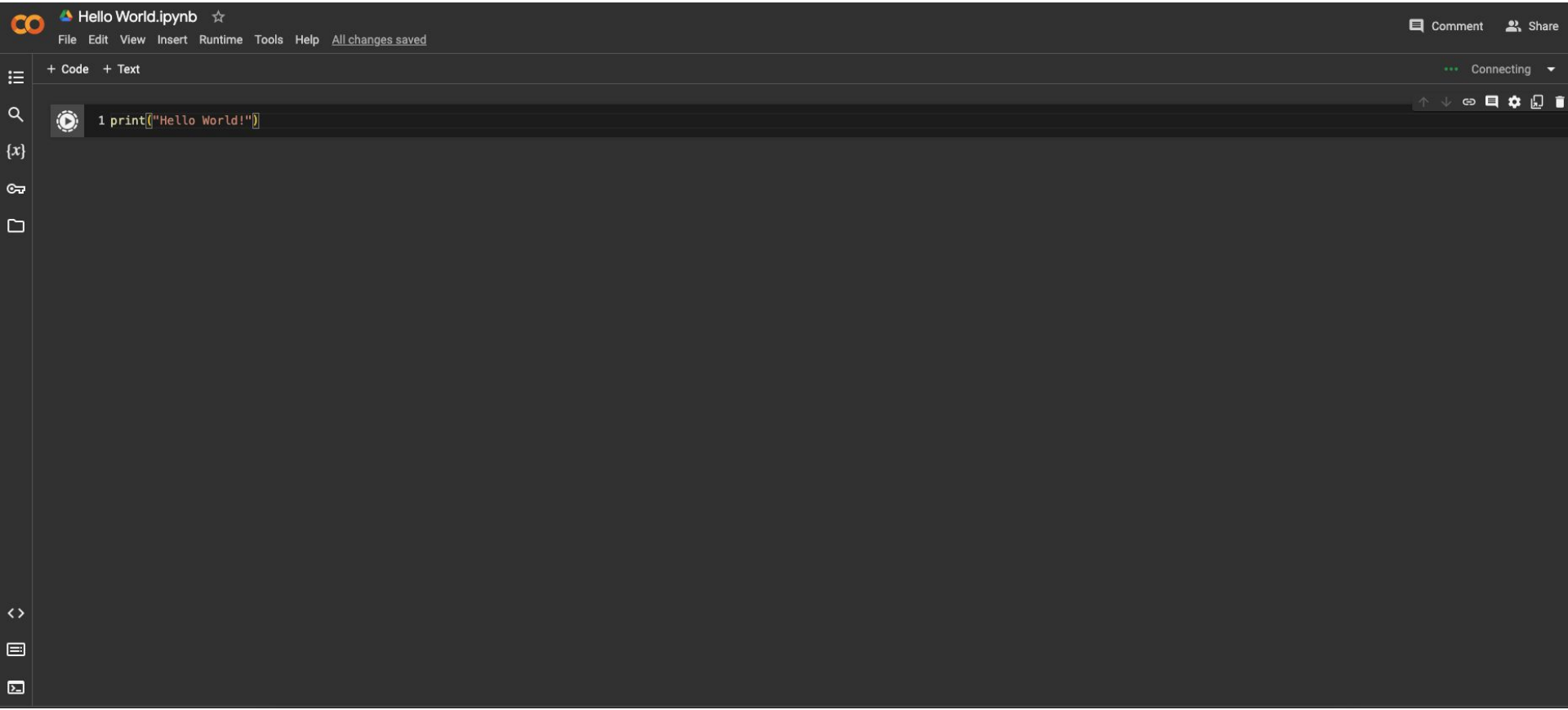
Accesse e faça login: https://colab.research.google.com/



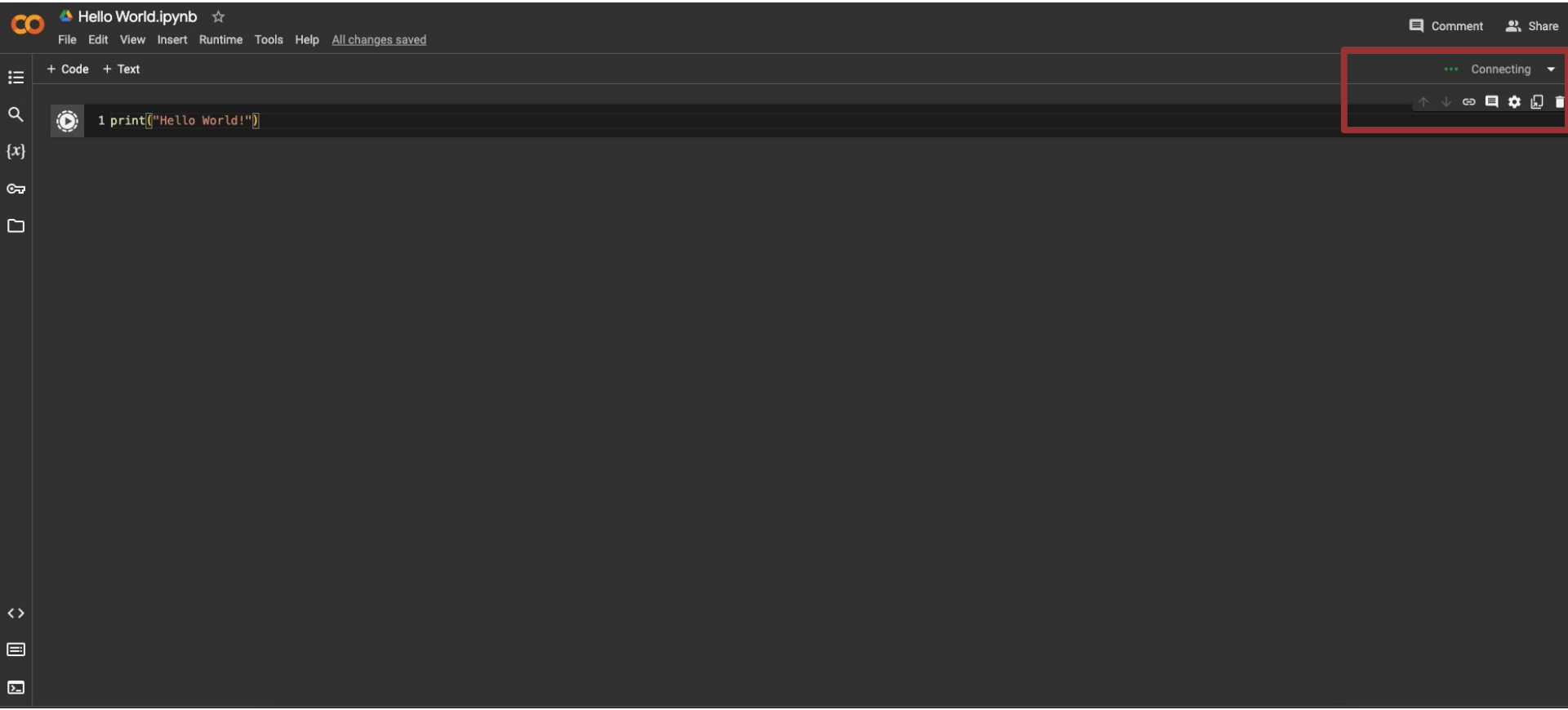
Abra ou Crie um Novo Notebook



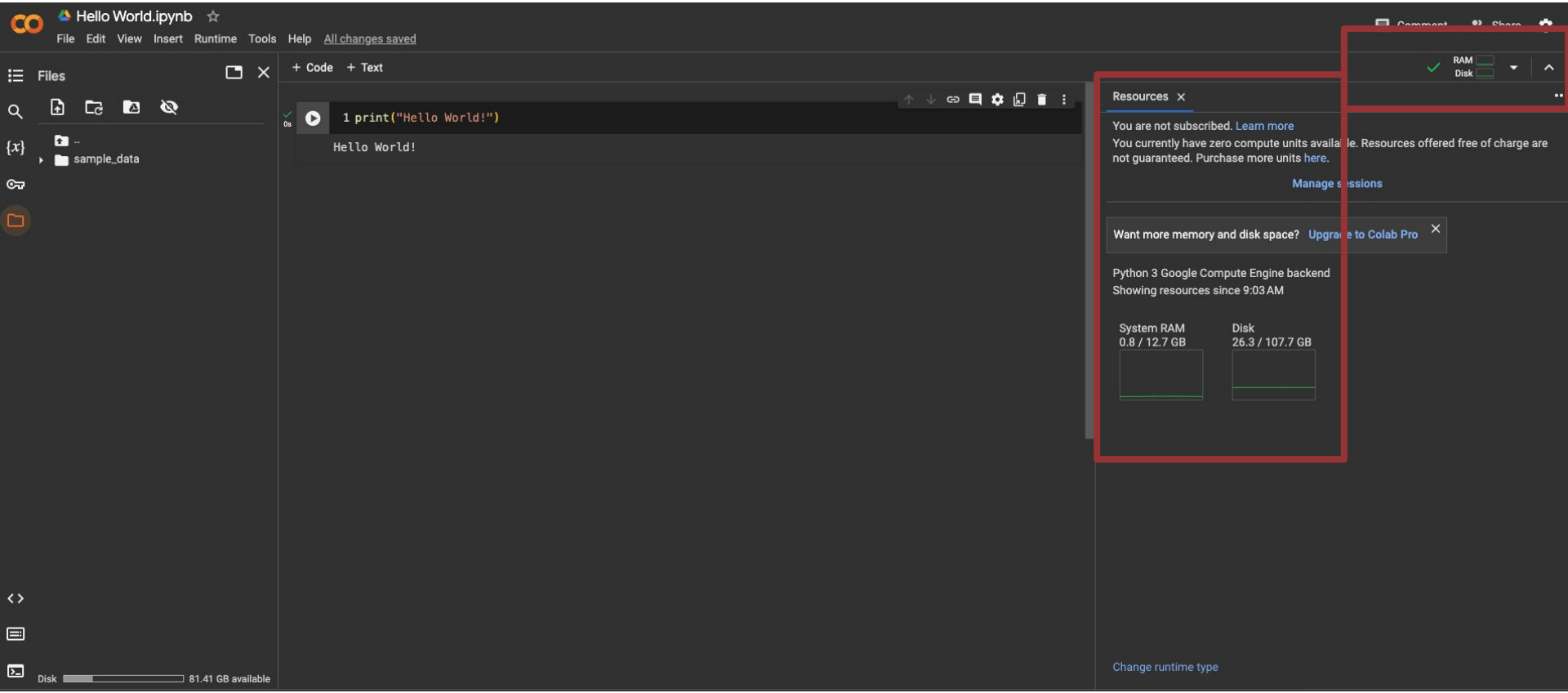
Hello World!


Hello World.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
+ Code + Text
Connecting
1 print("Hello World!")

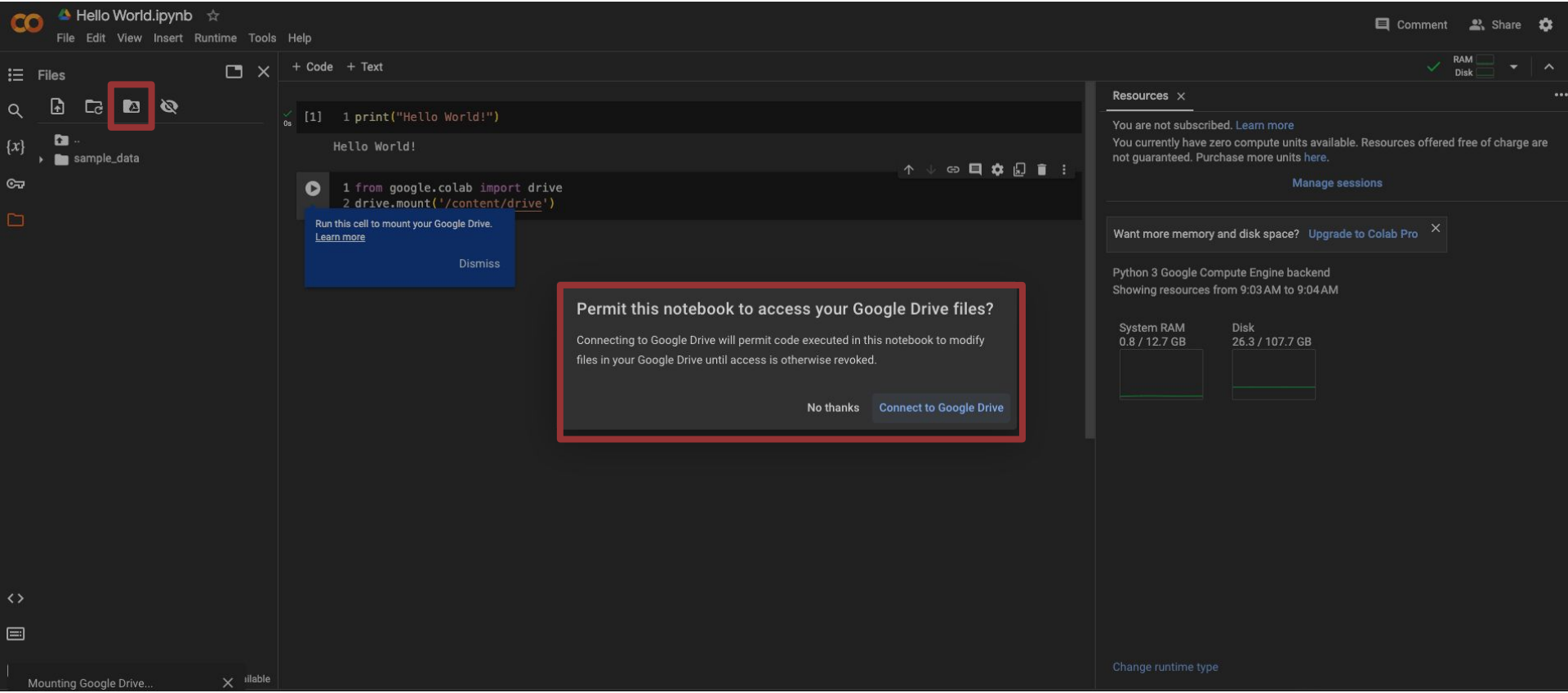
Hello World!


Hello World.ipynb
File Edit View Insert Runtime Tools Help All changes saved
Comment
Share
+ Code + Text
Connecting
1 print("Hello World!")

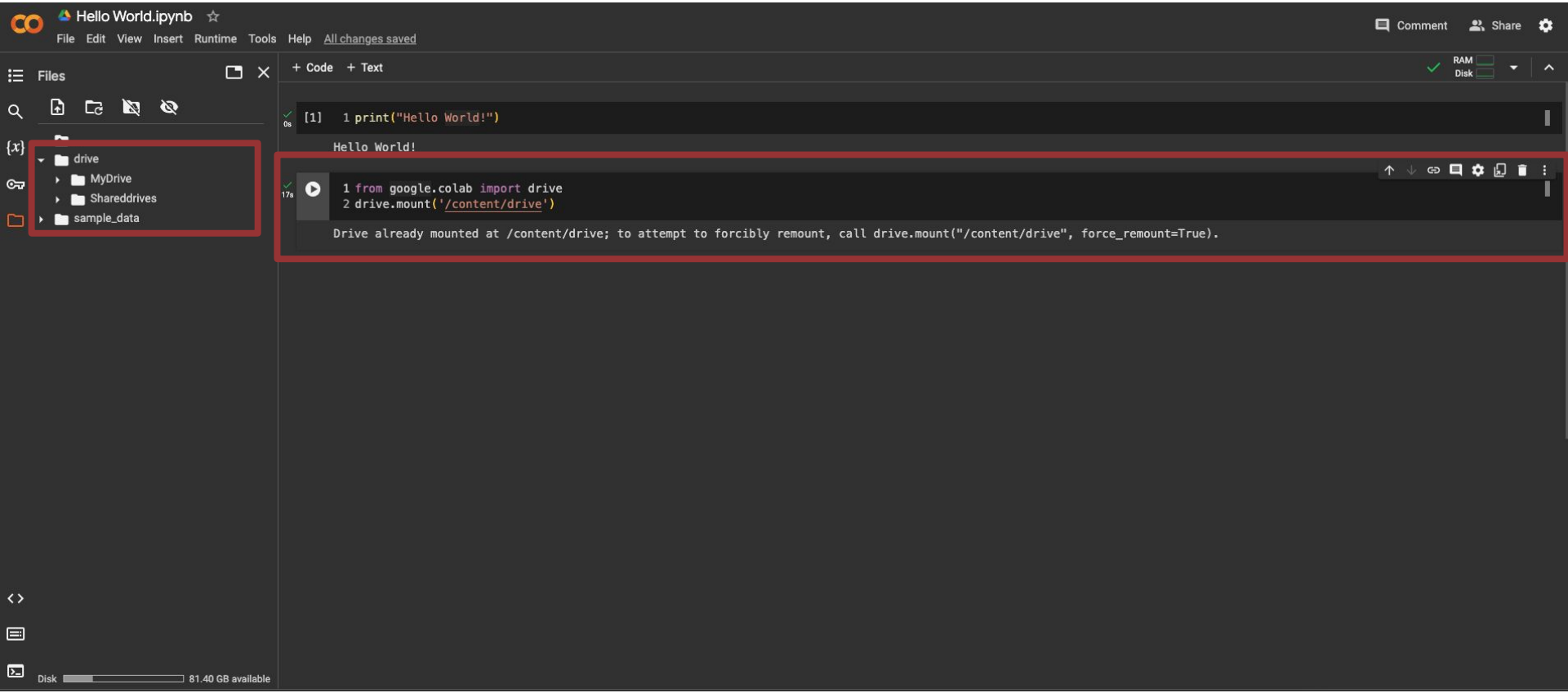
Recursos da Máquina



Conectando ao Google Drive



Conectando ao Google Drive



Conceitos Essenciais de Programação em Python

Python valoriza código fácil de ler, com **simplicidade** e **clareza**, parecendo "pseudocódigo" real

Prefere **indentação** a chaves, usando espaços para organizar o código, com **blocos marcados** por indentação e dois pontos

Não exige **ponto e vírgula** ao final das declarações, mas permite para separar declarações na mesma linha

```
for x in array:
    if x < pivot:
        less.append(x)
    else:
        greater.append(x)
```

Conceitos Essenciais de Programação em Python

Python trata todos os elementos, como números, strings, estruturas de dados, funções, classes e módulos, **como objetos**

Cada objeto possui um tipo associado (ex: inteiro, string, função) e dados internos

Esta característica torna a linguagem altamente flexível, permitindo tratamento uniforme de funções e outros objetos

Conceitos Essenciais de Programação em Python

Comentários: Utiliza o marcador # para adicionar comentários ao código, ignorados pelo interpretador Python

Exemplo:

```
results = []
for line in file_handle:
    # keep the empty lines for now
    # if len(line) == 0:
    #   continue
    results.append(line.replace("foo", "bar"))

print("Reached this line")  # Simple status report
```

Conceitos Essenciais de Programação em Python

Funções são invocadas usando parênteses e podem aceitar argumentos posicionais e de palavra-chave

```
result = f(x, y, z)
g()
```

Métodos, que são funções vinculadas a objetos, são chamados através da notação de ponto

```
obj.some_method(x, y, z)
```

Conceitos Essenciais de Programação em Python

Quase todos os objetos em Python possuem métodos, funções atreladas que acessam conteúdos internos do objeto, chamados com uma sintaxe específica

```
obj.some_method(x, y, z)
```

Funções podem aceitar tanto argumentos posicionais quanto de palavra-chave, permitindo flexibilidade na forma como os dados são passados

```
result = f(a, b, c, d=5, e="foo")
```

Conceitos Essenciais de Programação em Python

Ao atribuir uma variável em Python, você cria uma referência ao objeto à direita do sinal de igual

Em termos práticos, ao considerar uma lista de inteiros, você está referenciando diretamente essa lista, não criando uma cópia dela

```
In [8]: a = [1, 2, 3]
In [9]: b = a

In [10]: b
Out[10]: [1, 2, 3]
```

Conceitos Essenciais de Programação em Python

Ao passar objetos como argumentos para uma função, **novas variáveis locais** são criadas referenciando os objetos originais, **sem realizar cópias**

- Vincular um novo objeto a uma variável dentro de uma função não sobrescreve uma variável de mesmo nome no escopo externo à função (escopo pai)
- Assim, é possível alterar os conteúdos internos de um **argumento mutável**

```
In [13]: def append_element(some_list, element):
    ....:     some_list.append(element)
In [14]: data = [1, 2, 3]

In [15]: append_element(data, 4)

In [16]: data
Out[16]: [1, 2, 3, 4]
```

Conceitos Essenciais de Programação em Python

Variáveis são dinâmicas e não têm um tipo inerente associado a elas

- O que significa que uma variável pode se referir a objetos de tipos diferentes simplesmente realizando uma nova atribuição

```
In [17]: a = 5

In [18]: type(a)
Out[18]: int

In [19]: a = "foo"

In [20]: type(a)
Out[20]: str
```

Conceitos Essenciais de Programação em Python

Saber o tipo de um objeto é importante, especialmente quando se deseja escrever funções flexíveis que possam lidar com diferentes tipos de entrada

Para verificar se um objeto é instância de um determinado tipo, você pode usar a função “isinstance”

```
In [26]: a = 5

In [27]: isinstance(a, int)
Out[27]: True
```

Conceitos Essenciais de Programação em Python

Em Python, os objetos geralmente possuem atributos e métodos. Os atributos são outros objetos Python armazenados dentro do objeto

- Os métodos são funções associadas a um objeto que podem acessar os dados internos do objeto

```
In [1]: a = "foo"

In [2]: a.<Press Tab>
```

```
capitalize()  index()       isspace()      removesuffix()  startswith()
casefold()    isprintable()  istitle()      replace()       strip()
center()      isalnum()      isupper()      rfind()         swapcase()
count()       isalpha()      join()         rindex()        title()
encode()      isascii()      ljust()        rjust()         translate()
endswith()    isdecimal()    lower()        rpartition()
expandtabs()  isdigit()      lstrip()       rsplit()
find()        isidentifier() maketrans()    rstrip()
format()      islower()      partition()    split()
format_map()  isnumeric()    removeprefix() splitlines()
```

Entenda todos aqui: https://docs.python.org/pt-br/3/library/string.html

Importando Módulos e Bibliotecas

Um módulo é um arquivo contendo código Python, identificado pela extensão ".py"

```python
# some_module.py
PI = 3.14159

def f(x):
    return x + 2

def g(a, b):
    return a + b
```

```python
import some_module
result = some_module.f(5)
pi = some_module.PI
```

```python
from some_module import g, PI
result = g(5, PI)
```

```python
import some_module as sm
from some_module import PI as pi, g as gf

r1 = sm.f(pi)
r2 = gf(6, pi)
```

Operadores Binários e de Comparação

São bastante familiares com os das outras linguagens

Operation	Description
`a + b`	Add `a` and `b`
`a - b`	Subtract `b` from `a`
`a * b`	Multiply `a` by `b`
`a / b`	Divide `a` by `b`
`a // b`	Floor-divide `a` by `b`, dropping any fractional remainder
`a ** b`	Raise `a` to the `b` power
`a & b`	`True` if both `a` and `b` are `True`; for integers, take the bitwise `AND`
`a | b`	`True` if either `a` or `b` is `True`; for integers, take the bitwise `OR`
`a ^ b`	For Booleans, `True` if `a` or `b` is `True`, but not both; for integers, take the bitwise `EXCLUSIVE-OR`

Operadores Binários e de Comparação

São bastante familiares com os das outras linguagens

Operation	Description
`a == b`	`True` if `a` equals `b`
`a != b`	`True` if `a` is not equal to `b`
`a < b`, a <= b	`True` if `a` is less than (less than or equal to) `b`
`a > b, a >= b`	`True` if `a` is greater than (greater than or equal to) `b`
`a is b`	`True` if `a` and `b` reference the same Python object
`a is not b`	`True` if `a` and `b` reference different Python objects

Operadores Binários e de Comparação

Alguns exemplos:

```
In [37]: 5 - 7
Out[37]: -2

In [38]: 12 + 21.5
Out[38]: 33.5

In [39]: 5 <= 2
Out[39]: False
```

```
In [40]: a = [1, 2, 3]

In [41]: b = a

In [42]: c = list(a)

In [43]: a is b
Out[43]: True

In [44]: a is not c
Out[44]: True
```

Objetos Mutáveis e Não Mutáveis

Muitos objetos em Python, como listas, dicionários, arrays NumPy e a maioria dos tipos definidos por usuários (classes), são mutáveis

- Significa que o objeto ou valores que contêm podem ser modificados

```
In [48]: a_list = ["foo", 2, [4, 5]]

In [49]: a_list[2] = (3, 4)

In [50]: a_list
Out[50]: ['foo', 2, (3, 4)]
```

Objetos Mutáveis e Não Mutáveis

Outros, como strings e tuplas, são imutáveis, o que significa que seus dados internos não podem ser alterados

```
In [48]: a_list = ["foo", 2, [4, 5]]

In [49]: a_list[2] = (3, 4)

In [50]: a_list
Out[50]: ['foo', 2, (3, 4)]
```

```
In [51]: a_tuple = (3, 5, (4, 5))

In [52]: a_tuple[1] = "four"
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-52-cd2a018a7529> in <module>
----> 1 a_tuple[1] = "four"
TypeError: 'tuple' object does not support item assignment
```

Escalares

Python possui um conjunto reduzido de tipos nativos para manipular dados numéricos, strings, valores Booleanos, etc

- Esses tipos de “valor único” são às vezes chamados de tipos escalares

Type	Description
`None`	The Python “null” value (only one instance of the `None` object exists)
`str`	String type; holds Unicode strings
`bytes`	Raw binary data
`float`	Double-precision floating-point number (note there is no separate `double` type)
`bool`	A Boolean `True` or `False` value
`int`	Arbitrary precision integer

Strings

Muitas pessoas usam Python por suas capacidades integradas de manipulação de strings

Você pode escrever literais de string usando aspas simples ' ou aspas duplas "(aspas duplas são geralmente preferidas)

Para strings multilinhas com quebras de linha, você pode usar aspas triplas, seja ''' ou """

```
a = 'one way of writing a string'
b = "another way"
```

```
c = """
This is a longer string that
spans multiple lines
"""
```

Strings

Quando uma string contém o mesmo tipo de aspas usado para delimitá-la, essas aspas internas devem ser escapadas com uma barra invertida (\)

- Ou, alternativamente, usar o outro tipo de aspa para delimitar a string

Adicionar duas strings juntas as concatena e produz uma nova string

```
In [76]: a = "this is the first half "

In [77]: b = "and this is the second half"

In [78]: a + b
Out[78]: 'this is the first half and this is the second half'
```

Strings

Os objetos de string possuem um método format que pode ser usado para substituir argumentos formatados na string, produzindo uma nova string

Para substituir argumentos por esses parâmetros de formatação, passamos uma sequência de argumentos para o método format

```
In [79]: template = "{0:.2f} {1:s} are worth US${2:d}"
```

Strings

Os objetos de string possuem um método format que pode ser usado para substituir argumentos formatados na string, produzindo uma nova string

Para substituir argumentos por esses parâmetros de formatação, passamos uma sequência de argumentos para o método format

```
In [79]: template = "{0:.2f} {1:s} are worth US${2:d}"
```

Significa formatar o primeiro argumento [0] como um número de ponto flutuante com duas casas decimais

Strings

Os objetos de string possuem um método format que pode ser usado para substituir argumentos formatados na string, produzindo uma nova string

Para substituir argumentos por esses parâmetros de formatação, passamos uma sequência de argumentos para o método format

```
In [79]: template = "{0:.2f} {1:s} are worth US${2:d}"
```



Significa formatar o segundo argumento[1] como uma String

Strings

Os objetos de string possuem um método format que pode ser usado para substituir argumentos formatados na string, produzindo uma nova string

Para substituir argumentos por esses parâmetros de formatação, passamos uma sequência de argumentos para o método format

```
In [79]: template = "{0:.2f} {1:s} are worth US${2:d}"
```



Significa formatar o terceiro argumento [2] como um inteiro exato

Strings

Os objetos de string possuem um método format que pode ser usado para substituir argumentos formatados na string, produzindo uma nova string

Para substituir argumentos por esses parâmetros de formatação, passamos uma sequência de argumentos para o método format

```
In [79]: template = "{0:.2f} {1:s} are worth US${2:d}"

In [80]: template.format(88.46, "Argentine Pesos", 1)
Out[80]: '88.46 Argentine Pesos are worth US$1'
```

Strings

O Python 3.6 introduziu uma nova funcionalidade chamada f-strings (abreviação de formatted string literals)

- Torna a criação de strings formatadas ainda mais conveniente

Para criar uma f-string:

1. Escreva o caractere “f” imediatamente antes de um literal
2. Dentro da string, inclua expressões Python entre chaves para substituir o valor da expressão na string formatada. Formatos também podem ser adicionados

```
In [81]: amount = 10

In [82]: rate = 88.46

In [83]: currency = "Pesos"

In [84]: result = f"{amount} {currency} is worth US${amount / rate}"
```

```
In [85]: f"{amount} {currency} is worth US${amount / rate:.2f}"
Out[85]: '10 Pesos is worth US$0.11'
```

Booleanos

Os dois valores booleanos no Python são escritos como True (Verdadeiro) e False (Falso)

- Valores booleanos são combinados com as palavras-chave and (e) e or (ou)
- Se convertidos para inteiro, são representados por 0(False) e 1(True)

```
In [95]: True and True
Out[95]: True

In [96]: False or True
Out[96]: True
```

```
In [97]: int(False)
Out[97]: 0

In [98]: int(True)
Out[98]: 1
```

None

None é o tipo de valor nulo em Python, utilizado para representar a ausência de valor

```
In [109]: a = None

In [110]: a is None
Out[110]: True
```

```
In [111]: b = 5

In [112]: b is not None
Out[112]: True
```

Data e tempo

O módulo “datetime” do Python fornece os tipos “datetime”, “date” e “time”

- O tipo “datetime” combina as informações armazenadas em “date” (data) e “time” (hora)

```
In [113]: from datetime import datetime, date, time

In [114]: dt = datetime(2011, 10, 29, 20, 30, 21)

In [115]: dt.day
Out[115]: 29

In [116]: dt.minute
Out[116]: 30
```

Data e tempo

Dada uma instância de “datetime”, você pode extrair os objetos equivalentes de “date” e “time” chamando métodos no datetime com o mesmo nome

```
In [117]: dt.date()
Out[117]: datetime.date(2011, 10, 29)

In [118]: dt.time()
Out[118]: datetime.time(20, 30, 21)
```

O método “strftime” formata um “datetime” como uma string

```
In [119]: dt.strftime("%Y-%m-%d %H:%M")
Out[119]: '2011-10-29 20:30'
```

Data e tempo

Strings podem ser convertidas (analisadas) em objetos “datetime” com a função “strptime”

```
In [120]: datetime.strptime("20091031", "%Y%m%d")
Out[120]: datetime.datetime(2009, 10, 31, 0, 0)
```

As vezes é útil substituir um dado valor de um “datetime”

```
In [121]: dt_hour = dt.replace(minute=0, second=0)

In [122]: dt_hour
Out[122]: datetime.datetime(2011, 10, 29, 20, 0)
```

Data e tempo

Como “datetime.datetime” é um tipo imutável, métodos como esses sempre produzem novos objetos

- Portanto, no exemplo anterior, dt não é modificado por “replace”

```
In [123]: dt
Out[123]: datetime.datetime(2011, 10, 29, 20, 30, 21)
```

A diferença entre dois objetos datetime produz um tipo “datetime.timedelta”

```
In [124]: dt2 = datetime(2011, 11, 15, 22, 30)

In [125]: delta = dt2 - dt

In [126]: delta
Out[126]: datetime.timedelta(days=17, seconds=7179)

In [127]: type(delta)
Out[127]: datetime.timedelta
```

Data e tempo

Adicionar um “timedelta” a um “datetime” produz um novo “datetime” deslocado

```
In [128]: dt
Out[128]: datetime.datetime(2011, 10, 29, 20, 30, 21)

In [129]: dt + delta
Out[129]: datetime.datetime(2011, 11, 15, 22, 30)
```

Operações com “timedelta” são bastante úteis ao se trabalhar com dados temporais

Controle de Fluxo: if, elif, and else

Python possui várias instruções para lógica condicional, laços e outros conceitos padrão de controle de fluxo encontrados

- Nada muito diferente das outras linguagens

“if”: Verifica uma condição que, se verdadeira, executa um bloco de código

```
x = -5
if x < 0:
    print("It's negative")
```

Controle de Fluxo: if, elif, and else

Uma instrução “if” pode ser seguida opcionalmente por um ou mais blocos "elif"

- E um bloco "else" caso todas as condições anteriores forem Falsas

```
if x < 0:
    print("It's negative")
elif x == 0:
    print("Equal to zero")
elif 0 < x < 5:
    print("Positive but smaller than 5")
else:
    print("Positive and larger than or equal to 5")
```

Controle de Fluxo: for

Laços “for” são usados para iterar sobre uma coleção (como uma lista ou tupla) ou um iterador

```
for value in collection:
    # do something with value
```

Você pode avançar um laço “for” para a próxima iteração, pulando o restante do bloco, usando a palavra-chave “continue”

```
sequence = [1, 2, None, 4, None, 5]
total = 0
for value in sequence:
    if value is None:
        continue
    total += value
```

Controle de Fluxo: for

Um laço “for” pode ser encerrado completamente com a palavra-chave “break”

```
sequence = [1, 2, 0, 4, 6, 5, 2, 1]
total_until_5 = 0
for value in sequence:
    if value == 5:
        break
    total_until_5 += value
```

Caso o código acima tivesse um laço “for” externo, ele não seria encerrado

- A palavra-chave “break” termina apenas o laço “for” mais interno

Controle de Fluxo: while

Um laço “while” especifica uma condição em que um bloco de código deve ser executado em dois casos:

- Até que a condição seja avaliada como Falsa
- Até que o laço seja explicitamente encerrado com “break”

```
x = 256
total = 0
while x > 0:
    if total > 500:
        break
    total += x
    x = x // 2
```

Controle de Fluxo: pass

“pass” é a instrução de “nop” (ou "não fazer nada") em Python

Pode ser usada em blocos onde nenhuma ação deve ser tomada (ou como um espaço reservado para código ainda não implementado)

- É necessária apenas porque Python usa espaços em branco para delimitar blocos

```
if x < 0:
    print("negative!")
elif x == 0:
    # TODO: put something smart here
    pass
else:
    print("positive!")
```

Controle de Fluxo: range

A função “range” gera uma sequência de inteiros igualmente espaçados

```
In [135]: range(10)
Out[135]: range(0, 10)
```

Que pode ser transformada em uma lista

```
In [136]: list(range(10))
Out[136]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

E por de ter um início, fim e passo (que pode ser negativo)

```
In [137]: list(range(0, 20, 2))
Out[137]: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

In [138]: list(range(5, 0, -1))
Out[138]: [5, 4, 3, 2, 1]
```

Controle de Fluxo: range

“range” produz inteiros até, mas não incluindo, o número final

Um uso comum do range é para iterar através de sequências pelo índice

```
In [139]: seq = [1, 2, 3, 4]

In [140]: for i in range(len(seq)):
   .....:     print(f"element {i}: {seq[i]}")
element 0: 1
element 1: 2
element 2: 3
element 3: 4
```

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

Estrutura de Dados

Estruturas de dados amplamente usadas em Python:

- Tupla
- Lista
- Dicionário
- Conjunto (set)

Estrutura de Dados

Estruturas de dados amplamente usadas em Python:

- Tupla
- Lista
- Dicionário
- Conjunto (set)

Vamos passar rapidamente pelos principais aspectos delas!

Estrutura de Dados: Tupla

Uma tupla é uma sequência **imutável** de objetos Python de comprimento fixo, que, uma vez atribuída, não pode ser alterada

A maneira mais fácil de criar uma é com uma sequência de valores separados por vírgulas envolvida em parênteses

```
In [2]: tup = (4, 5, 6)

In [3]: tup
Out[3]: (4, 5, 6)
```

Em muitos contextos, os parênteses podem ser omitidos

```
In [4]: tup = 4, 5, 6

In [5]: tup
Out[5]: (4, 5, 6)
```

Estrutura de Dados: Tupla

Você pode converter qualquer sequência ou iterador em uma tupla invocando tuple

```
In [6]: tuple([4, 0, 2])
Out[6]: (4, 0, 2)

In [7]: tup = tuple('string')

In [8]: tup
Out[8]: ('s', 't', 'r', 'i', 'n', 'g')
```

Os elementos podem ser acessados com colchetes e são indexadas a partir de 0 em Python

```
In [9]: tup[0]
Out[9]: 's'
```

Estrutura de Dados: Tupla

Ao definir tuplas em expressões complexas, é comum usar parênteses, como ao criar uma tupla de tuplas

```
In [10]: nested_tup = (4, 5, 6), (7, 8)

In [11]: nested_tup
Out[11]: ((4, 5, 6), (7, 8))

In [12]: nested_tup[0]
Out[12]: (4, 5, 6)

In [13]: nested_tup[1]
Out[13]: (7, 8)
```

Estrutura de Dados: Tupla

Para concatenar tuplas, use o operador +

```
In [18]: (4, None, 'foo') + (6, 0) + ('bar',)
Out[18]: (4, None, 'foo', 6, 0, 'bar')
```

Multiplicar uma tupla por um inteiro, assim como com listas, tem o efeito de concatenar aquela quantidade de cópias da tupla

```
In [19]: ('foo', 'bar') * 4
Out[19]: ('foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'bar')
```

Estrutura de Dados: Tupla

Ao tentar atribuir valores a variáveis em formato de tupla, o Python buscará desempacotar o valor à direita do sinal de igualdade

```
In [20]: tup = (4, 5, 6)

In [21]: a, b, c = tup

In [22]: b
Out[22]: 5
```

Mesmo sequências com tuplas aninhadas podem ser desempacotadas

```
In [23]: tup = 4, 5, (6, 7)

In [24]: a, b, (c, d) = tup

In [25]: d
Out[25]: 7
```

Estrutura de Dados: Tupla

Um uso comum do desempacotamento de variáveis é iterar sobre sequências de tuplas ou listas

```
In [32]: seq = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]

In [33]: for a, b, c in seq:
   ....:     print(f'a={a}, b={b}, c={c}')
a=1, b=2, c=3
a=4, b=5, c=6
a=7, b=8, c=9
```

Estrutura de Dados: Tupla

Em certos casos, pode ser útil extrair elementos específicos do começo de uma tupla, para o qual existe uma sintaxe especial: “*rest”

- Utilizada também em assinaturas de funções para recolher uma quantidade indefinida de argumentos posicionais

```
In [34]: values = 1, 2, 3, 4, 5

In [35]: a, b, *rest = values

In [36]: a
Out[36]: 1

In [37]: b
Out[37]: 2

In [38]: rest
Out[38]: [3, 4, 5]
```

Por uma questão de convenção, muitos programadores Python usarão o sublinhado (_) para variáveis indesejadas

Estrutura de Dados: Listas

Em contraste com as tuplas, listas têm comprimento variável e seus conteúdos podem ser modificados diretamente

Listas são **mutáveis** e você pode defini-las usando colchetes [] ou utilizando a função de tipo list

```
In [42]: a_list = [2, 3, 7, None]

In [43]: tup = ("foo", "bar", "baz")

In [44]: b_list = list(tup)
```

```
In [45]: b_list
Out[45]: ['foo', 'bar', 'baz']

In [46]: b_list[1] = "peekaboo"

In [47]: b_list
Out[47]: ['foo', 'peekaboo', 'baz']
```

Estrutura de Dados: Listas

Listas e tuplas possuem semelhanças semânticas (exceto que tuplas são imutáveis) e podem ser usadas de forma intercambiável em muitos contextos

A função “list” é frequentemente utilizada no processamento de dados para transformar um iterador ou expressão geradora em uma lista concreta

```
In [48]: gen = range(10)

In [49]: gen
Out[49]: range(0, 10)

In [50]: list(gen)
Out[50]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Estrutura de Dados: Listas

Use “append” para adicionar elementos ao final da lista

```
In [51]: b_list.append("dwarf")

In [52]: b_list
Out[52]: ['foo', 'peekaboo', 'baz', 'dwarf']
```

Use “insert” para adicionar um elemento em uma posição específica

```
In [53]: b_list.insert(1, "red")

In [54]: b_list
Out[54]: ['foo', 'red', 'peekaboo', 'baz', 'dwarf']
```

Estrutura de Dados: Listas

Use “append” para inserir elementos ao final da lista

```
In [51]: b_list.append("dwarf")

In [52]: b_list
Out[52]: ['foo', 'peekaboo', 'baz', 'dwarf']
```

Use “insert” para inserir um elemento em uma posição específica entre 0 e n-1

```
In [53]: b_list.insert(1, "red")

In [54]: b_list
Out[54]: ['foo', 'red', 'peekaboo', 'baz', 'dwarf']
```

A operação “insert” é computacionalmente mais custosa em comparação com “append”

- As referências aos elementos subsequentes precisam ser deslocadas internamente para criar espaço para o novo elemento

Estrutura de Dados: Listas

Use “pop” para remover e retornar um elemento de um índice específico

```
In [55]: b_list.pop(2)
Out[55]: 'peekaboo'

In [56]: b_list
Out[56]: ['foo', 'red', 'baz', 'dwarf']
```

Use “remove” para excluir a primeira ocorrência de elemento pelo valor

```
In [57]: b_list.append("foo")

In [58]: b_list
Out[58]: ['foo', 'red', 'baz', 'dwarf', 'foo']

In [59]: b_list.remove("foo")

In [60]: b_list
Out[60]: ['red', 'baz', 'dwarf', 'foo']
```

Estrutura de Dados: Listas

Use a palavra-chave “in” para verificar se um valor está na lista:

```
In [61]: "dwarf" in b_list
Out[61]: True
```

Use “not in” para verificar se um valor não está na lista:

```
In [62]: "dwarf" not in b_list
Out[62]: False
```

Possui custo relativamente alto: Linear O(n)

- Pode ser feito de forma muito eficiência O(1) com conjuntos (“set”) e dicionários (“dict”)

Estrutura de Dados: Listas

Similar às tuplas, adicionar duas listas com + as concatena

```
In [63]: [4, None, "foo"] + [7, 8, (2, 3)]
Out[63]: [4, None, 'foo', 7, 8, (2, 3)]
```

Se você já tem uma lista definida, pode adicionar múltiplos elementos com o método “extend”

```
In [64]: x = [4, None, "foo"]

In [65]: x.extend([7, 8, (2, 3)])

In [66]: x
Out[66]: [4, None, 'foo', 7, 8, (2, 3)]
```

Porém, a concatenação de listas com o operador de adição é custosa, pois cria uma nova lista e copia os elementos

- Usar o método “extend” para adicionar elementos a uma lista existente é mais eficiente, especialmente ao construir listas grandes

Estrutura de Dados: Listas

Você pode ordenar uma lista diretamente (sem criar um novo objeto) chamando o método “sort”

```
In [67]: a = [7, 2, 5, 1, 3]

In [68]: a.sort()

In [69]: a
Out[69]: [1, 2, 3, 5, 7]
```

A função sort permite a ordenação de elementos com base em uma chave secundária, que é uma função que determina o valor usado para a ordenação

```
In [70]: b = ["saw", "small", "He", "foxes", "six"]

In [71]: b.sort(key=len)

In [72]: b
Out[72]: ['He', 'saw', 'six', 'small', 'foxes']
```

Estrutura de Dados: Listas

Você pode selecionar partes de sequências (como listas, strings) usando a notação de fatiamento

- Especifique o início e fim com a sintaxe "início:fim" dentro dos colchetes []
- O elemento no índice de início é incluído no fatiamento, mas o índice de parada não

```
In [73]: seq = [7, 2, 3, 7, 5, 6, 0, 1]

In [74]: seq[1:5]
Out[74]: [2, 3, 7, 5]
```

Fatiamentos também podem ser atribuídos com uma sequência:

```
In [75]: seq[3:5] = [6, 3]

In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

Estrutura de Dados: Listas

O início (parada) pode ser omitido e passar assumir o início (fim) da lista

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

Estrutura de Dados: Listas

O início (parada) pode ser omitido e passar assumir o início (fim) da lista

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

Estrutura de Dados: Listas

O início (parada) pode ser omitido e passar assumir o início (fim) da lista

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

Estrutura de Dados: Listas

O início (parada) pode ser omitido e passar assumir o início (fim) da lista

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

Os índices negativos cortam a sequência em relação ao final

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [79]: seq[-4:]
Out[79]: [3, 6, 0, 1]
```

Estrutura de Dados: Listas

O início (parada) pode ser omitido e passar assumir o início (fim) da lista

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 6, 0, 1]
```

Os índices negativos cortam a sequência em relação ao final

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]
```

```
In [79]: seq[-4:]
Out[79]: [3, 6, 0, 1]

In [80]: seq[-6:-2]
Out[80]: [3, 6, 3, 6]
```

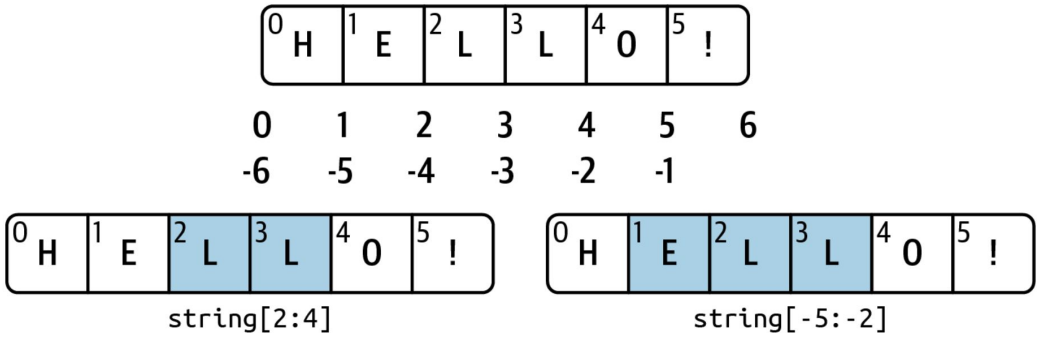
Estrutura de Dados: Listas

Um “passo” pode ser adicionado após um segundo dois-pontos para selecionar, por exemplo, um elemento a cada dois

```
In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 6, 0, 1]

In [81]: seq[::2]
Out[81]: [7, 3, 3, 0]
```

Visão geral da indexação



Estrutura de Dados: Listas

Compreensões de lista são uma forma concisa de criar listas, permitindo filtrar e transformar elementos com uma única expressão

```
[expr for value in collection if condition]
```

Equivale a um laço “for” com condição

```
result = []
for value in collection:
    if condition:
        result.append(expr)
```

A condição de filtro pode ser omitida, restando apenas a expressão

Estrutura de Dados: Listas

Por exemplo, para uma lista de strings, podemos filtrar as strings com comprimento de 2 ou menos e converter as demais para maiúsculas

```
In [155]: strings = ["a", "as", "bat", "car", "dove", "python"]

In [156]: [x.upper() for x in strings if len(x) > 2]
Out[156]: ['BAT', 'CAR', 'DOVE', 'PYTHON']
```

Compreensões de lista geralmente são mais rápidas do que loops porque utilizam um mecanismo interno mais otimizado para iterar sobre a coleção

- Também é possível usar compreensões de lista aninhadas

Acesse os demais métodos de lista aqui!

Estrutura de Dados: Dicionários

Dicionários são estruturas de dados fundamentais em Python, comparáveis a hash maps ou arrays associativos em outras linguagens

Eles armazenam coleções de pares chave-valor, nos quais tanto as chaves quanto os valores são objetos Python

Cada chave é associada a um valor, simplificando a recuperação, inserção, modificação ou exclusão de valores

Para criar um dicionário, utilizam-se chaves “{ }” e dois-pontos: para separar chaves de valores e vírgulas “,” para separar os elementos

Estrutura de Dados: Dicionários

Exemplo de dicionário:

```
In [83]: empty_dict = {}

In [84]: d1 = {"a": "some value", "b": [1, 2, 3, 4]}

In [85]: d1
Out[85]: {'a': 'some value', 'b': [1, 2, 3, 4]}
```

Para acessar elementos em um dicionário, você utiliza a chave entre colchetes [].

- Por exemplo, d1[chave] retorna o valor associado àquela chave

Para inserir ou modificar um elemento, a sintaxe é semelhante: d1[chave] = valor

- Se a chave já existir, o valor é atualizado; se não, um novo par chave-valor é adicionado ao dicionário

Estrutura de Dados: Dicionários

Use a palavra-chave “del” ou o método “pop” para deletar valores

- “pop” também retorna o valor ao deletar a chave

```
In [94]: del d1[5]

In [95]: d1
Out[95]:
{'a': 'some value',
 'b': [1, 2, 3, 4],
 7: 'an integer',
 'dummy': 'another value'}
```

```
In [96]: ret = d1.pop("dummy")

In [97]: ret
Out[97]: 'another value'
```

Estrutura de Dados: Dicionários

Os métodos “keys” e “values” fornecem iteradores das chaves e valores do dicionário, respectivamente

A ordem das chaves depende da ordem de inserção, e esses métodos retornam as chaves e valores na mesma ordem respectiva

```
In [99]: list(d1.keys())
Out[99]: ['a', 'b', 7]
```

```
In [100]: list(d1.values())
Out[100]: ['some value', [1, 2, 3, 4], 'an integer']
```

Estrutura de Dados: Dicionários

Use o método “update” para mesclar um dicionário em outro

```
In [102]: d1.update({"b": "foo", "c": 12})

In [103]: d1
Out[103]: {'a': 'some value', 'b': 'foo', 7: 'an integer', 'c': 12}
```

O método update modifica dicionários diretamente, descartando os valores antigos de chaves existentes no dado passado para atualização

Estrutura de Dados: Dicionários

Associar duas sequências elemento a elemento em um dicionário é uma prática útil para criar mapeamentos diretos entre conjuntos de dados

```
mapping = {}
for key, value in zip(key_list, value_list):
    mapping[key] = value
```

A função zip em Python combina elementos de duas ou mais sequências, criando um iterador de tuplas

- Cada tupla contém elementos de cada sequência, posicionados na mesma ordem

Estrutura de Dados: Dicionários

Um dicionário pode ser visto como uma coleção de pares chave-valor, onde cada par é representado por uma tupla de dois elementos

A função dict converte listas de tuplas (chave, valor) em dicionários

- Criando um mapeamento direto entre chaves e valores a partir dessas tuplas

```
In [104]: tuples = zip(range(5), reversed(range(5)))

In [105]: tuples
Out[105]: <zip at 0x7fefe4553a00>

In [106]: mapping = dict(tuples)

In [107]: mapping
Out[107]: {0: 4, 1: 3, 2: 2, 3: 1, 4: 0}
```

Estrutura de Dados: Dicionários

Os métodos “get” e “pop” de dicionários aceitam um valor padrão opcional

“dict.get(key, default)” retorna o valor padrão se a chave não for encontrada, evitando a necessidade de if-else

```
value = some_dict.get(key, default_value)
```

Sem um valor padrão, get retorna None e pop lança uma exceção se a chave estiver ausente

Estrutura de Dados: Dicionários

O método “setdefault” de um dicionário insere uma chave com um valor padrão se a chave não existir no dicionário

```
In [112]: by_letter = {}

In [113]: for word in words:
   .....:     letter = word[0]
   .....:     by_letter.setdefault(letter, []).append(word)
   .....:

In [114]: by_letter
Out[114]: {'a': ['apple', 'atom'], 'b': ['bat', 'bar', 'book']}
```

Se a chave já existir no dicionário, o método simplesmente retorna o valor atual associado a essa chave, sem modificar o dicionário

Estrutura de Dados: Dicionários

“defaultdict” do módulo Collections facilita a criação de dicionários com valores padrão

Ao criar um “defaultdict”, passe um tipo ou função que gere o valor padrão para cada entrada no dicionário

```
In [115]: from collections import defaultdict

In [116]: by_letter = defaultdict(list)

In [117]: for word in words:
   .....:     by_letter[word[0]].append(word)
```

Isso elimina a necessidade de verificar manualmente se uma chave está presente e de inicializar manualmente valores padrão

Estrutura de Dados: Dicionários

As chaves de um dicionário precisam ser objetos imutáveis para serem *hashable*

- Escalares (int, float, string)
- Tuplas (todos os objetos na tupla também devem ser imutáveis)
- **Não é o caso de:** Listas, dicionários e conjuntos

Use a função “hash” para verificar se um objeto é *hashable* (pode ser usado como chave em um dicionário)

- Valores podem variar de acordo com a versão do Python

Estrutura de Dados: Dicionários

Alguns exemplos:

```
In [118]: hash("string")
Out[118]: 3634226001988967898
```

Estrutura de Dados: Dicionários

Alguns exemplos:

```
In [118]: hash("string")
Out[118]: 3634226001988967898

In [119]: hash((1, 2, (2, 3)))
Out[119]: -9209053662355515447
```

Estrutura de Dados: Dicionários

Alguns exemplos:

```
In [118]: hash("string")
Out[118]: 3634226001988967898

In [119]: hash((1, 2, (2, 3)))
Out[119]: -9209053662355515447

In [120]: hash((1, 2, [2, 3])) # fails because lists are mutable
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-120-473c35a62c0b> in <module>
----> 1 hash((1, 2, [2, 3])) # fails because lists are mutable

TypeError: unhashable type: 'list'
```

Estrutura de Dados: Dicionários

As compreensões de dicionário são semelhantes às de lista:

```
dict_comp = {key-expr: value-expr for value in collection
             if condition}
```

Exemplo:

```
In [160]: loc_mapping = {value: index for index, value in enumerate(strings)}

In [161]: loc_mapping
Out[161]: {'a': 0, 'as': 1, 'bat': 2, 'car': 3, 'dove': 4, 'python': 5}
```

Acesse todos os métodos do objeto dict neste link!

Estrutura de Dados: Conjuntos (Sets)

Um conjunto é uma coleção desordenada de elementos únicos

Pode ser criado de duas maneiras: usando a função “set” ou um literal de conjunto com chaves

```
In [124]: set([2, 2, 2, 1, 3, 3])
Out[124]: {1, 2, 3}

In [125]: {2, 2, 2, 1, 3, 3}
Out[125]: {1, 2, 3}
```

Estrutura de Dados: Conjuntos (Sets)

Suporta várias operações matemáticas. Considere, por exemplo:

```
In [126]: a = {1, 2, 3, 4, 5}
In [127]: b = {3, 4, 5, 6, 7, 8}
```

É possível aplicar operações como união e intersecção:

```
In [128]: a.union(b)
Out[128]: {1, 2, 3, 4, 5, 6, 7, 8}

In [129]: a | b
Out[129]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [130]: a.intersection(b)
Out[130]: {3, 4, 5}

In [131]: a & b
Out[131]: {3, 4, 5}
```

Estrutura de Dados: Conjuntos (Sets)

Sets também suportam compressão:

```
set_comp = {expr for value in collection if condition}
```

Exemplo:

```
In [157]: unique_lengths = {len(x) for x in strings}

In [158]: unique_lengths
Out[158]: {1, 2, 3, 4, 6}
```

Acesse todos os métodos do objeto set neste link!

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

Funções de Sequência Integradas ao Python

Python oferece várias funções para trabalhar com sequências (listas, tuplas sets e strings)

São essenciais para manipulação eficiente de dados em Python, simplificando muitas operações comuns com sequências

Algumas funções úteis:

- Enumerate
- Sorted
- Zip
- Reversed
- Len, min, max e sum

Enumerate

Ao iterar sobre uma sequência, é comum precisar rastrear o índice do item atual

A função enumerate retorna uma sequência de tuplas (índice, valor):

```
index = 0
for value in collection:
    # do something with value
    index += 1
```

Enumerate

Ao iterar sobre uma sequência, é comum precisar rastrear o índice do item atual

A função enumerate retorna uma sequência de tuplas (índice, valor):

```
index = 0
for value in collection:
    # do something with value
    index += 1
```

```
for index, value in enumerate(collection):
    # do something with value
```

Sorted

A função sorted retorna uma nova lista ordenada a partir dos elementos de qualquer sequência

Aceita os mesmos argumentos que o método sort em listas (“key” and “reverse”)

```
In [145]: sorted([7, 1, 2, 6, 0, 3, 2])
Out[145]: [0, 1, 2, 2, 3, 6, 7]

In [146]: sorted("horse race")
Out[146]: [' ', 'a', 'c', 'e', 'e', 'h', 'o', 'r', 'r', 's']
```

Zip

Agrupa os elementos de várias listas, tuplas ou outras sequências, criando uma lista de tuplas

```
In [147]: seq1 = ["foo", "bar", "baz"]

In [148]: seq2 = ["one", "two", "three"]

In [149]: zipped = zip(seq1, seq2)

In [150]: list(zipped)
Out[150]: [('foo', 'one'), ('bar', 'two'), ('baz', 'three')]
```

Zip

“zip” é frequentemente usado para iterar simultaneamente sobre múltiplas sequências de forma combinada com o “enumerate”

```
In [147]: seq1 = ["foo", "bar", "baz"]

In [148]: seq2 = ["one", "two", "three"]

In [153]: for index, (a, b) in enumerate(zip(seq1, seq2)):
   .....:     print(f"{index}: {a}, {b}")
   .....:
0: foo, one
1: bar, two
2: baz, three
```

Zip

"zip" é frequentemente usado para iterar simultaneamente sobre múltiplas sequências de forma combinada com o "enumerate"

```
In [153]: for index, (a, b) in enumerate(zip(seq1, seq2)):
   .....:     print(f"{index}: {a}, {b}")
   .....:
0: foo, one
1: bar, two
2: baz, three
```

Também é possível limitar o tamanho das tuplas combinadas

```
In [151]: seq3 = [False, True]
In [152]: list(zip(seq1, seq2, seq3))
Out[152]: [('foo', 'one', False), ('bar', 'two', True)]
```

Funções em Python

Funções permitem organização, reusabilidade e legibilidade do código

Funções são declaradas com a palavra-chave “def”

Contêm um bloco de código e podem usar opcionalmente a palavra-chave return

```
In [173]: def my_function(x, y):
     .....:     return x + y
```

Funções em Python

Funções permitem organização, reusabilidade e legibilidade do código

Funções são declaradas com a palavra-chave “def”

Contêm um bloco de código e podem usar opcionalmente a palavra-chave return

```
In [173]: def my_function(x, y):
     .....:     return x + y
```

```
In [174]: my_function(1, 2)
Out[174]: 3
```

```
In [175]: result = my_function(1, 2)

In [176]: result
Out[176]: 3
```

Funções em Python

Funções podem ter argumentos **posicionais** e de **palavra-chave**

- Argumentos de palavra-chave são usados para valores padrão ou argumentos opcionais

Argumentos de palavra-chave podem ser passados com ou sem a palavra-chave, mas o uso da palavra-chave é recomendado

- Argumentos de palavra-chave devem vir após os argumentos posicionais
- A ordem dos argumentos de palavra-chave não importa, apenas seus nomes

Funções em Python

Considere a função:

```
def my_function2(x, y, z=1.5):
    if z > 1:
        return z * (x + y)
    else:
        return z / (x + y)
```

Funções em Python

Considere a função:

```
def my_function2(x, y, z=1.5):
    if z > 1:
        return z * (x + y)
    else:
        return z / (x + y)
```

Exemplos de chamadas:

```
In [181]: my_function2(5, 6, z=0.7)
Out[181]: 0.06363636363636363
```

Funções em Python

Considere a função:

```
def my_function2(x, y, z=1.5):
    if z > 1:
        return z * (x + y)
    else:
        return z / (x + y)
```

Exemplos de chamadas:

```
In [181]: my_function2(5, 6, z=0.7)
Out[181]: 0.06363636363636363
```

```
In [182]: my_function2(3.14, 7, 3.5)
Out[182]: 35.49
```

Funções em Python

Considere a função:

```
def my_function2(x, y, z=1.5):
    if z > 1:
        return z * (x + y)
    else:
        return z / (x + y)
```

Exemplos de chamadas:

```
In [181]: my_function2(5, 6, z=0.7)
Out[181]: 0.06363636363636363
```

```
In [182]: my_function2(3.14, 7, 3.5)
Out[182]: 35.49
```

```
In [183]: my_function2(10, 20)
Out[183]: 45.0
```

Namespaces, Escopo e Funções Locais

Funções podem acessar variáveis criadas dentro e fora delas, em escopos superiores ou globais

- O termo “namespace” descreve melhor o escopo de uma variável em Python
- Variáveis atribuídas dentro de uma função pertencem ao “namespace” local

O “namespace” local é criado ao chamar a função e inicialmente contém os argumentos da função

Após a execução da função, o “namespace” local é destruído, com algumas exceções (funções aninhadas)

Funções em Python

Exemplo de variável local e que não é preservada após a execução:

```
def func():
    a = []
    for i in range(5):
        a.append(i)
```

Quando ela é chamada, a lista vazia a é criada, cinco elementos são anexados e, então, a lista “a” é destruída

Considerando o escopo global:

```
In [184]: a = []

In [185]: def func():
   .....:     for i in range(5):
   .....:         a.append(i)
```

```
In [186]: func()

In [187]: a
Out[187]: [0, 1, 2, 3, 4]
```

Funções em Python

Atribuir variáveis fora do escopo de uma função é possível

- Mas essas variáveis precisam ser declaradas explicitamente usando a palavra-chave “global”

Exemplo:

```
In [190]: a = None

In [191]: def bind_a_variable():
    .....:     global a
    .....:     a = []
    .....: bind_a_variable()
    .....:
```

Funções em Python

Use “global” em funções para modificar variáveis globais diretamente

- Logo, “global” é necessário apenas quando **reatribuímos** um novo objeto a uma variável global

Para objetos mutáveis (listas, dicionários, conjuntos), modificar elementos não requer global

Funções em Python

Python facilita o retorno de múltiplos valores de uma função, empacotando-os automaticamente em uma tupla

Esta tupla pode ser desempacotada diretamente nas variáveis receptoras permitindo uma sintaxe concisa e flexível para o manuseio de múltiplos retornos

```
def f():
    a = 5
    b = 6
    c = 7
    return a, b, c

a, b, c = f()
```

```
def f():
    a = 5
    b = 6
    c = 7
    return {"a" : a, "b" : b, "c" : c}

return_value = f()
```

Geradores em Python

Objetos como listas e arquivos são iteráveis por meio de um protocolo iterador

- Por exemplo, ao usar “for key in some_dict”, python naturalmente cria um iterador de some_dict. Podemos fazer algo equivalente:

```
In [209]: dict_iterator = iter(some_dict)

In [210]: dict_iterator
Out[210]: <dict_keyiterator at 0x7fefe45465c0>
```

Um iterador fornece elementos ao Python em laços e é compatível com funções como “min”, “max”, “sum” e construtores de listas e tuplas

```
In [211]: list(dict_iterator)
Out[211]: ['a', 'b', 'c']
```

Geradores em Python

Um gerador é uma forma prática de criar um objeto iterável, semelhante a uma função normal

Porém, geradores não retornam os valores de imediato mas um de cada vez, interrompendo e retomando a execução conforme necessário

- Ao produzir elementos individualmente e não uma lista inteira de uma vez, o uso de memória é reduzido significativamente

Para criar um gerador em uma função, basta usar “yield” ao invés de “return”

Geradores em Python

Considere, por exemplo, a seguinte função:

```
def squares(n=10):
    print(f"Generating squares from 1 to {n ** 2}")
    for i in range(1, n + 1):
        yield i ** 2
```

Geradores em Python

Considere, por exemplo, a seguinte função:

```
def squares(n=10):
    print(f"Generating squares from 1 to {n ** 2}")
    for i in range(1, n + 1):
        yield i ** 2
```

Execução:

```
In [213]: gen = squares()

In [214]: gen
Out[214]: <generator object squares at 0x7fefe437d620>
```

Geradores em Python

Considere, por exemplo, a seguinte função:

```
def squares(n=10):
    print(f"Generating squares from 1 to {n ** 2}")
    for i in range(1, n + 1):
        yield i ** 2
```

Execução:

```
In [213]: gen = squares()

In [214]: gen
Out[214]: <generator object squares at 0x7fefe437d620>
```

O código só será de fato executado quando os elementos são solicitados

```
In [215]: for x in gen:
   .....:     print(x, end=" ")
Generating squares from 1 to 100
1 4 9 16 25 36 49 64 81 100
```

Geradores em Python

Expressões geradoras: Similar às compreensões de lista, dicionário e conjunto, mas cria um gerador

- Use parênteses em vez de colchetes para definir

```
In [216]: gen = (x ** 2 for x in range(100))

In [217]: gen
Out[217]: <generator object <genexpr> at 0x7fefe437d000>
```

Geradores em Python

Expressões geradoras: Similar às compreensões de lista, dicionário e conjunto, mas cria um gerador

- Use parênteses em vez de colchetes para definir

```
In [216]: gen = (x ** 2 for x in range(100))

In [217]: gen
Out[217]: <generator object <genexpr> at 0x7fefe437d000>
```

Podem ser usados diretamente como argumentos para funções

```
In [218]: sum(x ** 2 for x in range(100))
Out[218]: 328350

In [219]: dict((i, i ** 2) for i in range(5))
Out[219]: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Módulo Itertools

O módulo itertools disponibiliza geradores para operações como agrupamento, permutação e combinação, otimizando o trabalho com sequências em Python

```
In [220]: import itertools

In [221]: def first_letter(x):
   .....:     return x[0]

In [222]: names = ["Alan", "Adam", "Wes", "Will", "Albert", "Steven"]

In [223]: for letter, names in itertools.groupby(names, first_letter):
   .....:     print(letter, list(names)) # names is a generator
A ['Alan', 'Adam']
W ['Wes', 'Will']
A ['Albert']
S ['Steven']
```

“Groupby” percorre a lista names e para cada nome, usa “first_letter” para determinar a chave de agrupamento (neste caso, a primeira letra do nome)

- Assume que a sequência tem que estar ordenada

Módulo Itertools

O módulo itertools disponibiliza geradores para operações como agrupamento, permutação e combinação, otimizando o trabalho com sequências em Python

Function	Description
`chain(*iterables)`	Generates a sequence by chaining iterators together. Once elements from the first iterator are exhausted, elements from the next iterator are returned, and so on.
`combinations(iterable, k)`	Generates a sequence of all possible k-tuples of elements in the iterable, ignoring order and without replacement (see also the companion function `combinations_with_replacement`).
`permutations(iterable, k)`	Generates a sequence of all possible k-tuples of elements in the iterable, respecting order.
`groupby(iterable[, keyfunc])`	Generates `(key, sub-iterator)` for each unique key.
`product(*iterables, repeat=1)`	Generates the Cartesian product of the input iterables as tuples, similar to a nested `for` loop.

Veja mais em: https://docs.python.org/3/library/itertools.html

Leitura e Escrita de Arquivos

Para abrir um arquivo, use a função nativa “open” com um caminho de arquivo relativo ou absoluto e uma codificação de arquivo opcional

```
In [233]: path = "examples/segismundo.txt"

In [234]: f = open(path, encoding="utf-8")
```

Por padrão, arquivos são abertos no modo somente leitura "r". O objeto do arquivo pode ser iterado como uma lista para ler as linhas:

```
In [235]: lines = [x.rstrip() for x in open(path, encoding="utf-8")]

In [236]: lines
Out[236]:
['Sueña el rico en su riqueza,',
 'que más cuidados le ofrece;']
```

Leitura e Escrita de Arquivos

Use “f.close()” para fechar o arquivo e liberar recursos

```
In [237]: f.close()
```

Use “with” para fechar arquivos automaticamente ao sair do bloco

```
In [238]: with open(path, encoding="utf-8") as f:
   .....:     lines = [x.rstrip() for x in f]
```

Abrir arquivos com “f = open(caminho, "w")” cria um novo arquivo, sobrescrevendo existentes

- O modo “x” cria um arquivo, mas falha se o caminho do arquivo já existir

Saiba mais sobre todos estes recursos neste link!

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

Módulo Numpy

NumPy (*Numerical Python*) é uma biblioteca fundamental para computação científica em Python

Alguns motivos para isso:

- Oferece suporte a arrays multidimensionais e operações matemáticas muito mais rápidas, com menos memória e sem o uso de *loops* (vetorizadas)
- Possui uma API em C, permitindo integração com bibliotecas em C, C++ ou FORTRAN
- Facilita a integração de código legado em C/C++/FORTRAN com interfaces dinâmicas em Python

Módulo Numpy

Para se ter uma ideia, considere o seguinte cenário

```
In [7]: import numpy as np

In [8]: my_arr = np.arange(1_000_000)

In [9]: my_list = list(range(1_000_000))
```

Considere também a multiplicação de cada sequência por 2

Módulo Numpy

Para se ter uma ideia, considere o seguinte cenário

```
In [7]: import numpy as np

In [8]: my_arr = np.arange(1_000_000)

In [9]: my_list = list(range(1_000_000))
```

Considere também a multiplicação de cada sequência por 2

```
In [10]: %timeit my_arr2 = my_arr * 2
715 us +- 13.2 us per loop (mean +- std. dev. of 7 runs, 1000 loops each)

In [11]: %timeit my_list2 = [x * 2 for x in my_list]
48.8 ms +- 298 us per loop (mean +- std. dev. of 7 runs, 10 loops each)
```

Módulo Numpy

Para se ter uma ideia, considere o seguinte cenário

```
In [7]: import numpy as np

In [8]: my_arr = np.arange(1_000_000)

In [9]: my_list = list(range(1_000_000))
```

Considere também a multiplicação de cada sequência por 2

```
In [10]: %timeit my_arr2 = my_arr * 2
715 us +- 13.2 us per loop (mean +- std. dev. of 7 runs, 1000 loops each)

In [11]: %timeit my_list2 = [x * 2 for x in my_list]
48.8 ms +- 298 us per loop (mean +- std. dev. of 7 runs, 10 loops each)
```

Operações com NumPy são geralmente de 10 a 100x mais rápidas que suas versões puras em Python e consomem significativamente menos memória

Módulo Numpy

Criando um n-dimensional array

```
In [12]: import numpy as np

In [13]: data = np.array([[1.5, -0.1, 3], [0, -3, 6.5]])

In [14]: data
Out[14]:
array([[ 1.5, -0.1,  3. ],
       [ 0. , -3. ,  6.5]])
```

Realizando operações matemáticas

```
In [16]: data + data
Out[16]:
array([[ 3. , -0.2,  6. ],
       [ 0. , -6. , 13. ]])
```

Módulo Numpy

Um ndarray é um contêiner multidimensional genérico para dados **homogêneos**, ou seja, todos os elementos devem ser do mesmo tipo

Cada array possui uma tupla que indica o tamanho de cada dimensão (“shape”) e um objeto que descreve o tipo de dado do array (“dtype”)

```
In [17]: data.shape
Out[17]: (2, 3)

In [18]: data.dtype
Out[18]: dtype('float64')
```

Módulo Numpy

A maneira mais fácil de criar um array é usar a função “array” passando uma sequência (lista ou conjunto) n-dimensional

```
In [19]: data1 = [6, 7.5, 8, 0, 1]
In [20]: arr1 = np.array(data1)

In [21]: arr1
Out[21]: array([6. , 7.5, 8. , 0. , 1. ])
```

Módulo Numpy

A maneira mais fácil de criar um array é usar a função “array” passando uma sequência (lista ou conjunto) n-dimensional

```
In [19]: data1 = [6, 7.5, 8, 0, 1]
In [20]: arr1 = np.array(data1)

In [21]: arr1
Out[21]: array([6. , 7.5, 8. , 0. , 1. ])
```

```
In [22]: data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]

In [23]: arr2 = np.array(data2)

In [24]: arr2
Out[24]:
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
```

Módulo Numpy

A função “numpy.array” tenta inferir o melhor tipo de dado para o array que cria, a menos que um tipo seja explicitamente especificado

Considerando os arrays anteriores

```
In [27]: arr1.dtype
Out[27]: dtype('float64')

In [28]: arr2.dtype
Out[28]: dtype('int64')
```

Algumas vezes é útil você especificar o tipo de dados para economizar memória

- Por exemplo, em uma matriz em que os valores inteiros estão entre [0,255], use “uint8” ao invés do padrão que é “int32”

Módulo Numpy

Type	Type code	Description
`int8, uint8`	`i1, u1`	Signed and unsigned 8-bit (1 byte) integer types
`int16, uint16`	`i2, u2`	Signed and unsigned 16-bit integer types
`int32, uint32`	`i4, u4`	Signed and unsigned 32-bit integer types
`int64, uint64`	`i8, u8`	Signed and unsigned 64-bit integer types
`float16`	`f2`	Half-precision floating point
`float32`	`f4 or f`	Standard single-precision floating point; compatible with C float
`float64`	`f8 or d`	Standard double-precision floating point; compatible with C double and Python `float` object
`float128`	`f16 or g`	Extended-precision floating point
`complex64, complex128, complex256`	`c8, c16, c32`	Complex numbers represented by two 32, 64, or 128 floats, respectively
`bool`	`?`	Boolean type storing `True` and `False` values
`object`	`0`	Python object type; a value can be any Python object

Saiba mais sobre os recursos do Numpy neste link!

Numpy: Indexação e Fatiamento

A indexação em NumPy é versátil, permitindo várias maneiras de acessar subconjuntos de dados

Para arrays unidimensionais, a operação é intuitiva, assemelhando-se ao uso de listas em Python

```
In [61]: arr = np.arange(10)

In [62]: arr
Out[62]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [63]: arr[5]
Out[63]: 5

In [64]: arr[5:8]
Out[64]: array([5, 6, 7])
```

Numpy: Indexação e Fatiamento

Ao atribuir um valor escalar a uma fatia, como em arr[5:8] = 12, o valor é propagado (*Broadcasting*) para toda a seleção

```
In [65]: arr[5:8] = 12

In [66]: arr
Out[66]: array([ 0,  1,  2,  3,  4, 12, 12, 12,  8,  9])
```

***Broadcasting*:** Método que permite operações aritméticas em arrays de diferentes tamanhos

- Estende automaticamente um deles para corresponder ao formato do outro

Numpy: Indexação e Fatiamento

Diferente das listas Python, fatias de arrays NumPy são visões do array original, significando que os dados não são copiados

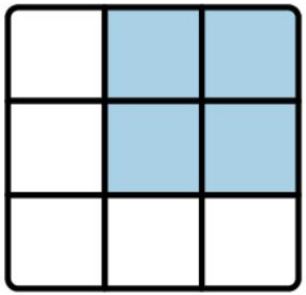
- Portanto, modificações na fatia afetam o array de origem

Use copy() após o fatiamento para criar uma cópia real da fatia, evitando alterar o array original

Esse comportamento visa eficiência com grandes volumes de dados, minimizando a memória usada por cópias desnecessárias

Numpy: Indexação e Fatiamento

Outras formas de indexação:

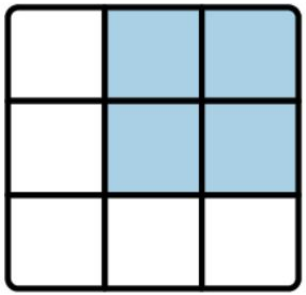


Expression	Shape
`arr[:2,1:]`	(2,2)

Seleciona as primeiras duas linhas (:2)

Numpy: Indexação e Fatiamento

Outras formas de indexação:

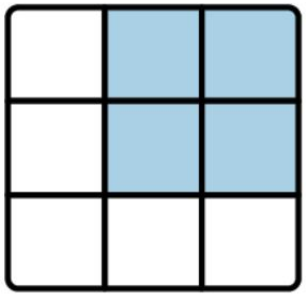


Expression	Shape
`arr[:2,1:]`	(2,2)

Seleciona as colunas da segunda em diante (1:)

Numpy: Indexação e Fatiamento

Outras formas de indexação:

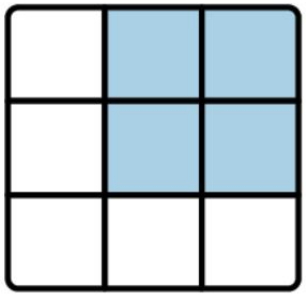


Expression	Shape
`arr[:2,1:]`	`(2,2)`

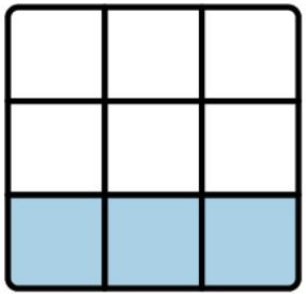
Produz uma **visão** (2,2)

Numpy: Indexação e Fatiamento

Outras formas de indexação:



Expression	Shape
`arr[:2,1:]`	`(2,2)`

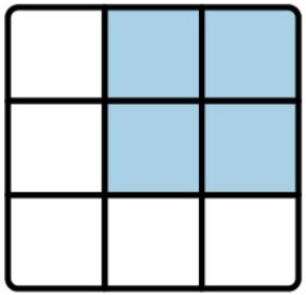


`arr[2]`	`(3,)`

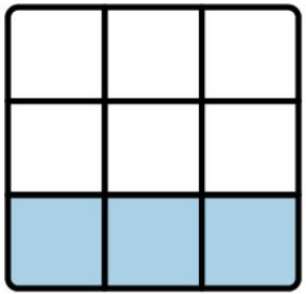
Seleciona a linha 2 inteira!

Numpy: Indexação e Fatiamento

Outras formas de indexação:



Expression	Shape
`arr[:2,1:]`	`(2,2)`

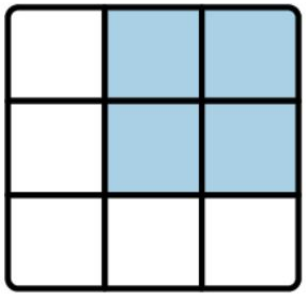


`arr[2]`	`(3,)`

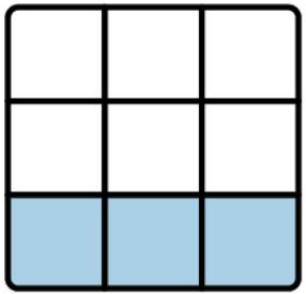
Produz uma visão **dessa** linha!

Numpy: Indexação e Fatiamento

Outras formas de indexação:



Expression	Shape
`arr[:2,1:]`	(2,2)

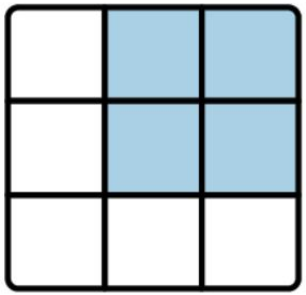


`arr[2]`	(3,)
`arr[2, :]`	(3,)

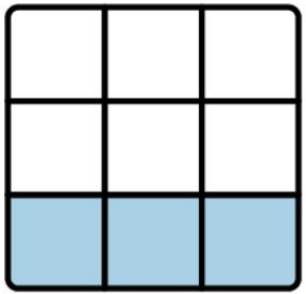
Alternativamente, pode-se selecionar a linha dois e especificar todas as colunas

Numpy: Indexação e Fatiamento

Outras formas de indexação:



Expression	Shape
`arr[:2,1:]`	(2,2)



`arr[2]`	(3,)
`arr[2, :]`	(3,)

Saiba mais sobre indexação, neste link!

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

Introdução ao Pandas

Biblioteca com estruturas e ferramentas para análise e limpeza de dados visando agilidade

Usada em conjunto com outras bibliotecas: NumPy, Scipy, statsmodels, matplotlib, seaborn, scikit-learn, etc

Possui duas estruturas de dados principais:

- Series
- DataFrame

Introdução ao Pandas: Series

Series: Estrutura unidimensional com rótulos (índices de eixo) capaz de armazenar qualquer tipo de dados

- Cada elemento pode ser acessado por um índice

```
In [1]: import numpy as np

In [2]: import pandas as pd
In [14]: obj = pd.Series([4, 7, -5, 3])

In [15]: obj
Out[15]:
0    4
1    7
2   -5
3    3
dtype: int64
```

```
In [14]: obj = pd.Series([4, 7, -5, 3])

In [15]: obj
Out[15]:
0    4
1    7
2   -5
3    3
dtype: int64
```

Introdução ao Pandas: Series

Também é possível definir os índices e usá-los para acessar os valores

```
In [18]: obj2 = pd.Series([4, 7, -5, 3], index=["d", "b", "a", "c"])

In [19]: obj2
Out[19]:
d    4
b    7
a   -5
c    3
dtype: int64

In [20]: obj2.index
Out[20]: Index(['d', 'b', 'a', 'c'], dtype='object')
```

Introdução ao Pandas: Series

Também é possível definir os índices e usá-los para acessar os valores

```
In [21]: obj2["a"]
Out[21]: -5

In [22]: obj2["d"] = 6

In [23]: obj2[["c", "a", "d"]]
Out[23]:
c    3
a   -5
d    6
dtype: int64
```

Introdução ao Pandas: Series

Exemplos de operações sobre series:

```
In [24]: obj2[obj2 > 0]
Out[24]:
d    6
b    7
c    3
dtype: int64
```

Introdução ao Pandas: Series

Exemplos de operações sobre series:

```
In [24]: obj2[obj2 > 0]
Out[24]:
d    6
b    7
c    3
dtype: int64
```

```
In [25]: obj2 * 2
Out[25]:
d    12
b    14
a   -10
c     6
dtype: int64
```

Introdução ao Pandas: Series

Exemplos de operações sobre series:

```
In [24]: obj2[obj2 > 0]
Out[24]:
d    6
b    7
c    3
dtype: int64
```

```
In [25]: obj2 * 2
Out[25]:
d    12
b    14
a   -10
c     6
dtype: int64
```

```
In [26]: import numpy as np

In [27]: np.exp(obj2)
Out[27]:
d     403.428793
b    1096.633158
a       0.006738
c      20.085537
dtype: float64
```

Introdução ao Pandas: Series

Exemplos de operações sobre series:

```
In [24]: obj2[obj2 > 0]
Out[24]:
d    6
b    7
c    3
dtype: int64
```

```
In [25]: obj2 * 2
Out[25]:
d    12
b    14
a   -10
c     6
dtype: int64
```

```
In [26]: import numpy as np

In [27]: np.exp(obj2)
Out[27]:
d     403.428793
b    1096.633158
a       0.006738
c      20.085537
dtype: float64
```

```
In [28]: "b" in obj2
Out[28]: True

In [29]: "e" in obj2
Out[29]: False
```

Introdução ao Pandas: Series

Converter um dicionário em uma *series* e vice-versa é simples e direto

```
In [30]: sdata = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000}

In [31]: obj3 = pd.Series(sdata)

In [32]: obj3
Out[32]:
Ohio      35000
Texas     71000
Oregon    16000
Utah       5000
dtype: int64

In [33]: obj3.to_dict()
Out[33]: {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}
```

Introdução ao Pandas: Series

Verificar a presença de dados faltantes NaN (*Not a Number*)

```
In [36]: obj4
Out[36]:
California        NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

Introdução ao Pandas: Series

Verificar a presença de dados faltantes NaN (*Not a Number*)

```
In [36]: obj4
Out[36]:
California        NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

```
In [37]: pd.isna(obj4)
Out[37]:
California     True
Ohio          False
Oregon        False
Texas         False
dtype: bool
```

Método “isna”

Introdução ao Pandas: Series

Verificar a presença de dados faltantes NaN (*Not a Number*)

```
In [36]: obj4
Out[36]:
California        NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

```
In [38]: pd.notna(obj4)
Out[38]:
California    False
Ohio           True
Oregon         True
Texas          True
dtype: bool
```

Método “notna”

Introdução ao Pandas: Series

Verificar a presença de dados faltantes NaN (*Not a Number*)

```
In [36]: obj4
Out[36]:
California        NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

```
In [39]: obj4.isna()
Out[39]:
California     True
Ohio          False
Oregon        False
Texas         False
dtype: bool
```

Esses métodos também são nativos nos objetos Series

Introdução ao Pandas: Series

Series facilitam operações **alinhadas** aos seus índices

```
In [40]: obj3
Out[40]:
Ohio      35000
Texas     71000
Oregon    16000
Utah       5000
dtype: int64
```

```
In [41]: obj4
Out[41]:
California        NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

```
In [42]: obj3 + obj4
Out[42]:
California         NaN
Ohio           70000.0
Oregon         32000.0
Texas         142000.0
Utah               NaN
dtype: float64
```

Introdução ao Pandas: DataFrames

DataFrame: Estrutura de dados tabular bidimensional com rótulos nas linhas e colunas

- É semelhante a uma planilha ou uma tabela SQL e pode armazenar vários tipos de dados em diferentes colunas
- Possui um índice tanto para linha quanto para coluna

Inclui várias funcionalidades:

- Manipulação, junção, mesclagem e transformação de dados, permitindo fácil combinação de dados de diferentes fontes
- Suporte para leitura/escrita em formatos variados como CSV, Excel, SQL e JSON

Introdução ao Pandas: DataFrames

Existem muitas formas de criar um DataFrame, embora as mais comuns sejam a partir das estruturas de dados básicas do python

```
data = {"state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
        "year": [2000, 2001, 2002, 2001, 2002, 2003],
        "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
frame = pd.DataFrame(data)
```

Introdução ao Pandas: DataFrames

Existem muitas formas de criar um DataFrame, embora as mais comuns sejam a partir das estruturas de dados básicas do python

```
data = {"state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
        "year": [2000, 2001, 2002, 2001, 2002, 2003],
        "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
frame = pd.DataFrame(data)
```

```
In [50]: frame
Out[50]:
    state  year  pop
0    Ohio  2000  1.5
1    Ohio  2001  1.7
2    Ohio  2002  3.6
3  Nevada  2001  2.4
4  Nevada  2002  2.9
5  Nevada  2003  3.2
```

Introdução ao Pandas: DataFrames

Você pode definir a ordem das colunas. Colunas inexistentes no dicionários serão totalmente preenchidas com NaN

```
In [54]: frame2 = pd.DataFrame(data, columns=["year", "state", "pop", "debt"])

In [55]: frame2
Out[55]:
   year   state  pop debt
0  2000    Ohio  1.5  NaN
1  2001    Ohio  1.7  NaN
2  2002    Ohio  3.6  NaN
3  2001  Nevada  2.4  NaN
4  2002  Nevada  2.9  NaN
5  2003  Nevada  3.2  NaN

In [56]: frame2.columns
Out[56]: Index(['year', 'state', 'pop', 'debt'], dtype='object')
```

Introdução ao Pandas: DataFrames

Algumas formas de acesso e visualização

```
In [51]: frame.head()
Out[51]:
    state  year  pop
0    Ohio  2000  1.5
1    Ohio  2001  1.7
2    Ohio  2002  3.6
3  Nevada  2001  2.4
4  Nevada  2002  2.9
```

Exibe as primeiras linhas

```
In [52]: frame.tail()
Out[52]:
    state  year  pop
1    Ohio  2001  1.7
2    Ohio  2002  3.6
3  Nevada  2001  2.4
4  Nevada  2002  2.9
5  Nevada  2003  3.2
```

Exibe as últimas linhas

É possível passar um valor para o parâmetro “n” indicando o número de linhas que você quer visualizar

- Por padrão, “n=5”

Introdução ao Pandas: DataFrames

Algumas formas de acesso e visualização

```
In [57]: frame2["state"]
Out[57]:
0      Ohio
1      Ohio
2      Ohio
3    Nevada
4    Nevada
5    Nevada
Name: state, dtype: object
```

Acessando uma coluna

```
In [58]: frame2.year
Out[58]:
0    2000
1    2001
2    2002
3    2001
4    2002
5    2003
Name: year, dtype: int64
```

Alternativa

Para nomes de colunas com espaços, caracteres especiais ou iguais a métodos do DataFrame, use a primeira alternativa

Introdução ao Pandas: DataFrames

Para selecionar linhas por posição ou nome do índice, use “iloc” para índices numéricos e “loc” para rótulos de índice

```
In [59]: frame2.loc[1]
Out[59]:
year      2001
state     Ohio
pop        1.7
debt       NaN
Name: 1, dtype: object
```

```
In [60]: frame2.iloc[2]
Out[60]:
year      2002
state     Ohio
pop        3.6
debt       NaN
Name: 2, dtype: object
```

No exemplo acima, como o índice é um inteiro, ambos funcionam

- Porém “loc” buscará a linha com esse valor de índice exato ao invés de acessá-la diretamente como “iloc”

Introdução ao Pandas: DataFrames

As colunas podem ser modificadas/criadas por atribuição direta

```
In [61]: frame2["debt"] = 16.5

In [62]: frame2
Out[62]:
   year   state  pop  debt
0  2000    Ohio  1.5  16.5
1  2001    Ohio  1.7  16.5
2  2002    Ohio  3.6  16.5
3  2001  Nevada  2.4  16.5
4  2002  Nevada  2.9  16.5
5  2003  Nevada  3.2  16.5
```

```
In [63]: frame2["debt"] = np.arange(6.)

In [64]: frame2
Out[64]:
   year   state  pop  debt
0  2000    Ohio  1.5   0.0
1  2001    Ohio  1.7   1.0
2  2002    Ohio  3.6   2.0
3  2001  Nevada  2.4   3.0
4  2002  Nevada  2.9   4.0
5  2003  Nevada  3.2   5.0
```

Ao criar ou modificar colunas, a correspondência dos índices é crucial

- Desalinhamentos podem levar a resultados inesperados

Introdução ao Pandas: DataFrames

Colunas também podem modificadas/criadas por modificação

```
In [68]: frame2["eastern"] = frame2["state"] == "Ohio"

In [69]: frame2
Out[69]:
   year   state  pop  debt  eastern
0  2000    Ohio  1.5   NaN     True
1  2001    Ohio  1.7   NaN     True
2  2002    Ohio  3.6   NaN     True
3  2001  Nevada  2.4   NaN    False
4  2002  Nevada  2.9   NaN    False
5  2003  Nevada  3.2   NaN    False
```

Introdução ao Pandas: DataFrames

Para remover uma coluna use “del”

```
In [70]: del frame2["eastern"]

In [71]: frame2.columns
Out[71]: Index(['year', 'state', 'pop', 'debt'], dtype='object')
```

Introdução ao Pandas: DataFrames

Possíveis entradas de dados do construtor do DataFrame

Type	Notes
2D ndarray	A matrix of data, passing optional row and column labels
Dictionary of arrays, lists, or tuples	Each sequence becomes a column in the DataFrame; all sequences must be the same length
NumPy structured/record array	Treated as the "dictionary of arrays" case
Dictionary of Series	Each value becomes a column; indexes from each Series are unioned together to form the result's row index if no explicit index is passed
Dictionary of dictionaries	Each inner dictionary becomes a column; keys are unioned to form the row index as in the "dictionary of Series" case
List of dictionaries or Series	Each item becomes a row in the DataFrame; unions of dictionary keys or Series indexes become the DataFrame's column labels
List of lists or tuples	Treated as the "2D ndarray" case
Another DataFrame	The DataFrame's indexes are used unless different ones are passed
NumPy MaskedArray	Like the "2D ndarray" case except masked values are missing in the DataFrame result

Acesse a documentação completa neste link!

Introdução ao Pandas: Objeto Index

Objetos Index: Responsáveis por conter os rótulos dos eixos e outros metadados

```
In [84]: obj = pd.Series(np.arange(3), index=["a", "b", "c"])

In [85]: index = obj.index

In [86]: index
Out[86]: Index(['a', 'b', 'c'], dtype='object')

In [87]: index[1:]
Out[87]: Index(['b', 'c'], dtype='object')
```

Introdução ao Pandas: Objeto Index

Contém vários métodos e propriedades que facilitam operações entre DataFrames e Series de forma eficiente

Method/Property	Description
`append()`	Concatenate with additional Index objects, producing a new Index
`difference()`	Compute set difference as an Index
`intersection()`	Compute set intersection
`union()`	Compute set union
`isin()`	Compute Boolean array indicating whether each value is contained in the passed collection
`delete()`	Compute new Index with element at Index `i` deleted
`drop()`	Compute new Index by deleting passed values
`insert()`	Compute new Index by inserting element at Index `i`
`is_monotonic`	Returns `True` if each element is greater than or equal to the previous element
`is_unique`	Returns `True` if the Index has no duplicate values
`unique()`	Compute the array of unique values in the Index

Introdução ao Pandas: Reindexação

É possível reindexar uma Série ou um DataFrame

```
In [98]: obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])

In [99]: obj
Out[99]:
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

Introdução ao Pandas: Reindexação

É possível reindexar uma Série ou um DataFrame

```
In [98]: obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])
```

```
In [99]: obj
Out[99]:
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

Para reindexá-lo, use “reindex"

```
In [100]: obj2 = obj.reindex(["a", "b", "c", "d", "e"])

In [101]: obj2
Out[101]:
a   -5.3
b    7.2
c    3.6
d    4.5
e    NaN
dtype: float64
```

Introdução ao Pandas: Reindexação

Também podemos reindexar as colunas

```
In [106]: frame
Out[106]:
   Ohio  Texas  California
a     0      1           2
c     3      4           5
d     6      7           8
```

```
In [109]: states = ["Texas", "Utah", "California"]

In [110]: frame.reindex(columns=states)
Out[110]:
   Texas  Utah  California
a      1   NaN           2
c      4   NaN           5
d      7   NaN           8
```

Introdução ao Pandas: Reindexação

Argument	Description
`labels`	New sequence to use as an index. Can be Index instance or any other sequence-like Python data structure. An Index will be used exactly as is without any copying.
`index`	Use the passed sequence as the new index labels.
`columns`	Use the passed sequence as the new column labels.
`axis`	The axis to reindex, whether `"index"` (rows) or `"columns"`. The default is `"index"`. You can alternately do `reindex(index=new_labels)` or `reindex(columns=new_labels)`.
`method`	Interpolation (fill) method; `"ffill"` fills forward, while `"bfill"` fills backward.
`fill_value`	Substitute value to use when introducing missing data by reindexing. Use `fill_value="missing"` (the default behavior) when you want absent labels to have null values in the result.
`limit`	When forward filling or backfilling, the maximum size gap (in number of elements) to fill.
`tolerance`	When forward filling or backfilling, the maximum size gap (in absolute numeric distance) to fill for inexact matches.
`level`	Match simple Index on level of MultiIndex; otherwise select subset of.
`copy`	If `True`, always copy underlying data even if the new index is equivalent to the old index; if `False`, do not copy the data when the indexes are equivalent.

Introdução ao Pandas: Remoção de Entradas

Remover entradas de um eixo é possível usar ".loc" com um array de índices ou lista para definir as entradas a serem excluídas

```
In [113]: obj = pd.Series(np.arange(5.), index=["a", "b", "c", "d", "e"])

In [114]: obj
Out[114]:
a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64
```

Introdução ao Pandas: Remoção de Entradas

Remover entradas de um eixo é possível usar ".loc" com um array de índices ou lista para definir as entradas a serem excluídas

```
In [113]: obj = pd.Series(np.arange(5.), index=["a", "b", "c", "d", "e"])

In [114]: obj
Out[114]:
a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64
```

```
In [115]: new_obj = obj.drop("c")

In [116]: new_obj
Out[116]:
a    0.0
b    1.0
d    3.0
e    4.0
dtype: float64
```

```
In [117]: obj.drop(["d", "c"])
Out[117]:
a    0.0
b    1.0
e    4.0
dtype: float64
```

Introdução ao Pandas: Remoção de Entradas

DataFrames também oferecem várias opções. Por exemplo:

```
In [118]: data = pd.DataFrame(np.arange(16).reshape((4, 4)),
   .....:                     index=["Ohio", "Colorado", "Utah", "New York"],
   .....:                     columns=["one", "two", "three", "four"])

In [119]: data
Out[119]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Introdução ao Pandas: Remoção de Entradas

Algumas opções são:

```
In [120]: data.drop(index=["Colorado", "Ohio"])
Out[120]:
          one  two  three  four
Utah        8    9     10    11
New York   12   13     14    15
```

Remover linhas as indicando por meio do do argumento “index”

```
In [121]: data.drop(columns=["two"])
Out[121]:
          one  three  four
Ohio        0      2     3
Colorado    4      6     7
Utah        8     10    11
New York   12     14    15
```

Remover colunas as indicando por meio do argumento “columns”

Introdução ao Pandas: Remoção de Entradas

Algumas opções são:

```
In [122]: data.drop("two", axis=1)
Out[122]:
          one  three  four
Ohio        0      2     3
Colorado    4      6     7
Utah        8     10    11
New York   12     14    15
```

Remover linhas (ou colunas) as indicando por meio do argumento “axis=1” ou “axis=0)

```
In [123]: data.drop(["two", "four"], axis="columns")
Out[123]:
          one  three
Ohio        0      2
Colorado    4      6
Utah        8     10
New York   12     14
```

Remover linhas (ou colunas) as indicando por meio do argumento “axis=rows” ou “axis=columns)

Introdução ao Pandas: Indexação/Seleção/Filtragem

Indexação das Series funciona de maneira análoga a indexação de arrays NumPy, exceto que valores de índices podem ser usados

```
In [124]: obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])

In [125]: obj
Out[125]:
a    0.0
b    1.0
c    2.0
d    3.0
dtype: float64
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

Indexação das Series funciona de maneira análoga a indexação de arrays NumPy, exceto que valores de índices podem ser usados

```
In [124]: obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])

In [125]: obj
Out[125]:
a    0.0
b    1.0
c    2.0
d    3.0
dtype: float64
```

```
In [126]: obj["b"]
Out[126]: 1.0

In [127]: obj[1]
Out[127]: 1.0

In [128]: obj[2:4]
Out[128]:
c    2.0
d    3.0
dtype: float64
```

Indexação e fatiamentos por meio dos índices

Introdução ao Pandas: Indexação/Seleção/Filtragem

Indexação das Series funciona de maneira análoga a indexação de arrays NumPy, exceto que valores de índices podem ser usados

```
In [124]: obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])
```

```
In [125]: obj
Out[125]:
a    0.0
b    1.0
c    2.0
d    3.0
dtype: float64
```

```
In [129]: obj[["b", "a", "d"]]
Out[129]:
b    1.0
a    0.0
d    3.0
dtype: float64
```

```
In [130]: obj[[1, 3]]
Out[130]:
b    1.0
d    3.0
dtype: float64
```

Indexação e fatiamentos por meio de sequências

Introdução ao Pandas: Indexação/Seleção/Filtragem

Indexação das Series funciona de maneira análoga a indexação de arrays NumPy, exceto que valores de índices podem ser usados

```
In [124]: obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])

In [125]: obj
Out[125]:
a    0.0
b    1.0
c    2.0
d    3.0
dtype: float64
```

```
In [131]: obj[obj < 2]
Out[131]:
a    0.0
b    1.0
dtype: float64
```

Filtragem por meio de uma condição

Introdução ao Pandas: Indexação/Seleção/Filtragem

O método “loc” é a forma mais recomendada de fazer seleção, pois evita comportamentos indesejados. Por exemplo:

```
In [133]: obj1 = pd.Series([1, 2, 3], index=[2, 0, 1])

In [134]: obj2 = pd.Series([1, 2, 3], index=["a", "b", "c"])
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

O método “loc” é a forma mais recomendada de fazer seleção, pois evita comportamentos indesejados. Por exemplo:

```
In [133]: obj1 = pd.Series([1, 2, 3], index=[2, 0, 1])

In [134]: obj2 = pd.Series([1, 2, 3], index=["a", "b", "c"])
```

```
In [137]: obj1[[0, 1, 2]]
Out[137]:
0    2
1    3
2    1
dtype: int64
```

Seleção desejada

```
In [138]: obj2[[0, 1, 2]]
Out[138]:
a    1
b    2
c    3
dtype: int64
```

Pode ser uma seleção indesejada porque o índice não é de fato numérico

Introdução ao Pandas: Indexação/Seleção/Filtragem

O método “loc” é a forma mais recomendada de fazer seleção, pois evita comportamentos indesejados. Por exemplo:

```
In [133]: obj1 = pd.Series([1, 2, 3], index=[2, 0, 1])

In [134]: obj2 = pd.Series([1, 2, 3], index=["a", "b", "c"])
In [134]: obj2.loc[[0, 1]]
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_804589/4185657903.py in <module>
----> 1 obj2.loc[[0, 1]]

^ LONG EXCEPTION ABBREVIATED ^

KeyError: "None of [Int64Index([0, 1], dtype="int64")] are in the [index]"
```

Com “loc”, um exceção seria lançada! Mais seguro!

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [146]: data["two"]
Out[146]:
Ohio         1
Colorado     5
Utah         9
New York    13
Name: two, dtype: int64
```

Seleção de uma dada coluna

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [147]: data[["three", "one"]]
Out[147]:
          three  one
Ohio          2    0
Colorado      6    4
Utah         10    8
New York     14   12
```

Seleção de uma sequência de colunas

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [148]: data[:2]
Out[148]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
```

Seleção de uma sequência de linhas

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [149]: data[data["three"] > 5]
Out[149]:
          one  two  three  four
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Filtragem com base em uma coluna

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [150]: data < 5
Out[150]:
             one    two  three   four
Ohio        True   True   True   True
Colorado    True  False  False  False
Utah       False  False  False  False
New York   False  False  False  False
```

Gerando uma máscara booleana para filtragem

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [145]: data
Out[145]:
          one  two  three  four
Ohio        0    1      2     3
Colorado    4    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [151]: data[data < 5] = 0

In [152]: data
Out[152]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Usando a máscara para filtragem

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [154]: data.loc["Colorado"]
Out[154]:
one      0
two      5
three    6
four     7
Name: Colorado, dtype: int64
```

Usando “loc” com um índice

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [155]: data.loc[["Colorado", "New York"]]
Out[155]:
          one  two  three  four
Colorado    0    5      6     7
New York   12   13     14    15
```

Usando “Loc” e uma sequência como índice

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [156]: data.loc["Colorado", ["two", "three"]]
Out[156]:
two      5
three    6
Name: Colorado, dtype: int64
```

Usando “Loc” nos dois eixos

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [157]: data.iloc[2]
Out[157]:
one       8
two       9
three    10
four     11
Name: Utah, dtype: int64
```

Usando “iloc” e um índice

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [158]: data.iloc[[2, 1]]
Out[158]:
          one  two  three  four
Utah        8    9     10    11
Colorado    0    5      6     7
```

Usando “iloc” e uma sequência como índice

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [159]: data.iloc[2, [3, 0, 1]]
Out[159]:
four    11
one      8
two      9
Name: Utah, dtype: int64
```

Usando “iloc” e uma sequência como índice

Introdução ao Pandas: Indexação/Seleção/Filtragem

A indexação em um DataFrame recupera uma ou mais colunas com um único valor ou uma sequência. Considere o seguinte DataFrame

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [160]: data.iloc[[1, 2], [3, 0, 1]]
Out[160]:
          four  one  two
Colorado     7    0    5
Utah        11    8    9
```

Usando "iloc" e uma sequência para cada eixo

Introdução ao Pandas: Indexação/Seleção/Filtragem

Ambas as indexações trabalham com fatias, além de rótulos individuais ou listas de rótulos

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

Ambas as indexações trabalham com fatias, além de rótulos individuais ou listas de rótulos

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [161]: data.loc[:"Utah", "two"]
Out[161]:
Ohio        0
Colorado    5
Utah        9
Name: two, dtype: int64
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

Ambas as indexações trabalham com fatias, além de rótulos individuais ou listas de rótulos

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15

In [162]: data.iloc[:, :3][data.three > 5]
Out[162]:
          one  two  three
Colorado    0    5      6
Utah        8    9     10
New York   12   13     14
```

```
In [161]: data.loc[:"Utah", "two"]
Out[161]:
Ohio        0
Colorado    5
Utah        9
Name: two, dtype: int64
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

Ambas as indexações trabalham com fatias, além de rótulos individuais ou listas de rótulos

```
In [153]: data
Out[153]:
          one  two  three  four
Ohio        0    0      0     0
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

```
In [161]: data.loc[:"Utah", "two"]
Out[161]:
Ohio        0
Colorado    5
Utah        9
Name: two, dtype: int64
```

```
In [162]: data.iloc[:, :3][data.three > 5]
Out[162]:
          one  two  three
Colorado    0    5      6
Utah        8    9     10
New York   12   13     14
```

```
In [163]: data.loc[data.three >= 2]
Out[163]:
          one  two  three  four
Colorado    0    5      6     7
Utah        8    9     10    11
New York   12   13     14    15
```

Introdução ao Pandas: Indexação/Seleção/Filtragem

Type	Notes
`df[column]`	Select single column or sequence of columns from the DataFrame; special case conveniences: Boolean array (filter rows), slice (slice rows), or Boolean DataFrame (set values based on some criterion)
`df.loc[rows]`	Select single row or subset of rows from the DataFrame by label
`df.loc[:, cols]`	Select single column or subset of columns by label
`df.loc[rows, cols]`	Select both row(s) and column(s) by label
`df.iloc[rows]`	Select single row or subset of rows from the DataFrame by integer position
`df.iloc[:, cols]`	Select single column or subset of columns by integer position
`df.iloc[rows, cols]`	Select both row(s) and column(s) by integer position
`df.at[row, col]`	Select a single scalar value by row and column label
`df.iat[row, col]`	Select a single scalar value by row and column position (integers)
`reindex` method	Select either rows or columns by labels

Introdução ao Pandas: Funções/Mapeamento

Algumas funções do NumPy também funcionam em Pandas. Por exemplo, a função “absolute”

```
In [224]: frame
Out[224]:
               b         d         e
Utah   -0.204708  0.478943 -0.519439
Ohio   -0.555730  1.965781  1.393406
Texas   0.092908  0.281746  0.769023
Oregon  1.246435  1.007189 -1.296221
```

```
In [225]: np.abs(frame)
Out[225]:
               b         d         e
Utah    0.204708  0.478943  0.519439
Ohio    0.555730  1.965781  1.393406
Texas   0.092908  0.281746  0.769023
Oregon  1.246435  1.007189  1.296221
```

Introdução ao Pandas: Funções/Mapeamento

Você pode querer fazer a sua própria função e aplicá-la usando o “Apply”

```
In [226]: def f1(x):
     .....:     return x.max() - x.min()
```

```
In [224]: frame
Out[224]:
               b         d         e
Utah   -0.204708  0.478943 -0.519439
Ohio   -0.555730  1.965781  1.393406
Texas   0.092908  0.281746  0.769023
Oregon  1.246435  1.007189 -1.296221
```

Introdução ao Pandas: Funções/Mapeamento

Você pode querer fazer a sua própria função e aplicá-la usando o “Apply”

```
In [226]: def f1(x):
     .....:     return x.max() - x.min()
```

```
In [224]: frame
Out[224]:
               b         d         e
Utah   -0.204708  0.478943 -0.519439
Ohio   -0.555730  1.965781  1.393406
Texas   0.092908  0.281746  0.769023
Oregon  1.246435  1.007189 -1.296221
```

```
In [227]: frame.apply(f1)
Out[227]:
b    1.802165
d    1.684034
e    2.689627
dtype: float64
```

Introdução ao Pandas: Funções/Mapeamento

Funções similares “applymap” e “map”

```
In [231]: def my_format(x):
     .....:     return f"{x:.2f}"
```

```
In [232]: frame.applymap(my_format)
Out[232]:
             b     d      e
Utah     -0.20  0.48  -0.52
Ohio     -0.56  1.97   1.39
Texas     0.09  0.28   0.77
Oregon    1.25  1.01  -1.30
```

Exclusivo para DataFrames: Aplica uma função elemento a elemento em todo o DataFrame

```
In [233]: frame["e"].map(my_format)
Out[233]:
Utah      -0.52
Ohio       1.39
Texas      0.77
Oregon    -1.30
Name: e, dtype: object
```

Ideal para mapear/substituir cada elemento de uma Series por um novo valor com base em uma função/dicionário

Introdução ao Pandas: Ordenação

Ordenação lexicográfica por rótulos de linhas ou colunas é realizada com a função “sort_index”

```
In [235]: obj
Out[235]:
d    0
a    1
b    2
c    3
dtype: int64
```

```
In [236]: obj.sort_index()
Out[236]:
a    1
b    2
c    3
d    0
dtype: int64
```

Alguns argumentos interessantes:

- “ascending” igual a True (False) para ordem crescente (Decrescente)
- “inplace” para modificar o DataFrame original ao invés de criar uma cópia
- “key” para customizar a ordenação com uma função

Introdução ao Pandas: Ordenação

Em um DataFrame, a ordenação pode ser feita em qualquer eixo

```
In [238]: frame
Out[238]:
       d  a  b  c
three  0  1  2  3
one    4  5  6  7
```

```
In [239]: frame.sort_index()
Out[239]:
       d  a  b  c
one    4  5  6  7
three  0  1  2  3
```

Ordenação ascendente pelo eixo 0

Introdução ao Pandas: Ordenação

Em um DataFrame, a ordenação pode ser feita em qualquer eixo

```
In [238]: frame
Out[238]:
       d  a  b  c
three  0  1  2  3
one    4  5  6  7
```

```
In [240]: frame.sort_index(axis="columns")
Out[240]:
       a  b  c  d
three  1  2  3  0
one    5  6  7  4
```

Ordenação ascendente pelo eixo 1

Introdução ao Pandas: Ordenação

Em um DataFrame, a ordenação pode ser feita em qualquer eixo

```
In [238]: frame
Out[238]:
       d  a  b  c
three  0  1  2  3
one    4  5  6  7
```

```
In [241]: frame.sort_index(axis="columns", ascending=False)
Out[241]:
       d  c  b  a
three  0  3  2  1
one    4  7  6  5
```

Ordenação descendente pelo eixo 1

Introdução ao Pandas: Ordenação

A ordenação também pode ser feita por uma ou mais colunas

```
In [248]: frame
Out[248]:
   b  a
0  4  0
1  7  1
2 -3  0
3  2  1
```

```
In [250]: frame.sort_values(["a", "b"])
Out[250]:
   b  a
2 -3  0
0  4  0
3  2  1
1  7  1
```

Você pode definir uma lista para o argumento “ascending” indicando qual coluna deve ser ascendente ou descendente

- Por padrão, esse argumento para o exemplo acima possui o seguinte valor “ascending=[True, True]”

Sumarização de dados

Pandas oferece um conjunto de métodos matemáticos e estatísticos para sumarização de dados. Por exemplo, “sum”:

```
In [268]: df
Out[268]:
    one  two
a  1.40  NaN
b  7.10 -4.5
c   NaN  NaN
d  0.75 -1.3
```

```
In [269]: df.sum()
Out[269]:
one    9.25
two   -5.80
dtype: float64
```

Soma das linhas

```
In [270]: df.sum(axis="columns")
Out[270]:
a    1.40
b    2.60
c    0.00
d   -0.55
dtype: float64
```

Soma das colunas

Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```

Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```

elementos (excluídos os “Nan”)

Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```

Média aritmética

Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```

Desvio padrão

Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```



Sumarização de dados

Você pode usar o método “describe” para produzir várias sumarizações de uma só vez

```
In [276]: df.describe()
Out[276]:
              one       two
count    3.000000  2.000000
mean     3.083333 -2.900000
std      3.493685  2.262742
min      0.750000 -4.500000
25%      1.075000 -3.700000
50%      1.400000 -2.900000
75%      4.250000 -2.100000
max      7.100000 -1.300000
```

Quartis

Sumarização de dados

Method	Description
`count`	Number of non-NA values
`describe`	Compute set of summary statistics
`min, max`	Compute minimum and maximum values
`argmin, argmax`	Compute index locations (integers) at which minimum or maximum value is obtained, respectively; not available on DataFrame objects
`idxmin, idxmax`	Compute index labels at which minimum or maximum value is obtained, respectively
`quantile`	Compute sample quantile ranging from 0 to 1 (default: 0.5)
`sum`	Sum of values
`mean`	Mean of values
`median`	Arithmetic median (50% quantile) of values
`mad`	Mean absolute deviation from mean value
`prod`	Product of all values

Sumarização de dados

Method	Description
`var`	Sample variance of values
`std`	Sample standard deviation of values
`skew`	Sample skewness (third moment) of values
`kurt`	Sample kurtosis (fourth moment) of values
`cumsum`	Cumulative sum of values
`cummin`, `cummax`	Cumulative minimum or maximum of values, respectively
`cumprod`	Cumulative product of values
`diff`	Compute first arithmetic difference (useful for time series)
`pct_change`	Compute percent changes

Sumarização de dados

Para encontrar os valores únicos e realizar contagens, utilize os métodos “unique” e “value_counts”, respectivamente

```
In [290]: obj = pd.Series(["c", "a", "d", "a", "a", "b", "b", "c", "c"])
```

```
In [291]: uniques = obj.unique()

In [292]: uniques
Out[292]: array(['c', 'a', 'd', 'b'], dtype=object)
```

```
In [293]: obj.value_counts()
Out[293]:
c    3
a    3
b    2
d    1
dtype: int64
```

Sumarização de dados

Para analisar o pertencimento a conjuntos de valores de uma série ou da coluna de um DataFrame, pode-se utilizar o método “isin”

```
In [295]: obj
Out[295]:
0    c
1    a
2    d
3    a
4    a
5    b
6    b
7    c
8    c
dtype: object
```

```
In [296]: mask = obj.isin(["b", "c"])
In [297]: mask
Out[297]:
0     True
1    False
2    False
3    False
4    False
5     True
6     True
7     True
8     True
dtype: bool
```

```
In [298]: obj[mask]
Out[298]:
0    c
5    b
6    b
7    c
8    c
dtype: object
```

De forma direta, poderia ser:

- obj[obj.isin([“b”, “c”])]

Leitura de dados

Pandas disponibiliza amplas opções para importar e exportar dados

Function	Description
`read_csv`	Load delimited data from a file, URL, or file-like object; use comma as default delimiter
`read_fwf`	Read data in fixed-width column format (i.e., no delimiters)
`read_clipboard`	Variation of `read_csv` that reads data from the clipboard; useful for converting tables from web pages
`read_excel`	Read tabular data from an Excel XLS or XLSX file
`read_hdf`	Read HDF5 files written by pandas
`read_html`	Read all tables found in the given HTML document
`read_json`	Read data from a JSON (JavaScript Object Notation) string representation, file, URL, or file-like object

Leitura de dados

Pandas disponibiliza amplas opções para importar e exportar dados

Function	Description
`read_feather`	Read the Feather binary file format
`read_orc`	Read the Apache ORC binary file format
`read_parquet`	Read the Apache Parquet binary file format
`read_pickle`	Read an object stored by pandas using the Python pickle format
`read_sas`	Read a SAS dataset stored in one of the SAS system's custom storage formats
`read_spss`	Read a data file created by SPSS
`read_sql`	Read the results of a SQL query (using SQLAlchemy)
`read_sql_table`	Read a whole SQL table (using SQLAlchemy); equivalent to using a query that selects everything in that table using `read_sql`
`read_stata`	Read a dataset from Stata file format
`read_xml`	Read a table of data from an XML file

Leitura de dados usando “read_csv”

Foco na função “pd.read_csv” por ser a mais utilizada

Considere o seguinte arquivo .csv em diretório chamado “examples”

```
In [10]: !cat examples/ex1.csv
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

Leitura de dados usando “read_csv”

Foco na função “pd.read_csv” por ser a mais utilizada

Considere o seguinte arquivo .csv em diretório chamado “examples”

```
In [10]: !cat examples/ex1.csv
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

```
In [11]: df = pd.read_csv("examples/ex1.csv")

In [12]: df
Out[12]:
   a   b   c   d message
0  1   2   3   4   hello
1  5   6   7   8   world
2  9  10  11  12     foo
```

Leitura de dados usando “read_csv”

Exploraremos alguns dos mais de 50 parâmetros do “read_csv”, como a leitura de arquivos sem cabeçalho

```
In [13]: !cat examples/ex2.csv
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

Leitura de dados usando “read_csv”

Exploraremos alguns dos mais de 50 parâmetros do “read_csv”, como a leitura de arquivos sem cabeçalho

```
In [13]: !cat examples/ex2.csv
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

```
In [14]: pd.read_csv("examples/ex2.csv", header=None)
Out[14]:
   0   1   2   3      4
0  1   2   3   4  hello
1  5   6   7   8  world
2  9  10  11  12    foo
```

O parâmetro “header” informa a ausência de cabeçalho. Assim, pandas enumera as colunas

Leitura de dados usando “read_csv”

Exploraremos alguns dos mais de 50 parâmetros do “read_csv”, como a leitura de arquivos sem cabeçalho

```
In [13]: !cat examples/ex2.csv
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

```
In [14]: pd.read_csv("examples/ex2.csv", header=None)
Out[14]:
   0   1   2   3      4
0  1   2   3   4  hello
1  5   6   7   8  world
2  9  10  11  12    foo
```

```
In [15]: pd.read_csv("examples/ex2.csv", names=["a", "b", "c", "d", "message"])
Out[15]:
   a   b   c   d message
0  1   2   3   4   hello
1  5   6   7   8   world
2  9  10  11  12     foo
```

Alternativamente, é possível definir os nomes das colunas com o parâmetro “names”

Leitura de dados usando “read_csv”

Especificando o nome e o índice

```
In [16]: names = ["a", "b", "c", "d", "message"]

In [17]: pd.read_csv("examples/ex2.csv", names=names, index_col="message")
Out[17]:
         a   b   c   d
message
hello    1   2   3   4
world    5   6   7   8
foo      9  10  11  12
```

O delimitador padrão no “read_csv” é a vírgula, mas pode ser alterado para espaço, ponto e vírgula, tabulação (“\t”), etc., usando o argumento “sep”

- Por exemplo, “sep='\t'” define a tabulação como delimitador

Leitura de dados usando “read_csv”

Argument	Description
`path`	String indicating filesystem location, URL, or file-like object.
`sep` or `delimiter`	Character sequence or regular expression to use to split fields in each row.
`header`	Row number to use as column names; defaults to 0 (first row), but should be `None` if there is no header row.
`index_col`	Column numbers or names to use as the row index in the result; can be a single name/number or a list of them for a hierarchical index.
`names`	List of column names for result.
`skiprows`	Number of rows at beginning of file to ignore or list of row numbers (starting from 0) to skip.
`na_values`	Sequence of values to replace with NA. They are added to the default list unless `keep_default_na=False` is passed.
`keep_default_na`	Whether to use the default NA value list or not (`True` by default).
`columns`	Specifies which columns to read from the CSV file into the DataFrame.

Leitura de dados usando “read_csv”

Argument	Description
`comment`	Character(s) to split comments off the end of lines.
`parse_dates`	Attempt to parse data to `datetime`; `False` by default. If `True`, will attempt to parse all columns. Otherwise, can specify a list of column numbers or names to parse. If element of list is tuple or list, will combine multiple columns together and parse to date (e.g., if date/time split across two columns).
`keep_date_col`	If joining columns to parse date, keep the joined columns; `False` by default.
`converters`	Dictionary containing column number or name mapping to functions (e.g., `{"foo": f}` would apply the function `f` to all values in the `"foo"` column).
`dayfirst`	When parsing potentially ambiguous dates, treat as international format (e.g., 7/6/2012 -> June 7, 2012); `False` by default.
`date_parser`	Function to use to parse dates.
`nrows`	Number of rows to read from beginning of file (not counting the header).
`iterator`	Return a `TextFileReader` object for reading the file piecemeal. This object can also be used with the `with` statement.

Leitura de dados usando “read_csv”

Argument	Description
`chunksize`	For iteration, size of file chunks.
`skip_footer`	Number of lines to ignore at end of file.
`verbose`	Print various parsing information, like the time spent in each stage of the file conversion and memory use information.
`encoding`	Text encoding (e.g., `"utf-8` for UTF-8 encoded text). Defaults to `"utf-8"` if `None`.
`squeeze`	If the parsed data contains only one column, return a Series.
`thousands`	Separator for thousands (e.g., `","` or `"."`); default is `None`.
`decimal`	Decimal separator in numbers (e.g., `"."` or `","`); default is `"."`.
`engine`	CSV parsing and conversion engine to use; can be one of `"c"`, `"python"`, or `"pyarrow"`. The default is `"c"`, though the newer `"pyarrow"` engine can parse some files much faster. The `"python"` engine is slower but supports some features that the other engines do not.

Leitura de dados por partes

Ao lidar com arquivos muito grandes, pode ser útil ler apenas uma pequena parte ou iterar em pedaços menores

```
In [41]: result = pd.read_csv("examples/ex6.csv")

In [42]: result
Out[42]:
           one       two     three      four key
0     0.467976 -0.038649 -0.295344 -1.824726   L
1    -0.358893  1.404453  0.704965 -0.200638   B
2    -0.501840  0.659254 -0.421691 -0.057688   G
3     0.204886  1.074134  1.388361 -0.982404   R
4     0.354628 -0.133116  0.283763 -0.837063   Q
...        ...       ...       ...       ...  ..
9995  2.311896 -0.417070 -1.409599 -0.515821   L
9996 -0.479893 -0.650419  0.745152 -0.646038   E
9997  0.523331  0.787112  0.486066  1.093156   K
9998 -0.362559  0.598894 -1.843201  0.887292   G
9999 -0.096376 -1.012999 -0.657431 -0.573315   0
[10000 rows x 5 columns]
```

Leitura de dados por partes

É possível ler um número qualquer de linhas usando o argumento “nrows”

```
In [43]: pd.read_csv("examples/ex6.csv", nrows=5)
Out[43]:
        one       two     three      four key
0  0.467976 -0.038649 -0.295344 -1.824726   L
1 -0.358893  1.404453  0.704965 -0.200638   B
2 -0.501840  0.659254 -0.421691 -0.057688   G
3  0.204886  1.074134  1.388361 -0.982404   R
4  0.354628 -0.133116  0.283763 -0.837063   Q
```

Leitura de dados por partes

Também é possível ler todo o arquivo em lotes usando o argumento “chuncksize”

```
In [44]: chunker = pd.read_csv("examples/ex6.csv", chunksize=1000)

In [45]: type(chunker)
Out[45]: pandas.io.parsers.readers.TextFileReader
```

O objeto “TextFileReader” retornado por “read_csv” permite iterar sobre partes do arquivo de acordo com o chunksize

- Você pode fazer processamentos ou contagens em todos os arquivos

Leitura de dados por partes

Por exemplo, pode-se iterar sobre o DataFrame agregando a contagem de valores na coluna "key" desta forma

```
chunker = pd.read_csv("examples/ex6.csv", chunksize=1000)

tot = pd.Series([], dtype='int64')
for piece in chunker:
    tot = tot.add(piece["key"].value_counts(), fill_value=0)

tot = tot.sort_values(ascending=False)
```

O método “add” soma contagens de valores únicos de cada pedaço à série.
“fill_value=0” garante que novos valores sejam inicializados como 0

Leitura de dados por partes

Por exemplo, pode-se iterar sobre o DataFrame agregando a contagem de valores na coluna "key" desta forma

```
chunker = pd.read_csv("examples/ex6.csv", chunksize=1000)

tot = pd.Series([], dtype='int64')
for piece in chunker:
    tot = tot.add(piece["key"].value_counts(), fill_value=0)

tot = tot.sort_values(ascending=False)
```

```
In [47]: tot[:10]
Out[47]:
E    368.0
X    364.0
L    346.0
O    343.0
Q    340.0
M    338.0
J    337.0
F    335.0
K    334.0
H    330.0
dtype: float64
```

O método “add” soma contagens de valores únicos de cada pedaço à série. “fill_value=0” garante que novos valores sejam inicializados como 0

Escrita de dados em texto

Os dados podem ser exportados para um formato delimitado usando o método to_csv do DataFrame

```
In [48]: data = pd.read_csv("examples/ex5.csv")

In [49]: data
Out[49]:
  something  a   b     c   d message
0       one  1   2   3.0   4     NaN
1       two  5   6   NaN   8   world
2     three  9  10  11.0  12     foo
```

```
In [50]: data.to_csv("examples/out.csv")

In [51]: !cat examples/out.csv
,something,a,b,c,d,message
0,one,1,2,3.0,4,
1,two,5,6,,8,world
2,three,9,10,11.0,12,foo
```

Outros separadores também podem ser usados: espaço, ponto e vírgula, etc

Use “index=False” ao salvar DataFrames quando o índice for desnecessário, evitando a criação de colunas extras no arquivo

Manipulação de Dados Ausentes

A ausência de dados é um problema frequente no processo de análise de dados e pode representar ameaças durante o processo

Por exemplo, as sumarizações realizadas anteriormente desconsideram os dados faltantes

Pandas usa “NaN” (Not a Number) para representar dados ausentes

- Valores internos None também são tratados como “NaN”

Existem várias maneira de verificar a presença desses valores

Verificação de Valores Ausentes

Verificando a presença de dados ausentes como “NaN” e None

```
In [14]: float_data = pd.Series([1.2, -3.5, np.nan, 0])

In [15]: float_data
Out[15]:
0    1.2
1   -3.5
2    NaN
3    0.0
dtype: float64
```

```
In [16]: float_data.isna()
Out[16]:
0    False
1    False
2     True
3    False
dtype: bool
```

Verificação de Valores Ausentes

Verificando a presença de dados ausentes como “NaN” e None

```
In [14]: float_data = pd.Series([1.2, -3.5, np.nan, 0])

In [15]: float_data
Out[15]:
0    1.2
1   -3.5
2    NaN
3    0.0
dtype: float64
```

```
In [16]: float_data.isna()
Out[16]:
0    False
1    False
2     True
3    False
dtype: bool
```

```
In [17]: string_data = pd.Series(["aardvark", np.nan, None, "avocado"])

In [18]: string_data
Out[18]:
0    aardvark
1         NaN
2        None
3     avocado
dtype: object
```

```
In [19]: string_data.isna()
Out[19]:
0    False
1     True
2     True
3    False
dtype: bool
```

Verificação de Valores Ausentes

"None": objeto em Python usado para representar a ausência de valor, usado em qualquer tipo de dados

"*np.nan*": representação do NumPy para um valor de ponto flutuante "não número" (NaN), específico para operações numéricas

Method	Description
`dropna`	Filter axis labels based on whether values for each label have missing data, with varying thresholds for how much missing data to tolerate.
`fillna`	Fill in missing data with some value or using an interpolation method such as `"ffill"` or `"bfill"`.
`isna`	Return Boolean values indicating which values are missing/NA.
`notna`	Negation of `isna`, returns `True` for non-NA values and `False` for NA values.

Remoção de Valores Ausentes

Para filtrar dados ausentes, pode-se utilizar o método “dropna( )”

```
In [23]: data = pd.Series([1, np.nan, 3.5, np.nan, 7])

In [24]: data.dropna()
Out[24]:
0    1.0
2    3.5
4    7.0
dtype: float64
```

Existem diferentes maneiras de remover dados ausente em DataFrames, cada uma com regras específicas

Remoção de Valores Ausentes

Alguns exemplos:

```
In [27]: data
Out[27]:
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
```

Remoção de Valores Ausentes

Alguns exemplos:

```
In [27]: data
Out[27]:
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
```

```
In [28]: data.dropna()
Out[28]:
     0    1    2
0  1.0  6.5  3.0
```

Remove **qualquer** linha
que tiver valor ausente

Remoção de Valores Ausentes

Alguns exemplos:

```
In [27]: data
Out[27]:
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  6.5  3.0
```

```
In [28]: data.dropna()
Out[28]:
     0    1    2
0  1.0  6.5  3.0
```

Remove **qualquer** linha que tiver valor ausente

```
In [29]: data.dropna(how="all")
Out[29]:
     0    1    2
0  1.0  6.5  3.0
1  1.0  NaN  NaN
3  NaN  6.5  3.0
```

Remove linhas que contenham **somente** valores ausentes

Você pode usar o argumento “axis” para fazer o mesmo por coluna

Remoção de Valores Ausentes

É possível definir um limiar para excluir linhas/colunas com o argumento “thresh”

```
In [33]: df = pd.DataFrame(np.random.standard_normal((7, 3)))

In [34]: df.iloc[:4, 1] = np.nan

In [35]: df.iloc[:2, 2] = np.nan

In [36]: df
Out[36]:
          0         1         2
0 -0.204708       NaN       NaN
1 -0.555730       NaN       NaN
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

Remoção de Valores Ausentes

É possível definir um limiar para excluir linhas/colunas com o argumento “thresh”

```
In [33]: df = pd.DataFrame(np.random.standard_normal((7, 3)))

In [34]: df.iloc[:4, 1] = np.nan

In [35]: df.iloc[:2, 2] = np.nan

In [36]: df
Out[36]:
          0         1         2
0 -0.204708       NaN       NaN
1 -0.555730       NaN       NaN
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

```
In [38]: df.dropna(thresh=2)
Out[38]:
          0         1         2
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

Remove linhas apenas com mais do que 2 (thresh) valores ausentes

Preenchimento de Dados Ausentes

Use o método “fillna” para preencher valores ausentes e preservar dados valiosos sem descartá-los

```
In [36]: df
Out[36]:
          0         1         2
0 -0.204708       NaN       NaN
1 -0.555730       NaN       NaN
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

Preenchimento de Dados Ausentes

Use o método “fillna” para preencher valores ausentes e preservar dados valiosos sem descartá-los

```
In [36]: df
Out[36]:
          0         1         2
0 -0.204708       NaN       NaN
1 -0.555730       NaN       NaN
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

```
In [39]: df.fillna(0)
Out[39]:
          0         1         2
0 -0.204708  0.000000  0.000000
1 -0.555730  0.000000  0.000000
2  0.092908  0.000000  0.769023
3  1.246435  0.000000 -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

Preenchimento de Dados Ausentes

Também é possível passar um dicionário com um valor diferente de preenchimento para cada coluna

```
In [36]: df
Out[36]:
          0         1         2
0 -0.204708       NaN       NaN
1 -0.555730       NaN       NaN
2  0.092908       NaN  0.769023
3  1.246435       NaN -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

```
In [40]: df.fillna({1: 0.5, 2: 0})
Out[40]:
          0         1         2
0 -0.204708  0.500000  0.000000
1 -0.555730  0.500000  0.000000
2  0.092908  0.500000  0.769023
3  1.246435  0.500000 -1.296221
4  0.274992  0.228913  1.352917
5  0.886429 -2.001637 -0.371843
6  1.669025 -0.438570 -0.539741
```

Remoção de Duplicidades

Para identificar e tratar linhas duplicadas em um DataFrame, existem métodos específicos disponíveis

```
In [50]: data
Out[50]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
6  two   4
```

Remoção de Duplicidades

Para identificar e tratar linhas duplicadas em um DataFrame, existem métodos específicos disponíveis

```
In [50]: data
Out[50]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
6  two   4
```

Remoção de Duplicidades

Para identificar e tratar linhas duplicadas em um DataFrame, existem métodos específicos disponíveis

```
In [50]: data
Out[50]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
6  two   4
```

```
In [51]: data.duplicated()
Out[51]:
0    False
1    False
2    False
3    False
4    False
5    False
6     True
dtype: bool
```

“duplicated”

```
In [52]: data.drop_duplicates()
Out[52]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
```

“drop_duplicates”

Remoção de Duplicidades

Para um controle mais preciso sobre duplicatas, é possível especificar subconjuntos de colunas e selecionar qual duplicata manter

```
In [53]: data["v1"] = range(7)

In [54]: data
Out[54]:
    k1  k2  v1
0  one   1   0
1  two   1   1
2  one   2   2
3  two   3   3
4  one   3   4
5  two   4   5
6  two   4   6
```

```
In [55]: data.drop_duplicates(subset=["k1"])
Out[55]:
    k1  k2  v1
0  one   1   0
1  two   1   1
```

Por padrão, todas as colunas são consideradas. Neste caso, apenas “k1”

```
In [56]: data.drop_duplicates(["k1", "k2"], keep="last")
Out[56]:
    k1  k2  v1
0  one   1   0
1  two   1   1
2  one   2   2
3  two   3   3
4  one   3   4
6  two   4   6
```

Mas você pode especificar um subconjunto delas e também especificar qual dado manter de acordo com a ordem de ocorrência: “last”, “first”

Transformação de Dados com Map

O método “map” em pandas transforma e substitui valores usando um dicionário, função ou Series

Por exemplo, é possível criar uma nova coluna em data de acordo com o valor da coluna “food”

```
meat_to_animal = {
  "bacon": "pig",
  "pulled pork": "pig",
  "pastrami": "cow",
  "corned beef": "cow",
  "honey ham": "pig",
  "nova lox": "salmon"
}
```

```
In [58]: data
Out[58]:
          food  ounces
0        bacon     4.0
1  pulled pork     3.0
2        bacon    12.0
3     pastrami     6.0
4  corned beef     7.5
5        bacon     8.0
6     pastrami     3.0
7    honey ham     5.0
8     nova lox     6.0
```

Transformação de Dados com Map

Ao usar uma função com “map”, ela é aplicada elemento a elemento, transformando cada valor individualmente sem sumarizar linhas ou colunas

```
In [62]: def get_animal(x):
   ....:     return meat_to_animal[x]

In [63]: data["food"].map(get_animal)
Out[63]:
0       pig
1       pig
2       pig
3       cow
4       cow
5       pig
6       cow
7       pig
8    salmon
Name: food, dtype: object
```

Transformação de Dados com Map

Resultado:

```
In [60]: data["animal"] = data["food"].map(meat_to_animal)

In [61]: data
Out[61]:
          food  ounces  animal
0        bacon     4.0     pig
1  pulled pork     3.0     pig
2        bacon    12.0     pig
3     pastrami     6.0     cow
4  corned beef     7.5     cow
5        bacon     8.0     pig
6     pastrami     3.0     cow
7    honey ham     5.0     pig
8     nova lox     6.0  salmon
```

Ao usar “map” com uma Series, os valores são substituídos com base no alinhamento dos índices entre a Series original e o dicionário fornecido

Substituição de Valores

Frequentemente, é preciso lidar com substituição de valores

Pandas oferece um método, chamado “replace”, que fornece uma forma mais rápida e flexível

```
In [65]: data
Out[65]:
0       1.0
1    -999.0
2       2.0
3    -999.0
4   -1000.0
5       3.0
dtype: float64
```

```
In [66]: data.replace(-999, np.nan)
Out[66]:
0       1.0
1       NaN
2       2.0
3       NaN
4   -1000.0
5       3.0
dtype: float64
```

Substitui um único valor

Substituição de Valores

Frequentemente, é preciso lidar com substituição de valores

Pandas oferece um método, chamado “replace”, que fornece uma forma mais rápida e flexível

```
In [65]: data
Out[65]:
0       1.0
1    -999.0
2       2.0
3    -999.0
4   -1000.0
5       3.0
dtype: float64
```

```
In [67]: data.replace([-999, -1000], np.nan)
Out[67]:
0    1.0
1    NaN
2    2.0
3    NaN
4    NaN
5    3.0
dtype: float64
```

Substituição de múltiplos valores

Substituição de Valores

Frequentemente, é preciso lidar com substituição de valores

Pandas oferece um método, chamado “replace”, que fornece uma forma mais rápida e flexível

```
In [65]: data
Out[65]:
0       1.0
1    -999.0
2       2.0
3    -999.0
4   -1000.0
5       3.0
dtype: float64
```

```
In [68]: data.replace([-999, -1000], [np.nan, 0])
Out[68]:
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

Substituição de múltiplos valores por múltiplos valores

Substituição de Valores

Frequentemente, é preciso lidar com substituição de valores

Pandas oferece um método, chamado “replace”, que fornece uma forma mais rápida e flexível

```
In [65]: data
Out[65]:
0       1.0
1    -999.0
2       2.0
3    -999.0
4   -1000.0
5       3.0
dtype: float64
```

```
In [69]: data.replace({-999: np.nan, -1000: 0})
Out[69]:
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
```

Substituição de múltiplos valores por múltiplos valores usando um dicionário

Manipulação de Strings

A maioria das operações em strings são simplificadas com métodos internos do objeto string em Python

Para busca de padrões e manipulações de texto mais complexas

- Expressões regulares são, tipicamente, usadas

O Pandas complementa as opções de manipulação de strings e aplicações expressões regulares

Métodos Internos de Strings do Python

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [152]: val.split(",")
Out[152]: ['a', 'b', '  guido']
```

Separa uma string de acordo com um critério. Por ex., “,”

Métodos Internos de Strings do Python

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [152]: val.split(",")
Out[152]: ['a', 'b', '  guido']
```

Separa uma string de acordo com um critério. Por ex., “,”

```
In [153]: pieces = [x.strip() for x in val.split(",")]

In [154]: pieces
Out[154]: ['a', 'b', 'guido']
```

Geralmente, “split” é combinado com “strip” para remoção de espaços em branco

Métodos Internos de Strings do Python

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [152]: val.split(",")
Out[152]: ['a', 'b', '  guido']
```

Separa uma string de acordo com um critério. Por ex., “,”

```
In [153]: pieces = [x.strip() for x in val.split(",")]

In [154]: pieces
Out[154]: ['a', 'b', 'guido']
```

Geralmente, “split” é combinado com “strip” para remoção de espaços em branco

```
In [155]: first, second, third = pieces

In [156]: first + "::" + second + "::" + third
Out[156]: 'a::b::guido'
```

Concatenação usando soma de strings

Métodos Internos de Strings do Python

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [152]: val.split(",")
Out[152]: ['a', 'b', '  guido']
```

Separa uma string de acordo com um critério. Por ex., “,”

```
In [153]: pieces = [x.strip() for x in val.split(",")]

In [154]: pieces
Out[154]: ['a', 'b', 'guido']
```

Geralmente, “split” é combinado com “strip” para remoção de espaços em branco

```
In [155]: first, second, third = pieces

In [156]: first + "::" + second + "::" + third
Out[156]: 'a::b::guido'
```

Concatenação usando soma de strings

```
In [157]: "::".join(pieces)
Out[157]: 'a::b::guido'
```

Mas poderia ser com o método “join”

Manipulação de Strings

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [158]: "guido" in val
Out[158]: True
```



Retorna True or False ao buscar por uma substring

Manipulação de Strings

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [158]: "guido" in val
Out[158]: True

In [159]: val.index(",")
Out[159]: 1
```

Retorna a posição, mas poderá gerar uma exceção se o valor não existir ao buscar por uma substring

Manipulação de Strings

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [158]: "guido" in val
Out[158]: True

In [159]: val.index(",")
Out[159]: 1

In [160]: val.find(":")
Out[160]: -1
```

Retorna a posição e, caso o valor não exista, -1 ao buscar por uma substring

Manipulação de Strings

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [158]: "guido" in val
Out[158]: True

In [159]: val.index(",")
Out[159]: 1

In [160]: val.find(":")
Out[160]: -1

In [162]: val.count(",")
Out[162]: 2

In [163]: val.replace(",", "::")
Out[163]: 'a::b::  guido'

In [164]: val.replace(",", "")
Out[164]: 'ab  guido'
```

Retorna o número de ocorrências de uma dada substring

Manipulação de Strings

Alguns métodos úteis:

```
In [151]: val = "a,b,  guido"

In [158]: "guido" in val
Out[158]: True

In [159]: val.index(",")
Out[159]: 1

In [160]: val.find(":")
Out[160]: -1

In [162]: val.count(",")
Out[162]: 2

In [163]: val.replace(",", "::")
Out[163]: 'a::b::  guido'

In [164]: val.replace(",", "")
Out[164]: 'ab  guido'
```

Substitui a ocorrência de uma data substring

Manipulação de Strings

Method	Description
`count`	Return the number of nonoverlapping occurrences of substring in the string
`endswith`	Return `True` if string ends with suffix
`startswith`	Return `True` if string starts with prefix
`join`	Use string as delimiter for concatenating a sequence of other strings
`index`	Return starting index of the first occurrence of passed substring if found in the string; otherwise, raises `ValueError` if not found
`find`	Return position of first character of *first* occurrence of substring in the string; like `index`, but returns –1 if not found
`rfind`	Return position of first character of *last* occurrence of substring in the string; returns –1 if not found

Manipulação de Strings

Method	Description
`replace`	Replace occurrences of string with another string
`strip, rstrip, lstrip`	Trim whitespace, including newlines on both sides, on the right side, or on the left side, respectively
`split`	Break string into list of substrings using passed delimiter
`lower`	Convert alphabet characters to lowercase
`upper`	Convert alphabet characters to uppercase
`casefold`	Convert characters to lowercase, and convert any region-specific variable character combinations to a common comparable form
`ljust, rjust`	Left justify or right justify, respectively; pad opposite side of string with spaces (or some other fill character) to return a string with a minimum width

Funções de Strings do Pandas

```
In [184]: data = {"Dave": "dave@google.com", "Steve": "steve@gmail.com",
   .....:         "Rob": "rob@gmail.com", "Wes": np.nan}

In [185]: data = pd.Series(data)

In [186]: data
Out[186]:
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                  NaN
dtype: object
```

Funções de Strings do Pandas

```
In [184]: data = {"Dave": "dave@google.com", "Steve": "steve@gmail.com",
   .....:         "Rob": "rob@gmail.com", "Wes": np.nan}

In [185]: data = pd.Series(data)

In [186]: data
Out[186]:
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                  NaN
dtype: object
```

```
In [188]: data.str.contains("gmail")
Out[188]:
Dave     False
Steve     True
Rob       True
Wes        NaN
dtype: object
```

Verifica a ocorrência de uma substring

Funções de Strings do Pandas

```
In [184]: data = {"Dave": "dave@google.com", "Steve": "steve@gmail.com",
   .....:         "Rob": "rob@gmail.com", "Wes": np.nan}

In [185]: data = pd.Series(data)

In [186]: data
Out[186]:
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                  NaN
dtype: object
```

```
In [189]: data_as_string_ext = data.astype('string')

In [190]: data_as_string_ext
Out[190]:
Dave     dave@google.com
Steve    steve@gmail.com
Rob        rob@gmail.com
Wes                 <NA>
dtype: string
```

Converte uma coluna para o tipo string oferecendo uma lista enorme de métodos

Funções de Strings do Pandas

Method	Description
`cat`	Concatenate strings element-wise with optional delimiter
`contains`	Return Boolean array if each string contains pattern/regex
`count`	Count occurrences of pattern
`extract`	Use a regular expression with groups to extract one or more strings from a Series of strings; the result will be a DataFrame with one column per group
`endswith`	Equivalent to `x.endswith(pattern)` for each element
`startswith`	Equivalent to `x.startswith(pattern)` for each element
`findall`	Compute list of all occurrences of pattern/regex for each string

Funções de Strings do Pandas

Method	Description
`get`	Index into each element (retrieve *i*-th element)
`isalnum`	Equivalent to built-in `str.alnum`
`isalpha`	Equivalent to built-in `str.isalpha`
`isdecimal`	Equivalent to built-in `str.isdecimal`
`isdigit`	Equivalent to built-in `str.isdigit`
`islower`	Equivalent to built-in `str.islower`
`isnumeric`	Equivalent to built-in `str.isnumeric`
`isupper`	Equivalent to built-in `str.isupper`
`join`	Join strings in each element of the Series with passed separator

Funções de Strings do Pandas

Method	Description
`len`	Compute length of each string
`lower, upper`	Convert cases; equivalent to `x.lower()` or `x.upper()` for each element
`match`	Use `re.match` with the passed regular expression on each element, returning `True` or `False` whether it matches
`pad`	Add whitespace to left, right, or both sides of strings
`center`	Equivalent to `pad(side="both")`
`repeat`	Duplicate values (e.g., `s.str.repeat(3)` is equivalent to `x * 3` for each string)

Funções de Strings do Pandas

Method	Description
`replace`	Replace occurrences of pattern/regex with some other string
`slice`	Slice each string in the Series
`split`	Split strings on delimiter or regular expression
`strip`	Trim whitespace from both sides, including newlines
`rstrip`	Trim whitespace on right side
`lstrip`	Trim whitespace on left side

Combinação e Mesclagem de Dados

Os dados contidos em objetos pandas podem ser combinados usando

- “pandas.merge”: Conecta linhas de um data frame com uma ou mais chaves
- “pandas.concat”: Concatena objetos ao longo de um eixo
- “pandas.join”: Faz junções de DataFrames por índice

Essas funções do pandas possuem uma infinidade de recursos

- Mas seremos breve

Pandas.merge

Considere os seguintes DataFrames:

```
In [40]: df1
Out[40]:
  key  data1
0   b      0
1   b      1
2   a      2
3   c      3
4   a      4
5   a      5
6   b      6
```

```
In [41]: df2
Out[41]:
  key  data2
0   a      0
1   b      1
2   d      2
```

```
In [43]: pd.merge(df1, df2, on="key")
Out[43]:
  key  data1  data2
0   b      0      1
1   b      1      1
2   b      6      1
3   a      2      0
4   a      4      0
5   a      5      0
```

Usando a função pd.merge é possível fazer uma junção N:1

Pandas.merge

Considere os seguintes DataFrames:

```
In [40]: df1
Out[40]:
   key  data1
0    b      0
1    b      1
2    a      2
3    c      3
4    a      4
5    a      5
6    b      6
```

```
In [41]: df2
Out[41]:
   key  data2
0    a      0
1    b      1
2    d      2
```

```
In [43]: pd.merge(df1, df2, on="key")
Out[43]:
   key  data1  data2
0    b      0      1
1    b      1      1
2    b      6      1
3    a      2      0
4    a      4      0
5    a      5      0
```

Usando a função pd.merge é possível fazer uma junção N:1

Pandas.merge

Outras opções de junções são:

Option	Behavior
`how="inner"`	Use only the key combinations observed in both tables
`how="left"`	Use all key combinations found in the left table
`how="right"`	Use all key combinations found in the right table
`how="outer"`	Use all key combinations observed in both tables together

Também é possível usar os índices com o argumento (left_index e right_index)

Pandas.join

Para simplificar a mesclagem pelo índice, o DataFrame contém um método que pode ser usado para combinar objetos com índices iguais

Em comparação com a função “pd.merge”, DataFrames possui o método “pd.join” executa uma junção à esquerda baseada na chave da junção

- Permite combinar múltiplos DataFrames simultaneamente, aumentando a eficiência
- Embora focado em índices, também permite junções por colunas chave

Pandas.join

Exemplo:

```
In [75]: left2
Out[75]:
   Ohio  Nevada
a     1       2
c     3       4
e     5       6
```

```
In [76]: right2
Out[76]:
   Missouri  Alabama
b         7        8
c         9       10
d        11       12
e        13       14
```

Pandas.join

Exemplo:

```
In [75]: left2
Out[75]:
   Ohio  Nevada
a     1       2
c     3       4
e     5       6
```

```
In [76]: right2
Out[76]:
   Missouri  Alabama
b         7        8
c         9       10
d        11       12
e        13       14
```

Junção pelo índice usando pd.merge:

```
In [77]: pd.merge(left2, right2, how="outer", left_index=True, right_index=True)
Out[77]:
   Ohio  Nevada  Missouri  Alabama
a     1       2      <NA>     <NA>
b  <NA>    <NA>         7        8
c     3       4         9       10
d  <NA>    <NA>        11       12
e     5       6        13       14
```

Pandas.join

Exemplo:

```
In [75]: left2
Out[75]:
   Ohio  Nevada
a     1       2
c     3       4
e     5       6
```

```
In [76]: right2
Out[76]:
   Missouri  Alabama
b         7        8
c         9       10
d        11       12
e        13       14
```

Junção pelo índice usando pd.merge:

```
In [77]: pd.merge(left2, right2, how="outer", left_index=True, right_index=True)
Out[77]:
   Ohio  Nevada  Missouri  Alabama
a     1       2      <NA>     <NA>
b  <NA>    <NA>         7        8
c     3       4         9       10
d  <NA>    <NA>        11       12
e     5       6        13       14
```

Pandas.join

Exemplo:

```
In [75]: left2
Out[75]:
   Ohio  Nevada
a     1       2
c     3       4
e     5       6
```

```
In [76]: right2
Out[76]:
   Missouri  Alabama
b         7        8
c         9       10
d        11       12
e        13       14
```

Junção pelo índice usando pd.merge:

```
In [77]: pd.merge(left2, right2, how="outer", left_index=True, right_index=True)
Out[77]:
   Ohio  Nevada  Missouri  Alabama
a     1       2      <NA>     <NA>
b  <NA>    <NA>         7        8
c     3       4         9       10
d  <NA>    <NA>        11       12
e     5       6        13       14
```

Pandas.join

Exemplo:

```
In [75]: left2
Out[75]:
   Ohio  Nevada
a     1       2
c     3       4
e     5       6
```

```
In [76]: right2
Out[76]:
   Missouri  Alabama
b         7        8
c         9       10
d        11       12
e        13       14
```

Junção pelo índice usando df.join é direta e mais rápida:

```
In [78]: left2.join(right2, how="outer")
Out[78]:
   Ohio  Nevada  Missouri  Alabama
a     1       2      <NA>     <NA>
b  <NA>    <NA>         7        8
c     3       4         9       10
d  <NA>    <NA>        11       12
e     5       6        13       14
```

Pandas.concat

A função “concat” do pandas permite concatenar objetos pandas

Exemplo:

```
In [90]: s1
Out[90]:
a    0
b    1
dtype: Int64
```

```
In [91]: s2
Out[91]:
c    2
d    3
e    4
dtype: Int64
```

```
In [92]: s3
Out[92]:
f    5
g    6
dtype: Int64
```

Pandas.concat

A função “concat” do pandas permite concatenar objetos pandas

Exemplo:

```
In [90]: s1
Out[90]:
a    0
b    1
dtype: Int64
```

```
In [91]: s2
Out[91]:
c    2
d    3
e    4
dtype: Int64
```

```
In [92]: s3
Out[92]:
f    5
g    6
dtype: Int64
```

```
In [93]: pd.concat([s1, s2, s3])
Out[93]:
a    0
b    1
c    2
d    3
e    4
f    5
g    6
dtype: Int64
```

Pandas.concat

Por padrão, “concat” utiliza o eixo axis="index", mas pode-se especificar axis="columns" para alternar a direção

```
In [94]: pd.concat([s1, s2, s3], axis="columns")
Out[94]:
      0     1     2
a     0  <NA>  <NA>
b     1  <NA>  <NA>
c  <NA>     2  <NA>
d  <NA>     3  <NA>
e  <NA>     4  <NA>
f  <NA>  <NA>     5
g  <NA>  <NA>     6
```

Pandas.concat

Argument	Description
`objs`	List or dictionary of pandas objects to be concatenated; this is the only required argument
`axis`	Axis to concatenate along; defaults to concatenating along rows (`axis="index"`)
`join`	Either `"inner"` or `"outer"` (`"outer"` by default); whether to intersect (inner) or union (outer) indexes along the other axes
`keys`	Values to associate with objects being concatenated, forming a hierarchical index along the concatenation axis; can be a list or array of arbitrary values, an array of tuples, or a list of arrays (if multiple-level arrays passed in `levels`)

Pandas.concat

Argument	Description
`keys`	Values to associate with objects being concatenated, forming a hierarchical index along the concatenation axis; can be a list or array of arbitrary values, an array of tuples, or a list of arrays (if multiple-level arrays passed in `levels`)
`levels`	Specific indexes to use as hierarchical index level or levels if keys passed
`names`	Names for created hierarchical levels if `keys` and/or `levels` passed
`verify_integrity`	Check new axis in concatenated object for duplicates and raise ar exception if so; by default (`False`) allows duplicates
`ignore_index`	Do not preserve indexes along concatenation `axis`, instead produce a new `range(total_length)` index

Agregação de Dados

A agregação de dados e aplicação de alguma função a cada grupo agregado é bastante útil durante a análise de dados

- É possível aplicar funções como soma, média a grupos de dados, enriquecendo a análise

O Pandas oferece uma interface que permite esmiuçar e resumir conjuntos de dados de forma natural

Possui maior flexibilidade do que bancos de dados relacionais, facilitando análises complexas

Dividir, Aplicar e Combinar

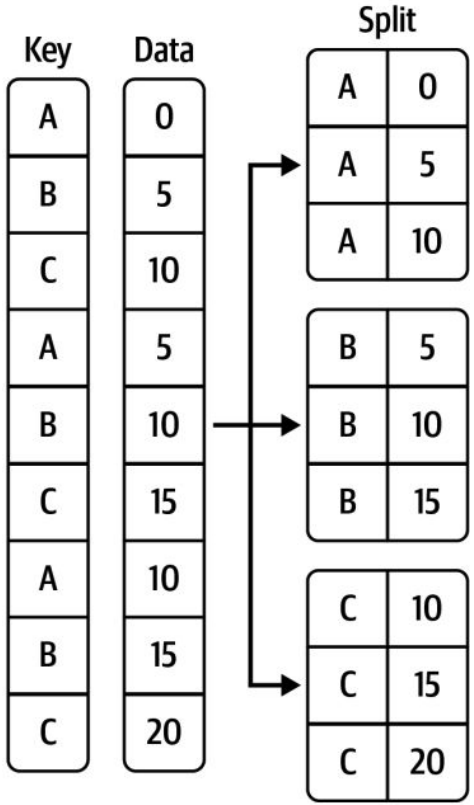
Processo dividido em três estágios:

Key	Data
A	0
B	5
C	10
A	5
B	10
C	15
A	10
B	15
C	20

Considere dados em um objeto Pandas (Serie ou DataFrame) com uma chave

Dividir, Aplicar e Combinar

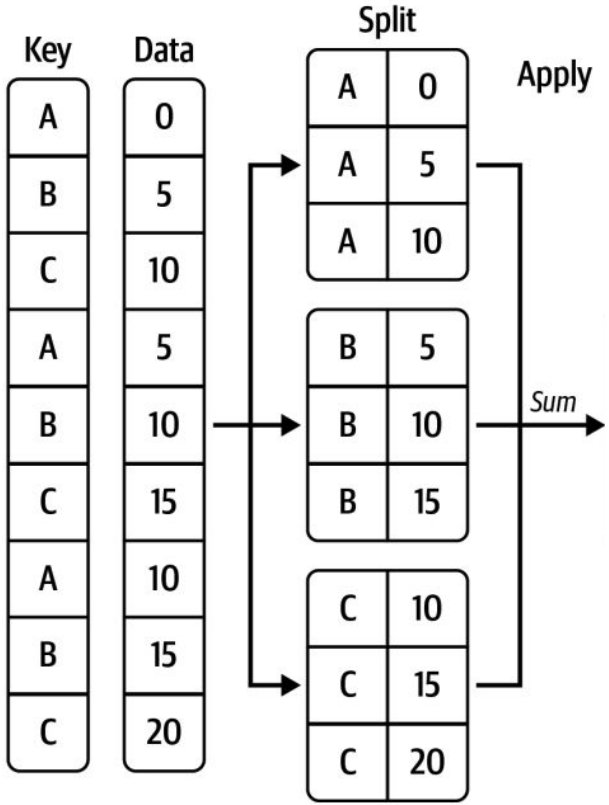
Processo dividido em três estágios:



1º Estágio: *Dividir* os dados em grupos de acordo com uma ou mais chaves fornecidas!

Dividir, Aplicar e Combinar

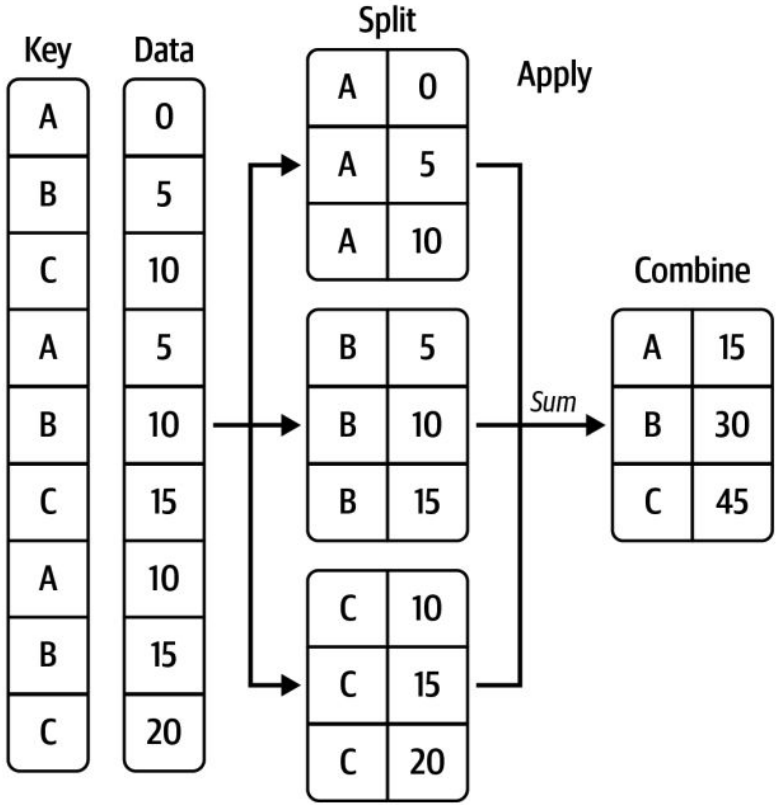
Processo dividido em três estágios:



2º Estágio: *Aplicar* a cada grupo, produzindo um novo valor

Dividir, Aplicar e Combinar

Processo dividido em três estágios:



3º Estágio: *Combinar* os resultados de todas as aplicações em um objeto de resultado

Utilizando o Groupby

Considere o seguinte DataFrame

```
In [15]: df
Out[15]:
   key1  key2     data1     data2
0     a     1 -0.204708  0.281746
1     a     2  0.478943  0.769023
2  None     1 -0.519439  1.246435
3     b     2 -0.555730  1.007189
4     b     1  1.965781 -1.296221
5     a  <NA>  1.393406  0.274992
6  None     1  0.092908  0.228913
```

Exemplo de como obter a média da coluna “data1” usando os rótulos “de key1”

```
In [16]: grouped = df["data1"].groupby(df["key1"])

In [17]: grouped
Out[17]: <pandas.core.groupby.generic.SeriesGroupBy object at 0x7fa9270e0a00>
```

Utilizando o Groupby

O método “Groupby” retorna um objeto especial que está pronto para receber a aplicação de alguma função

Para calcular a média, basta chamar o método “mean”

```
In [18]: grouped.mean()
Out[18]:
key1
a    0.555881
b    0.705025
Name: data1, dtype: float64
```

Note que o índice resultante tem o nome da chave

Utilizando o Groupby

Múltiplas chaves podem usadas

```
In [19]: means = df["data1"].groupby([df["key1"], df["key2"]]).mean()

In [20]: means
Out[20]:
key1  key2
a     1      -0.204708
      2       0.478943
b     1       1.965781
      2      -0.555730
Name: data1, dtype: float64
```

Também é possível aplicar a própria função usando o método “apply”

Utilizando o Groupby

Um método geralmente útil é o “size” que retorna o tamanho dos grupos. O argumento “dropna” permite tratar valores ausentes

```
In [28]: df.groupby(["key1", "key2"]).size()
Out[28]:
key1  key2
a     1       1
      2       1
b     1       1
      2       1
dtype: int64
```

Valores ausentes não são considerados

```
In [29]: df.groupby("key1", dropna=False).size()
Out[29]:
key1
a      3
b      2
NaN    2
dtype: int64
```

Valores ausentes são considerados

Utilizando o Groupby

Uma função semelhante de grupo com finalidade semelhante é a “count”

```
In [31]: df.groupby("key1").count()
Out[31]:
      key2  data1  data2
key1
a        2      3      3
b        2      2      2
```

Automaticamente, “count” não considera valores não nulos

Seleção de Colunas

É possível selecionar um subconjuntos de colunas após o agrupamento e então aplicar funções específicas

```
In [38]: df.groupby(["key1", "key2"])[["data2"]].mean()
Out[38]:
              data2
key1 key2
a    1     0.281746
     2     0.769023
b    1    -1.296221
     2     1.007189
```

Também é possível limitar as colunas do agrupamento antes de aplicá-lo fazendo uma filtragem no DataFrame: df[df[[Colunas]].groupby...

Agrupamento Com Funções

Pode-se agrupar dados com funções do Python: cada valor de índice é processado pela função, cujo retorno define os nomes dos grupos

Agrupamento Com Funções

Pode-se agrupar dados com funções do Python: cada valor de índice é processado pela função, cujo retorno define os nomes dos grupos

Por exemplo:

```
In [44]: people
Out[44]:
               a         b         c         d         e
Joe     1.352917  0.886429 -2.001637 -0.371843  1.669025
Steve  -0.438570 -0.539741  0.476985  3.248944 -1.021228
Wanda  -0.577087       NaN       NaN  0.523772  0.000940
Jill    1.343810 -0.713544 -0.831154 -2.370232 -1.860761
Trey   -0.860757  0.560145 -1.265934  0.119827 -1.063512
```

Agrupamento Com Funções

Pode-se agrupar dados com funções do Python: cada valor de índice é processado pela função, cujo retorno define os nomes dos grupos

Por exemplo:

```
In [44]: people
Out[44]:
              a         b         c         d         e
Joe    1.352917  0.886429 -2.001637 -0.371843  1.669025
Steve -0.438570 -0.539741  0.476985  3.248944 -1.021228
Wanda -0.577087       NaN       NaN  0.523772  0.000940
Jill   1.343810 -0.713544 -0.831154 -2.370232 -1.860761
Trey  -0.860757  0.560145 -1.265934  0.119827 -1.063512
```

Para agrupá-lo pelo tamanho dos nomes. Note que o índice é o tamanho, retorno da função "len"

```
In [51]: people.groupby(len).sum()
Out[51]:
          a         b         c         d         e
3  1.352917  0.886429 -2.001637 -0.371843  1.669025
4  0.483052 -0.153399 -2.097088 -2.250405 -2.924273
5 -1.015657 -0.539741  0.476985  3.772716 -1.020287
```

Resumindo Groupby

Function name	Description
`any, all`	Return `True` if any (one or more values) or all non-NA values are "truthy"
`count`	Number of non-NA values
`cummin, cummax`	Cumulative minimum and maximum of non-NA values
`cumsum`	Cumulative sum of non-NA values
`cumprod`	Cumulative product of non-NA values
`first, last`	First and last non-NA values
`mean`	Mean of non-NA values
`median`	Arithmetic median of non-NA values

Resumindo Groupby

Function name	Description
`min, max`	Minimum and maximum of non-NA values
`nth`	Retrieve value that would appear at position `n` with the data in sorted order
`ohlc`	Compute four "open-high-low-close" statistics for time series-like data
`prod`	Product of non-NA values
`quantile`	Compute sample quantile
`rank`	Ordinal ranks of non-NA values, like calling `Series.rank`
`size`	Compute group sizes, returning result as a Series
`sum`	Sum of non-NA values
`std, var`	Sample standard deviation and variance

Agregação de Dados

Agregação condensa *arrays* em valores únicos através de operações como *min*, *max*, *média e soma*

Pandas permite que você crie a sua própria função de agregação e passe-a para um método chamado "*aggregate (agg)*"

- Permitem extrair métricas específicas que atendam às suas necessidades

O método "*agg*" otimiza o desempenho ao permitir múltiplas agregações em uma única passagem, economizando tempo e recursos

Agregação de Dados

Exemplo de agregação com função própria:

```
In [58]: df
Out[58]:
   key1  key2     data1     data2
0     a     1 -0.204708  0.281746
1     a     2  0.478943  0.769023
2  None     1 -0.519439  1.246435
3     b     2 -0.555730  1.007189
4     b     1  1.965781 -1.296221
5     a  <NA>  1.393406  0.274992
6  None     1  0.092908  0.228913

In [59]: grouped = df.groupby("key1")
```

Agregação de Dados

Exemplo de agregação com função própria:

```
In [58]: df
Out[58]:
   key1  key2     data1     data2
0     a     1 -0.204708  0.281746
1     a     2  0.478943  0.769023
2  None     1 -0.519439  1.246435
3     b     2 -0.555730  1.007189
4     b     1  1.965781 -1.296221
5     a  <NA>  1.393406  0.274992
6  None     1  0.092908  0.228913
```

```
In [59]: grouped = df.groupby("key1")
```

```
In [61]: def peak_to_peak(arr):
   ....:     return arr.max() - arr.min()

In [62]: grouped.agg(peak_to_peak)
Out[62]:
      key2     data1     data2
key1
a        1  1.598113  0.494031
b        1  2.521511  2.303410
```

Agregação de Dados

Exemplo de agregação com função própria:

```
In [58]: df
Out[58]:
   key1  key2      data1      data2
0     a     1 -0.204708   0.281746
1     a     2  0.478943   0.769023
2  None     1 -0.519439   1.246435
3     b     2 -0.555730   1.007189
4     b     1  1.965781  -1.296221
5     a  <NA>  1.393406   0.274992
6  None     1  0.092908   0.228913
```

```
In [59]: grouped = df.groupby("key1")
```

```
In [61]: def peak_to_peak(arr):
    ....:     return arr.max() - arr.min()
```

```
In [62]: grouped.agg(peak_to_peak)
Out[62]:
      key2     data1     data2
key1
a        1  1.598113  0.494031
b        1  2.521511  2.303410
```

Agregação de Dados

Exemplo de agregação com função própria:

```
In [58]: df
Out[58]:
   key1  key2     data1     data2
0     a     1 -0.204708  0.281746
1     a     2  0.478943  0.769023
2  None     1 -0.519439  1.246435
3     b     2 -0.555730  1.007189
4     b     1  1.965781 -1.296221
5     a  <NA>  1.393406  0.274992
6  None     1  0.092908  0.228913
```

```
In [59]: grouped = df.groupby("key1")
```

```
In [61]: def peak_to_peak(arr):
    ....:     return arr.max() - arr.min()
```

```
In [62]: grouped.agg(peak_to_peak)
Out[62]:
      key2     data1     data2
key1
a        1  1.598113  0.494031
b        1  2.521511  2.303410
```

Agregação de Dados

Exemplo de agregação com função própria:

```
In [58]: df
Out[58]:
   key1  key2     data1     data2
0     a     1 -0.204708  0.281746
1     a     2  0.478943  0.769023
2  None     1 -0.519439  1.246435
3     b     2 -0.555730  1.007189
4     b     1  1.965781 -1.296221
5     a  <NA>  1.393406  0.274992
6  None     1  0.092908  0.228913
```

```
In [61]: def peak_to_peak(arr):
   ....:     return arr.max() - arr.min()

In [62]: grouped.agg(peak_to_peak)
Out[62]:
      key2     data1     data2
key1
a        1  1.598113  0.494031
b        1  2.521511  2.303410
```

Também é possível realizar agregações com várias funções ao mesmo tempo

- Você pode usar funções do Pandas, Numpy, e/ou as criadas por você

Agregação de Dados

Exemplo de agregação com múltiplas funções:

```
In [67]: tips.head()
Out[67]:
   total_bill   tip smoker  day    time  size   tip_pct
0       16.99  1.01     No  Sun  Dinner     2  0.059447
1       10.34  1.66     No  Sun  Dinner     3  0.160542
2       21.01  3.50     No  Sun  Dinner     3  0.166587
3       23.68  3.31     No  Sun  Dinner     2  0.139780
4       24.59  3.61     No  Sun  Dinner     4  0.146808

In [68]: grouped = tips.groupby(["day", "smoker"])
In [69]: grouped_pct = grouped["tip_pct"]
```

Agregação de Dados

Exemplo de agregação com múltiplas funções:

```
In [67]: tips.head()
Out[67]:
   total_bill   tip smoker  day    time  size   tip_pct
0       16.99  1.01     No  Sun  Dinner     2  0.059447
1       10.34  1.66     No  Sun  Dinner     3  0.160542
2       21.01  3.50     No  Sun  Dinner     3  0.166587
3       23.68  3.31     No  Sun  Dinner     2  0.139780
4       24.59  3.61     No  Sun  Dinner     4  0.146808
```

```
In [68]: grouped = tips.groupby(["day", "smoker"])
In [69]: grouped_pct = grouped["tip_pct"]
```

Lista de funções aplicadas

```
In [71]: grouped_pct.agg(["mean", "std", peak_to_peak])
Out[71]:
                 mean       std  peak_to_peak
day  smoker
Fri  No     0.151650  0.028123      0.067349
     Yes    0.174783  0.051293      0.159925
Sat  No     0.158048  0.039767      0.235193
     Yes    0.147906  0.061375      0.290095
Sun  No     0.160113  0.042347      0.193226
```

Agregação de Dados

Personalize o nome da coluna atribuindo um nome específico a cada função, utilizando tuplas

```
In [72]: grouped_pct.agg([("average", "mean"), ("stdev", np.std)])
Out[72]:
                 average     stdev
day  smoker
Fri  No        0.151650  0.028123
     Yes       0.174783  0.051293
Sat  No        0.158048  0.039767
     Yes       0.147906  0.061375
Sun  No        0.160113  0.042347
     Yes       0.187250  0.154134
Thur No        0.160298  0.038774
     Yes       0.163863  0.039389
```

Agregação de Dados

É possível aplicar várias funções a várias colunas:

```
In [73]: functions = ["count", "mean", "max"]

In [74]: result = grouped[["tip_pct", "total_bill"]].agg(functions)

In [75]: result
Out[75]:
            tip_pct                         total_bill
              count      mean       max       count      mean    max
day  smoker
Fri  No           4  0.151650  0.187735           4  18.420000  22.75
     Yes         15  0.174783  0.263480          15  16.813333  40.17
Sat  No          45  0.158048  0.291990          45  19.661778  48.33
     Yes         42  0.147906  0.325733          42  21.276667  50.81
Sun  No          57  0.160113  0.252672          57  20.506667  48.17
     Yes         19  0.187250  0.710345          19  24.120000  45.35
Thur No          45  0.160298  0.266312          45  17.113111  41.19
     Yes         17  0.163863  0.241255          17  19.190588  43.11
```

Agregação de Dados

Cada coluna pode receber a aplicações de funções específicas

```
In [80]: grouped.agg({"tip_pct" : ["min", "max", "mean", "std"],
   ....:              "size" : "sum"})
Out[80]:
              tip_pct                                     size
                  min       max      mean       std      sum
day  smoker
Fri  No      0.120385  0.187735  0.151650  0.028123        9
     Yes     0.103555  0.263480  0.174783  0.051293       31
Sat  No      0.056797  0.291990  0.158048  0.039767      115
     Yes     0.035638  0.325733  0.147906  0.061375      104
Sun  No      0.059447  0.252672  0.160113  0.042347      167
     Yes     0.065660  0.710345  0.187250  0.154134       49
Thur No      0.072961  0.266312  0.160298  0.038774      112
     Yes     0.090014  0.241255  0.163863  0.039389       40
```

Agregação de Dados

Cada coluna pode receber a aplicações de funções específicas

```
In [80]: grouped.agg({"tip_pct" : ["min", "max", "mean", "std"],
   ....:              "size" : "sum"})
Out[80]:
                tip_pct                               size
                    min       max      mean       std  sum
day  smoker
Fri  No        0.120385  0.187735  0.151650  0.028123    9
     Yes       0.103555  0.263480  0.174783  0.051293   31
Sat  No        0.056797  0.291990  0.158048  0.039767  115
     Yes       0.035638  0.325733  0.147906  0.061375  104
Sun  No        0.059447  0.252672  0.160113  0.042347  167
     Yes       0.065660  0.710345  0.187250  0.154134   49
Thur No        0.072961  0.266312  0.160298  0.038774  112
     Yes       0.090014  0.241255  0.163863  0.039389   40
```

Agregação de Dados

Cada coluna pode receber a aplicações de funções específicas

```
In [80]: grouped.agg({"tip_pct" : ["min", "max", "mean", "std"],
   ....:              "size" : "sum"})
Out[80]:
              tip_pct                                     size
                  min       max      mean       std      sum
day  smoker
Fri  No      0.120385  0.187735  0.151650  0.028123        9
     Yes     0.103555  0.263480  0.174783  0.051293       31
Sat  No      0.056797  0.291990  0.158048  0.039767      115
     Yes     0.035638  0.325733  0.147906  0.061375      104
Sun  No      0.059447  0.252672  0.160113  0.042347      167
     Yes     0.065660  0.710345  0.187250  0.154134       49
Thur No      0.072961  0.266312  0.160298  0.038774      112
     Yes     0.090014  0.241255  0.163863  0.039389       40
```

Agregação de Dados

Para uma agregação sem índice, utilize o argumento “as_index=False”

```
In [81]: tips.groupby(["day", "smoker"], as_index=False).mean()
Out[81]:
    day smoker  total_bill       tip      size   tip_pct
0   Fri     No   18.420000  2.812500  2.250000  0.151650
1   Fri    Yes   16.813333  2.714000  2.066667  0.174783
2   Sat     No   19.661778  3.102889  2.555556  0.158048
3   Sat    Yes   21.276667  2.875476  2.476190  0.147906
4   Sun     No   20.506667  3.167895  2.929825  0.160113
5   Sun    Yes   24.120000  3.516842  2.578947  0.187250
6  Thur     No   17.113111  2.673778  2.488889  0.160298
7  Thur    Yes   19.190588  3.030000  2.352941  0.163863
```

Generalizando Agrupamentos com “Apply”

O método de GroupBy mais genérico é o “Apply”, capaz de dividir-aplicar-combinar o objeto que está sendo manipulado

```
In [82]: def top(df, n=5, column="tip_pct"):
   ....:     return df.sort_values(column, ascending=False)[:n]

In [83]: top(tips, n=6)
Out[83]:
     total_bill   tip smoker  day    time  size   tip_pct
172        7.25  5.15    Yes  Sun  Dinner     2  0.710345
178        9.60  4.00    Yes  Sun  Dinner     2  0.416667
67         3.07  1.00    Yes  Sat  Dinner     1  0.325733
232       11.61  3.39     No  Sat  Dinner     2  0.291990
183       23.17  6.50    Yes  Sun  Dinner     4  0.280535
109       14.31  4.00    Yes  Sat  Dinner     2  0.279525
```

Generalizando Agrupamentos com “Apply”

Aplicando o “apply” junto ao “groupby”

```
In [84]: tips.groupby("smoker").apply(top)
Out[84]:
            total_bill   tip smoker   day    time  size   tip_pct
smoker
No     232       11.61  3.39     No   Sat  Dinner     2  0.291990
       149        7.51  2.00     No  Thur   Lunch     2  0.266312
       51        10.29  2.60     No   Sun  Dinner     2  0.252672
       185       20.69  5.00     No   Sun  Dinner     5  0.241663
       88        24.71  5.85     No  Thur   Lunch     2  0.236746
Yes    172        7.25  5.15    Yes   Sun  Dinner     2  0.710345
       178        9.60  4.00    Yes   Sun  Dinner     2  0.416667
       67         3.07  1.00    Yes   Sat  Dinner     1  0.325733
       183       23.17  6.50    Yes   Sun  Dinner     4  0.280535
       109       14.31  4.00    Yes   Sat  Dinner     2  0.279525
```

Supressão das Chaves de Grupos

Para suprimir os índices, utilize group_keys = False

```
In [89]: tips.groupby("smoker", group_keys=False).apply(top)
Out[89]:
     total_bill   tip smoker   day    time  size   tip_pct
232       11.61  3.39     No   Sat  Dinner     2  0.291990
149        7.51  2.00     No  Thur   Lunch     2  0.266312
51        10.29  2.60     No   Sun  Dinner     2  0.252672
185       20.69  5.00     No   Sun  Dinner     5  0.241663
88        24.71  5.85     No  Thur   Lunch     2  0.236746
172        7.25  5.15    Yes   Sun  Dinner     2  0.710345
178        9.60  4.00    Yes   Sun  Dinner     2  0.416667
67         3.07  1.00    Yes   Sat  Dinner     1  0.325733
183       23.17  6.50    Yes   Sun  Dinner     4  0.280535
109       14.31  4.00    Yes   Sat  Dinner     2  0.279525
```

Séries temporais

Dados de séries temporais são cruciais em diversos campos da ciência

Este dados podem ter uma frequência **regular** (dias, minutos, etc) ou **irregulares**, sem uma unidade fixa de tempo entre as unidades

A forma como se marca e refere aos dados de séries temporais varia conforme a aplicação, podendo ser:

- ***Timestamps***: Instantes específicos no tempo
- **Períodos fixos:** Como todo o mês de janeiro de 2017 ou o ano inteiro de 2020
- **Intervalos de tempo:** Indicados por um timestamp de início e fim. Períodos podem ser vistos como casos especiais de intervalos
- **Tempo de experimento ou tempo decorrido:** Cada marcação de tempo é medida em relação a um momento de início específico, começando do zero

Tipos de dados e Ferramentas de Data/Hora

A biblioteca padrão do Python inclui tipos de dados para datas e horários, além de funcionalidades relacionadas à calendários

Alguns módulos úteis em Python:

- **“datetime”**: Fornece classes para manipular datas e horas, permitindo operações como a criação, manipulação e formatação de objetos de data e hora
- **“time”**: Oferece funções relacionadas ao tempo, incluindo funcionalidades para trabalhar com medições e a conversão entre diferentes formatos de tempo
- **“calendar”**: Oferece funções para gerar calendários e determinar dias da semana de datas específicas."

Tipo “datetime”

O objeto datetime do módulo datetime armazena tanto a data quanto a hora até o nível de milisegundos

```
In [14]: from datetime import datetime

In [15]: now = datetime.now()

In [16]: now
Out[16]: datetime.datetime(2022, 8, 12, 14, 9, 11, 337033)

In [17]: now.year, now.month, now.day
Out[17]: (2022, 8, 12)
```

Para acessar o tempo use, respectivamente: “now.hour”, “now.minute”, “now.second”, “now.millisecond”

Tipo “timedelta”

O tipo “timedelta” representa a diferença de tempo entre dois objetos “datetime”

```
In [18]: delta = datetime(2011, 1, 7) - datetime(2008, 6, 24, 8, 15)

In [19]: delta
Out[19]: datetime.timedelta(days=926, seconds=56700)

In [20]: delta.days
Out[20]: 926

In [21]: delta.seconds
Out[21]: 56700
```

Tipo “timedelta”

É possível realizar operações com um “timedelta” criando um novo objeto deslocado

```
In [22]: from datetime import timedelta

In [23]: start = datetime(2011, 1, 7)

In [24]: start + timedelta(12)
Out[24]: datetime.datetime(2011, 1, 19, 0, 0)

In [25]: start - 2 * timedelta(12)
Out[25]: datetime.datetime(2010, 12, 14, 0, 0)
```

Na ausência de especificação do tipo de unidade, o número fornecido é considerado como dias

Tipos de Dados do Módulo “datetime”

Type	Description
`date`	Store calendar date (year, month, day) using the Gregorian calendar
`time`	Store time of day as hours, minutes, seconds, and microseconds
`datetime`	Store both date and time
`timedelta`	The difference between two `datetime` values (as days, seconds, and microseconds)
`tzinfo`	Base type for storing time zone information

Conversões entre string e “datetime”

Frequentemente, temos que formatar objetos datetime Timestamp do pandas para strings e vice-versa

Obtendo uma string a partir de um “datetime” com o método “strftime”

```
In [26]: stamp = datetime(2011, 1, 3)

In [27]: str(stamp)
Out[27]: '2011-01-03 00:00:00'

In [28]: stamp.strftime("%Y-%m-%d")
Out[28]: '2011-01-03'
```

Conversões entre string e “datetime”

Obtendo um “datetime” a partir de uma string com o método “strftime”

```
In [29]: value = "2011-01-03"

In [30]: datetime.strptime(value, "%Y-%m-%d")
Out[30]: datetime.datetime(2011, 1, 3, 0, 0)

In [31]: datestrs = ["7/6/2011", "8/6/2011"]

In [32]: [datetime.strptime(x, "%m/%d/%Y") for x in datestrs]
Out[32]:
[datetime.datetime(2011, 7, 6, 0, 0),
 datetime.datetime(2011, 8, 6, 0, 0)]
```

Conversões entre string e “datetime”: Formatos

Type	Description
`%Y`	Four-digit year
`%y`	Two-digit year
`%m`	Two-digit month [01, 12]
`%d`	Two-digit day [01, 31]
`%H`	Hour (24-hour clock) [00, 23]
`%I`	Hour (12-hour clock) [01, 12]
`%M`	Two-digit minute [00, 59]
`%S`	Second [00, 61] (seconds 60, 61 account for leap seconds)
`%f`	Microsecond as an integer, zero-padded (from 000000 to 999999)

Conversões entre string e “datetime”: Formatos

Type	Description
`%j`	Day of the year as a zero-padded integer (from 001 to 336)
`%w`	Weekday as an integer [0 (Sunday), 6]
`%u`	Weekday as an integer starting from 1, where 1 is Monday.
`%U`	Week number of the year [00, 53]; Sunday is considered the first day of the week, and days before the first Sunday of the year are “week 0”
`%W`	Week number of the year [00, 53]; Monday is considered the first day of the week, and days before the first Monday of the year are “week 0”
`%z`	UTC time zone offset as `+HHMM` or `-HHMM`; empty if time zone naive
`%Z`	Time zone name as a string, or empty string if no time zone
`%F`	Shortcut for `%Y-%m-%d` (e.g., `2012-4-18`)
`%D`	Shortcut for `%m/%d/%y` (e.g., `04/18/12`)

Conversões entre string e “datetime”

Pandas faz o mapeamento de strings para datetime de forma natural a partir de vários formatos distintos

Caso você tenha algum problema, converta a string para o formato ISSO 8601. Só então, use a função “to_datetime”

```
In [33]: datestrs = ["2011-07-06 12:00:00", "2011-08-06 00:00:00"]

In [34]: pd.to_datetime(datestrs)
Out[34]: DatetimeIndex(['2011-07-06 12:00:00', '2011-08-06 00:00:00'], dtype='dat
etime64[ns]', freq=None)
```

Valores faltantes são tratados como NaT “Not a Time” e também podem ser checados com “isna”

Indexação, seleção e fatiamento com “datetime”

Series e DataFrames podem ser indexados com objetos “datetime”

```
In [40]: ts = pd.Series(np.random.standard_normal(6), index=dates)

In [41]: ts
Out[41]:
2011-01-02   -0.204708
2011-01-05    0.478943
2011-01-07   -0.519439
2011-01-08   -0.555730
2011-01-10    1.965781
2011-01-12    1.393406
dtype: float64

In [42]: ts.index
Out[42]:
DatetimeIndex(['2011-01-02', '2011-01-05', '2011-01-07', '2011-01-08',
               '2011-01-10', '2011-01-12'],
              dtype='datetime64[ns]', freq=None)
```

Indexação, seleção e fatiamento com “datetime”

Series e DataFrames podem ser indexados com objetos “datetime”

```
In [40]: ts = pd.Series(np.random.standard_normal(6), index=dates)

In [41]: ts
Out[41]:
2011-01-02   -0.204708
2011-01-05    0.478943
2011-01-07   -0.519439
2011-01-08   -0.555730
2011-01-10    1.965781
2011-01-12    1.393406
dtype: float64
```

```
In [43]: ts + ts[::2]
Out[43]:
2011-01-02   -0.409415
2011-01-05         NaN
2011-01-07   -1.038877
2011-01-08         NaN
2011-01-10    3.931561
2011-01-12         NaN
dtype: float64
```

Como antes, as operações são alinhadas de acordo com os índices

Indexação, seleção e fatiamento com “datetime”

A indexação pode ser feita normalmente com base no índice ou por uma (sub)-string que seja interpretável como uma data

```
In [47]: stamp = ts.index[2]

In [48]: ts[stamp]
Out[48]: -0.5194387150567381
```

Seleção pela posição do índice

```
In [49]: ts["2011-01-10"]
Out[49]: 1.9657805725027142
```

Seleção pela data exata

Indexação, seleção e fatiamento com “datetime”

A indexação pode ser feita normalmente com base no índice ou por uma (sub)-string que seja interpretável como uma data

```
In [51]: longer_ts
Out[51]:
2000-01-01    0.092908
2000-01-02    0.281746
2000-01-03    0.769023
2000-01-04    1.246435
2000-01-05    1.007189
                ...
2002-09-22    0.930944
2002-09-23   -0.811676
2002-09-24   -1.830156
2002-09-25   -0.138730
2002-09-26    0.334088
Freq: D, Length: 1000, dtype: float64
```

```
In [52]: longer_ts["2001"]
Out[52]:
2001-01-01    1.599534
2001-01-02    0.474071
2001-01-03    0.151326
2001-01-04   -0.542173
2001-01-05   -0.475496
                ...
2001-12-27    0.057874
2001-12-28   -0.433739
2001-12-29    0.092698
2001-12-30   -1.397820
```

Seleção pelo ano

Indexação, seleção e fatiamento com “datetime”

A indexação pode ser feita normalmente com base no índice ou por uma (sub)-string que seja interpretável como uma data

```
In [51]: longer_ts
Out[51]:
2000-01-01    0.092908
2000-01-02    0.281746
2000-01-03    0.769023
2000-01-04    1.246435
2000-01-05    1.007189
                ...
2002-09-22    0.930944
2002-09-23   -0.811676
2002-09-24   -1.830156
2002-09-25   -0.138730
2002-09-26    0.334088
Freq: D, Length: 1000, dtype: float64
```

```
In [52]: longer_ts["2001"]
Out[52]:
2001-01-01    1.599534
2001-01-02    0.474071
2001-01-03    0.151326
2001-01-04   -0.542173
2001-01-05   -0.475496
                ...
2001-12-27    0.057874
2001-12-28   -0.433739
2001-12-29    0.092698
2001-12-30   -1.397820
```

```
In [53]: longer_ts["2001-05"]
Out[53]:
2001-05-01   -0.622547
2001-05-02    0.936289
2001-05-03    0.750018
2001-05-04   -0.056715
2001-05-05    2.300675
                ...
2001-05-27    0.235477
2001-05-28    0.111835
2001-05-29   -1.251504
2001-05-30   -2.949343
2001-05-31    0.634634
Freq: D, Length: 31, dtype: float64
```

Seleção pelo ano e mês

Geração de Intervalos de Datas

Para gerar intervalos de datas, pode-se utilizar a função “date_range” do pandas. Se o intervalo não é passado, ele assume como sendo dia “D”

```
In [74]: index = pd.date_range("2012-04-01", "2012-06-01")

In [75]: index
Out[75]:
DatetimeIndex(['2012-04-01', '2012-04-02', '2012-04-03', '2012-04-04',
               '2012-04-05', '2012-04-06', '2012-04-07', '2012-04-08',
               '2012-04-09', '2012-04-10', '2012-04-11', '2012-04-12',
               '2012-04-13', '2012-04-14', '2012-04-15', '2012-04-16',
               '2012-04-17', '2012-04-18', '2012-04-19', '2012-04-20',
               '2012-04-21', '2012-04-22', '2012-04-23', '2012-04-24',
               '2012-04-25', '2012-04-26', '2012-04-27', '2012-04-28',
               '2012-04-29', '2012-04-30', '2012-05-01', '2012-05-02',
               '2012-05-03', '2012-05-04', '2012-05-05', '2012-05-06',
               '2012-05-07', '2012-05-08', '2012-05-09', '2012-05-10',
               '2012-05-11', '2012-05-12', '2012-05-13', '2012-05-14',
               '2012-05-15', '2012-05-16', '2012-05-17', '2012-05-18',
               '2012-05-19', '2012-05-20', '2012-05-21', '2012-05-22',
               '2012-05-23', '2012-05-24', '2012-05-25', '2012-05-26',
               '2012-05-27', '2012-05-28', '2012-05-29', '2012-05-30',
               '2012-05-31', '2012-06-01'],
              dtype='datetime64[ns]', freq='D')
```

Algumas Opções de Frequência

Alias	Offset type	Description
`D`	`Day`	Calendar daily
`B`	`BusinessDay`	Business daily
`H`	`Hour`	Hourly
`T` or `min`	`Minute`	Once a minute
`S`	`Second`	Once a second
`L` or `ms`	`Milli`	Millisecond (1/1,000 of 1 second)
`U`	`Micro`	Microsecond (1/1,000,000 of 1 second)
`M`	`MonthEnd`	Last calendar day of month
`BM`	`BusinessMonthEnd`	Last business day (weekday) of month
`MS`	`MonthBegin`	First calendar day of month
`BMS`	`BusinessMonthBegin`	First weekday of month

Veja outras aqui!

Roteiro

Introdução a Python

Estrutura de Dados

Aplicações e Desenvolvimento de Funções

Módulo Numpy

Módulo Pandas

Plotagem e Visualização

Plotagem e Visualização em Python

Criar visualizações informativas é uma das tarefas mais importantes na análise de dados

- Pode ser parte do processo exploratório, ajudando a entender melhor os dados e identificar valores discrepantes

Python possui muitos módulos adicionais para fazer visualizações estáticas ou dinâmicas

Matplotlib é uma das mais populares, projetada para criar gráficos e figuras adequados

- É a base para várias outros, como a *Seaborn*

Plotagem e Visualização em Python

O comando “%matplotlib inline” é utilizado em notebooks Jupyter para exibir gráficos diretamente na página do notebook

- Facilita a integração das visualizações geradas com matplotlib ao fluxo da análise de dados

Foco em um visão geral e rápida do Matplotlib

Para importar o matplotlib, a seguinte convenção é adotada

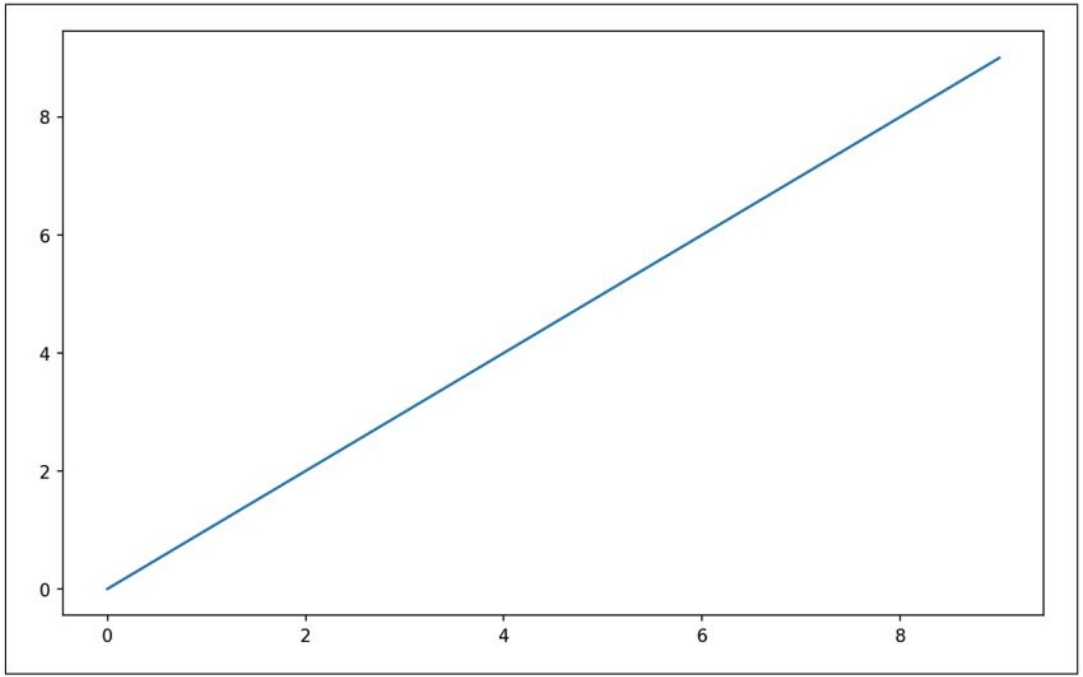
```
In [13]: import matplotlib.pyplot as plt
```

Gráfico de Linha Simples

```
In [14]: data = np.arange(10)

In [15]: data
Out[15]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [16]: plt.plot(data)
```



Plots e Subplots

As plotagens do “matplotlib” residem dentro de um objeto Figure

```
In [17]: fig = plt.figure()
```

“plt.figure” tem várias opções de argumentos. Uma importante é o figsize, que garante o tamanho e proporção da figura a ser salva

- **Dica:** Use algo em torno de 4x3 para bom ajuste no textos
    - Figuras nunca devem ter eixos/ticks maiores do que a fonte do texto (E isso vai te ajudar)
    - Deixe o tamanho padrão e ajuste apenas o tamanho da figura

Plots e Subplots

Para criar uma ou mais subplotagens (subplots) utilize “add_suplot”

```
In [18]: ax1 = fig.add_subplot(2, 2, 1)
```

Figura com dimensão 2x2 (logo, até quatro plotagens no total) e que a primeira está sendo selecionada

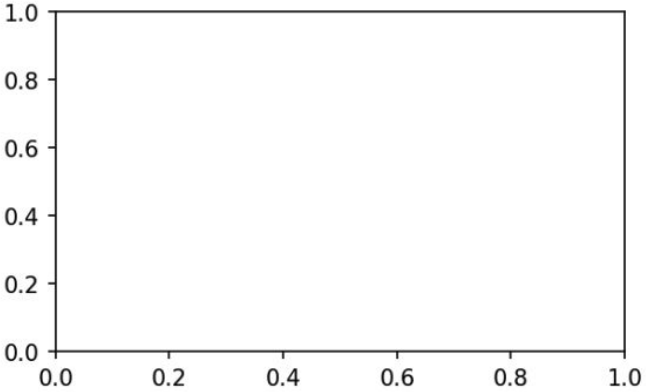
- OBS: A numeração, começa a partir de 1

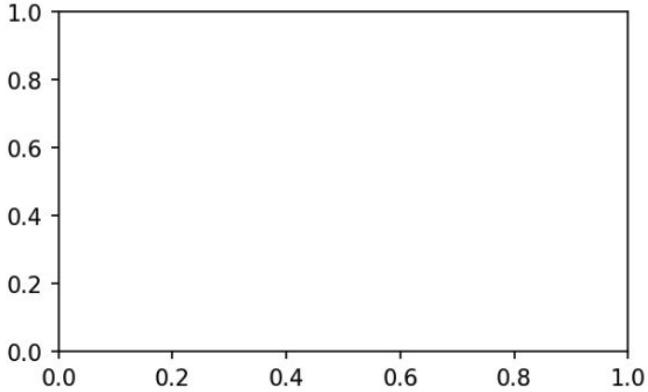
Você poderia pegar as outras duas visualizações

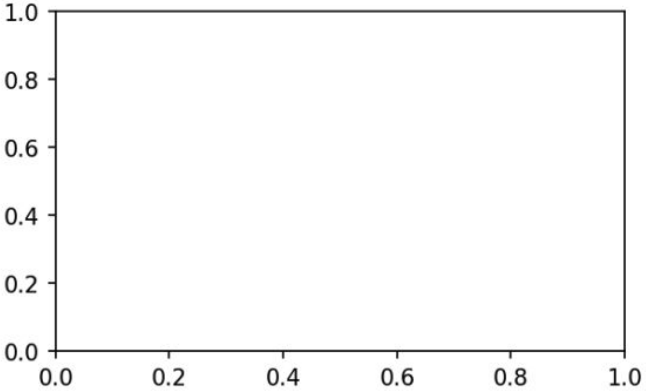
```
In [19]: ax2 = fig.add_subplot(2, 2, 2)
```

```
In [20]: ax3 = fig.add_subplot(2, 2, 3)
```

Plots e Subplots







Plots e Subplots

Por convenção e para evitar problemas de redefinição, plotagens devem estabelecidas em uma única célula no Jupyter

```
fig = plt.figure()
ax1 = fig.add_subplot(2, 2, 1)
ax2 = fig.add_subplot(2, 2, 2)
ax3 = fig.add_subplot(2, 2, 3)
```

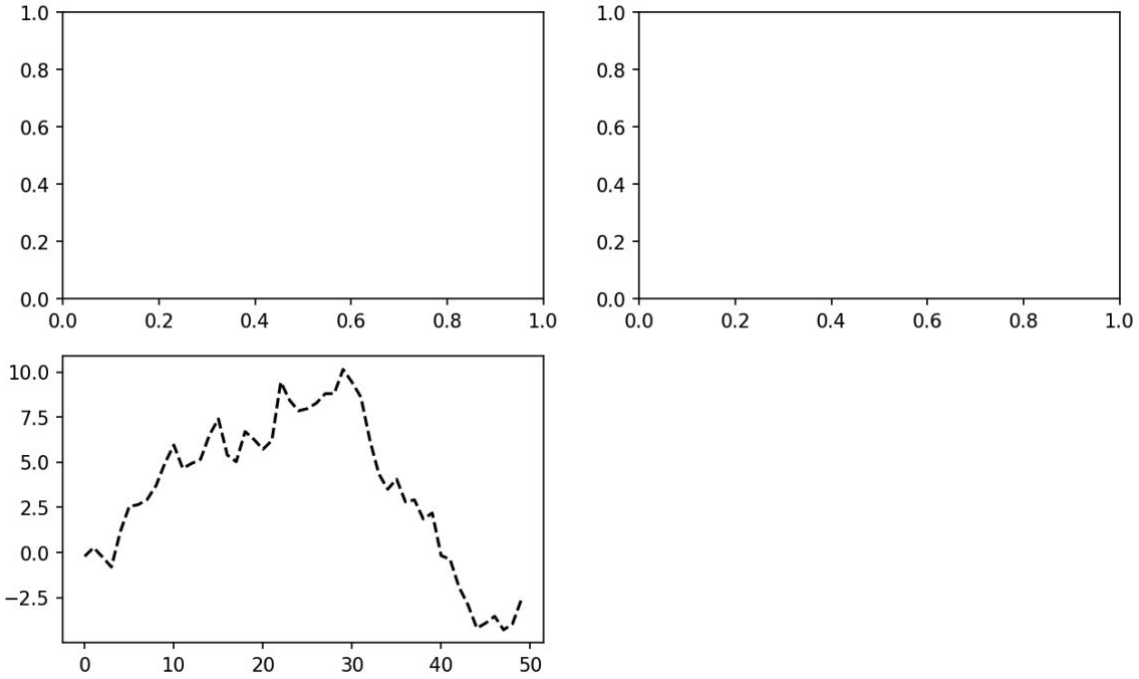
Cada objeto de eixo (ax) tem vários métodos que permitem criar diferentes tipos de plotagem

- Você pode consultar todos neste link
- Alguns que valem a pena olhar com calma: plot, scatter, hist, ecdf, bar ...

Plots e Subplots

Exemplo de um gráfico de linha:

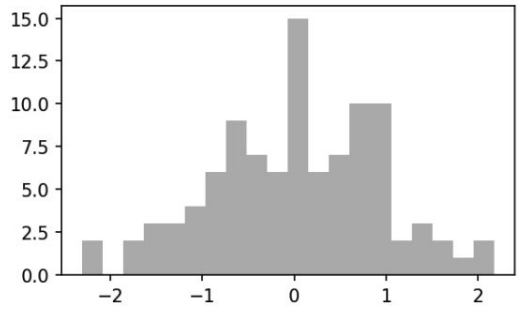
```
In [21]: ax3.plot(np.random.standard_normal(50).cumsum(), color="black",
    ....:             linestyle="dashed")
```

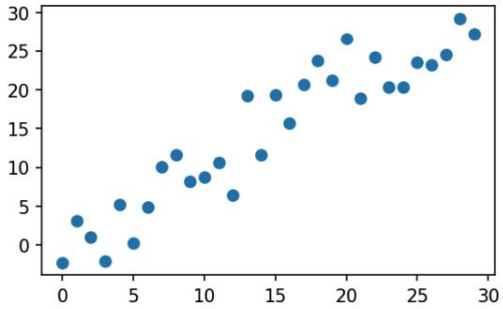


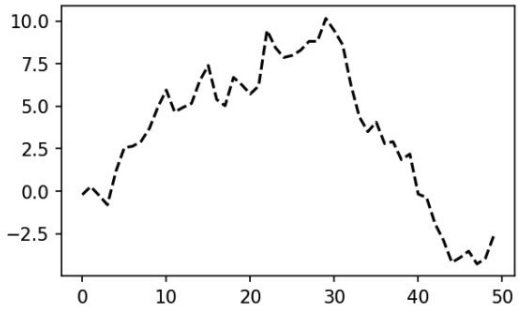
Plots e Subplots

Exemplo de um histograma e de um scatter plot:

```
In [22]: ax1.hist(np.random.standard_normal(100), bins=20, color="black", alpha=0
.3);
In [23]: ax2.scatter(np.arange(30), np.arange(30) + 3 * np.random.standard_normal
(30));
```



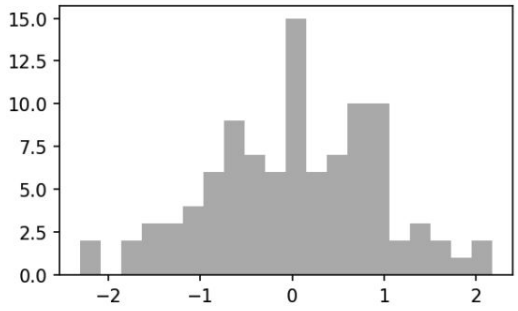


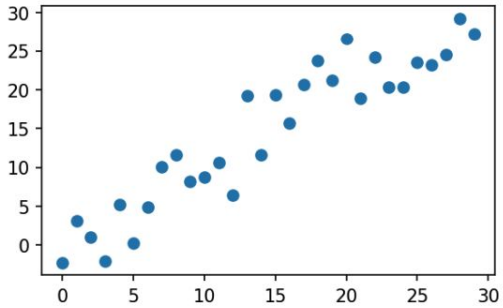


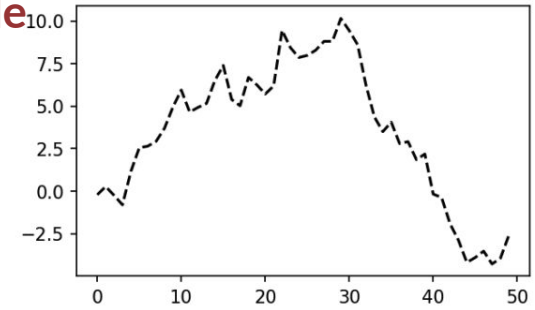
Plots e Subplots

Exemplo de um histograma e de um scatter plot:

```
In [22]: ax1.hist(np.random.standard_normal(100), bins=20, color="black", alpha=0
.3);
In [23]: ax2.scatter(np.arange(30), np.arange(30) + 3 * np.random.standard_normal
(30));
```







";" suprime algumas mensagens acerca de referências dos objetos

Plots e Subplots

O método “subplots” do Matplotlib simplifica a criação de subplots, gerando uma nova figura e retornando um array de NumPy

- O array contém os objetos de subplot

```
In [25]: fig, axes = plt.subplots(2, 3)

In [26]: axes
Out[26]:
array([[<AxesSubplot:>, <AxesSubplot:>, <AxesSubplot:>],
       [<AxesSubplot:>, <AxesSubplot:>, <AxesSubplot:>]], dtype=object)
```

Argumentos do Método “Subplots”

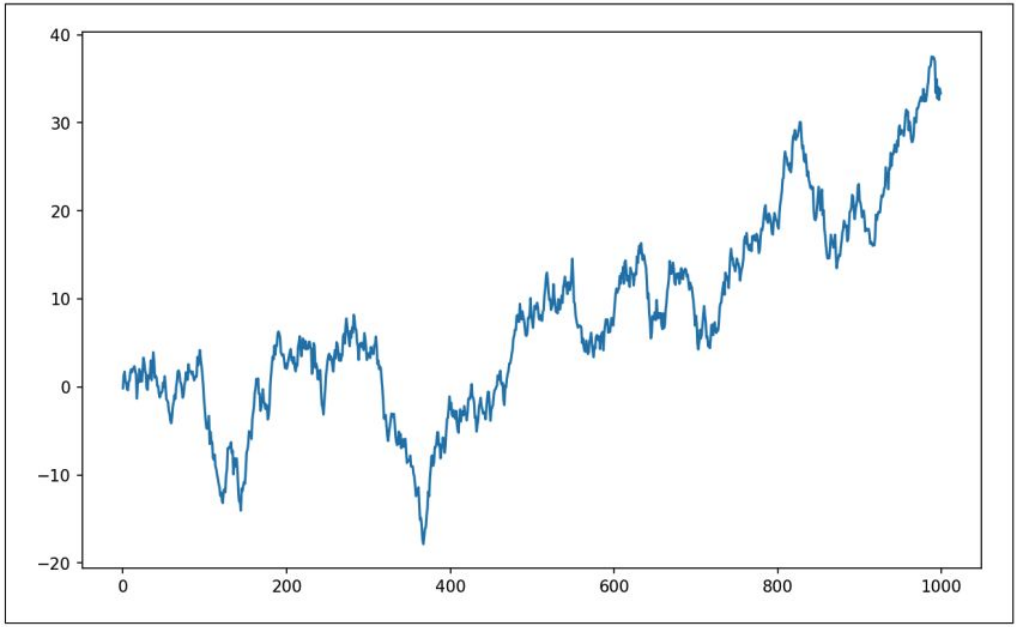
Argument	Description
`nrows`	Number of rows of subplots
`ncols`	Number of columns of subplots
`sharex`	All subplots should use the same x-axis ticks (adjusting the `xlim` will affect all subplots)
`sharey`	All subplots should use the same y-axis ticks (adjusting the `ylim` will affect all subplots)
`subplot_kw`	Dictionary of keywords passed to `add_subplot` call used to create each subplot
`**fig_kw`	Additional keywords to `subplots` are used when creating the figure, such as `plt.subplots(2, 2, figsize=(8, 6))`

Marcações, Rótulos e Legenda

Propriedades de figuras no matplotlib, como limites e rótulos dos eixos, são ajustadas pelos métodos dos objetos de eixo

```
In [40]: fig, ax = plt.subplots()

In [41]: ax.plot(np.random.standard_normal(1000).cumsum());
```



Marcações, Rótulos e Legenda

Alteração das marcações dos eixos x:

```
In [42]: ticks = ax.set_xticks([0, 250, 500, 750, 1000])
```

Marcações, Rótulos e Legenda

Alteração das marcações dos eixos x:

```
In [42]: ticks = ax.set_xticks([0, 250, 500, 750, 1000])
```

Alteração do nome das marcações e ângulo:

```
In [43]: labels = ax.set_xticklabels(["one", "two", "three", "four", "five"],
    ....:                            rotation=30, fontsize=8)
```

Marcações, Rótulos e Legenda

Alteração das marcações dos eixos x:

```
In [42]: ticks = ax.set_xticks([0, 250, 500, 750, 1000])
```

Alteração do nome das marcações e ângulo:

```
In [43]: labels = ax.set_xticklabels(["one", "two", "three", "four", "five"],
   ....:                             rotation=30, fontsize=8)
```

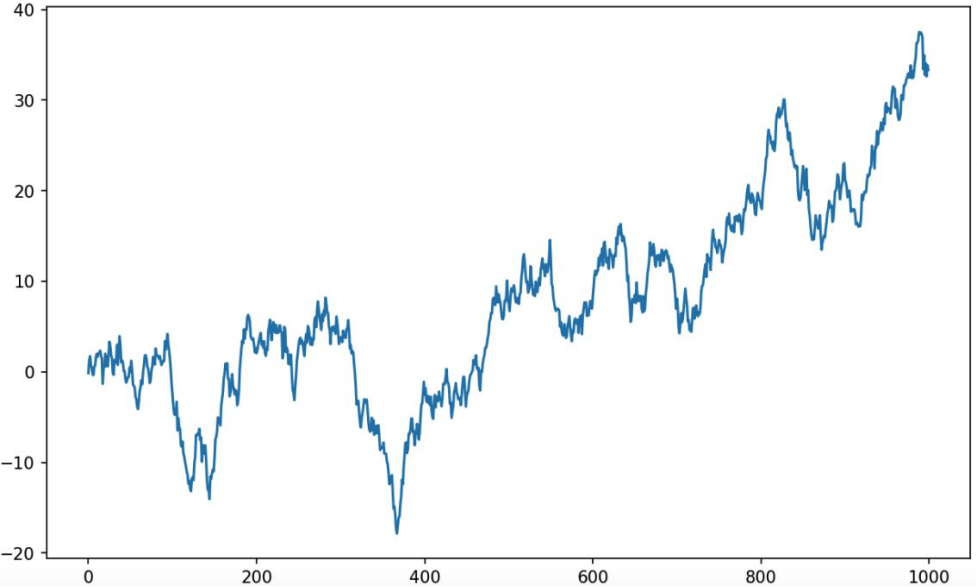
Alteração do rótulo do eixo x e do título:

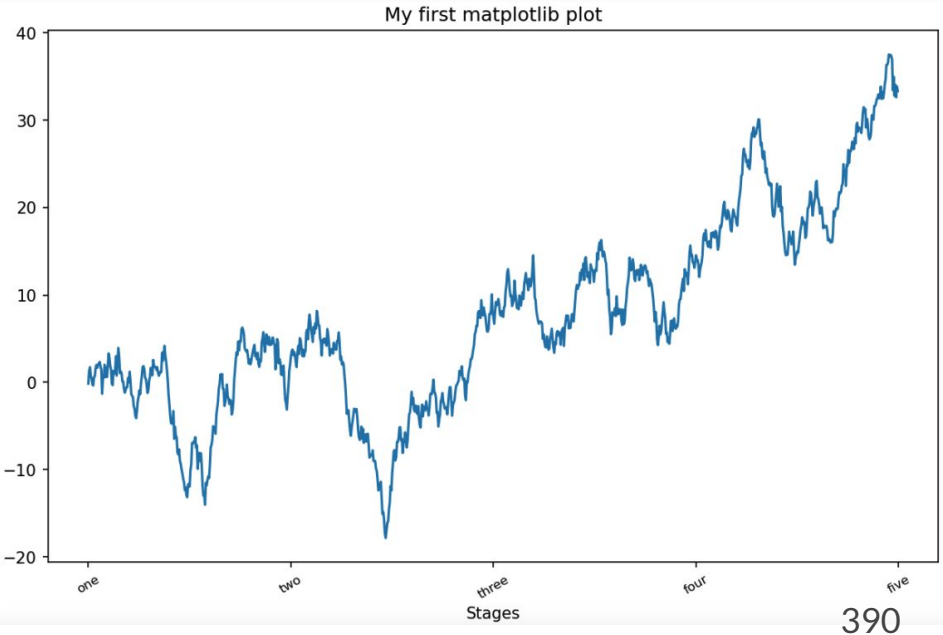
```
In [44]: ax.set_xlabel("Stages")
Out[44]: Text(0.5, 6.666666666666652, 'Stages')

In [45]: ax.set_title("My first matplotlib plot")
```

Marcações, Rótulos e Legenda

Resultado





Legendas

Para adicionar uma legenda em uma figura no Matplotlib, passe o argumento “label” ao adicionar cada parte da plotagem

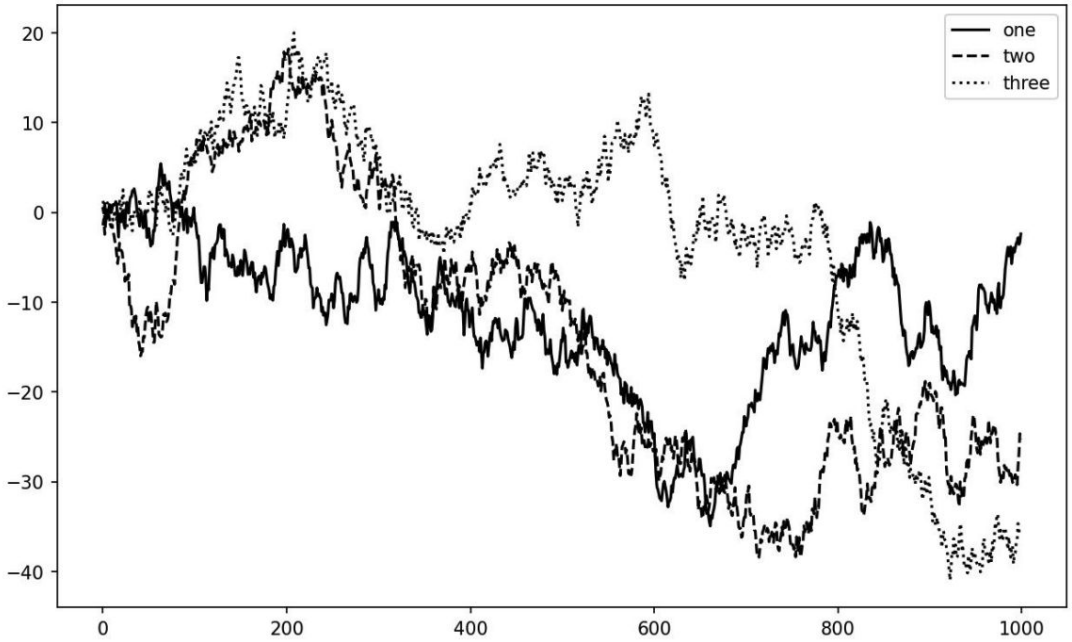
```
In [46]: fig, ax = plt.subplots()

In [47]: ax.plot(np.random.randn(1000).cumsum(), color="black", label="one");
In [48]: ax.plot(np.random.randn(1000).cumsum(), color="black", linestyle="dashed
",
   ....:          label="two");
In [49]: ax.plot(np.random.randn(1000).cumsum(), color="black", linestyle="dotted
",
   ....:          label="three");
```

Marcações, Rótulos e Legenda

Com isso, basta chamar o método “legend”:

```
In [50]: ax.legend()
```



O método “legend” possui vários argumentos importantes (Ver aqui)

Salvando Plotagens

Para salvar uma figura gerada, utilize o método “savefig” do objeto da figura (“fig”)

O tipo de arquivo é inferido pela extensão. Por exemplo, para salvar uma figura em PDF use

```
fig.savefig("figpath.png", dpi=400)
```

Existem vários parâmetros, com o número de pontos por polegada DPI (Pontos por polegada). Consulte aqui!

- **Dica**: Sempre chame o método “tight_layout” antes de salvar a figura!
    - Fornece um melhor ajuste para os espaçamentos e bordas

Resumindo

Matplotlib possui muitos recursos, mas é uma ferramenta de baixo nível e que exige a interação com todos os recursos de uma figura

Existem várias formas de realizar plotagens em Python em mais alto nível do que o Matplotlib

- **Pandas:** Oferece um conjunto de visualizações padrões (barras, setores, histogramas, etc. Link!
- **Seaborn:** Alto nível e bastante completa, oferecendo várias plotagens desde as mais básicas até as mais modernas de forma elegante. Link!

Exploraremos algumas opções durante o curso!

Dúvidas?

Créditos

Este material é baseado nos livros textos da disciplina em questão e foi desenvolvido pelo professor Carlos Henrique Gomes Ferreira. Além disso, ele está licenciado sob uma **Licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0)**. Versão 1.0. 02/2024.